# Certifying behaviour rates of language-model policies: what holds, what a label buys, and what does not carry

Draft of 2026-10-19, not formatted for any venue

**Abstract**

A developer releasing a language model wants to say something like "this model gives a harmful answer to at most 5% of requests", and to be wrong about it rarely. We call the frequency of an unwanted behaviour (a harmful answer, a needless refusal, a successful attack) its *breach rate*, and a tested statement that it is below a limit a *certificate*. A Seldonian algorithm (Thomas et al., 2019) returns a trained model only if such a certificate passes on held-out data. We test when the certificate can be trusted, and how to get it from fewer labels, where the true breach rate is known: simulated models, resampled real responses and published benchmark runs.

Standard exact methods can be trusted on data sampled as they assume: promised to be wrong in at most 5% of repeated trials, the standard one for yes-or-no labels was wrong in at most 5.3% of ours, a difference within the noise of the check. Two published shortcuts that let an automatic judge stand in for some labels are guaranteed only for large samples; at the sizes of a safety test, in our implementation, they were wrong more often than promised in most of our tests with a judge's score: in up to 24% of trials, and in 90% in one setting where an allocation rule published with one of them leaves some groups with only the 2 labels we set as a minimum (and in 59% with a minimum of 10).

A simpler design makes each label go further. Before training, sample the starting model 8 times on every prompt, sort the prompts into 8 equal groups by how often it breaches, and label the same number from each group. On new prompts, with predictions registered in advance, a certificate from this grouped sample with an approximate formula was wrong no more often than promised at breach rates of 12%, 46% and 82%, and, for the prompts sampled, at 19% on a second model family (for all prompts of that kind it ran over in one of eight checks there) and at 26% and 58% on that family after further training moved it. The grouped sample did the work of a random sample 1.5 to 1.7 times its size at 12% and 2.1 to 2.6 times at 46%, where much of the gain comes from mixing two prompt sources, and 1.5 to 1.8 times on the second family; once training had reshuffled which prompts it refused (at 58%), the grouping saved almost nothing. The gain is smaller, 1.3 to 1.5 at 12% and 46%, when the certificate must cover all prompts of that kind and not only those sampled. At 82% it gained little or lost, at 98% it lost, and for rare behaviours (under 4%) it gained at most 1.2 times; there exact methods remain the ones to use. For a costly human label grouped by a model's cheap judged responses the gain was about 1.2, which, for 100 human labels on 250 prompts, pays only where a human label costs more than about 120 judged responses. Grouping helped the exact methods little: of the two exact grouped certificates we tried, one never passed more often than an exact certificate from a random sample of the same size, and the other did so in about half of the real-response tests at rates from 5% to 95% and in none of those on the costly human label. The approximate formula has no guarantee: on simulated data it was wrong too often. Of 72 predictions registered in advance, 53 held, 18 failed and one could not be tested; leaving out the five that say only that an exact method is exact and the ten written after a dry run on earlier responses, 39 of 57 held. The models we trained are small, the new prompts come from one benchmark, the labels in these tests come from an automatic judge apart from published human ones, and only one training run, on one small model, moved a model far from its starting behaviour.

Three things do not carry over. A judge's measured error rates did not hold on different prompts. After training aimed at the judged behaviour, the error rates of a judge built from a written rubric did not hold in any of three training runs of one set-up; those of a second, purpose-trained judge did. Methods that treat test cases as independent fail on benchmarks that pair tasks with attacks: on AgentDojo's published runs the usual certificate is wrong too often for 27 of 28 agent configurations, and with the methods that allow for the pairing and kept their promise in our checks, on simulated copies of the benchmark, with the benchmark's attacks taken as fixed, at most one of the 28 certifies an attack success rate under 5%. An exact method agrees only narrowly and only for the rate averaged over user tasks; its version for the rate over task pairs certifies none. And a labelling sheet that deliberately over-samples some responses, read as a random sample with a judge's scores, gave certificates that were wrong in up to 99% of trials.

Our certificates of trained models rest on an automatic judge's labels, with one exception: from one person's labels of 220 responses, an exact method certifies that a trained model refuses outright at most 20.1% of its responses to a pool of 490 benign prompts, but nothing certifies, in these labels, the claim it was trained to meet, that its refusal rate rose by no more than 2 points. A certificate of that claim in human labels is designed and costed, not run: if training left the refusal rate unchanged, about 420 labelled prompt pairs would give an even chance of certifying it with an approximate limit; the exact limit is about twice as wide.

# 1 Introduction

Reports on language-model policies state rates: how often a model refuses a benign request, how often a tool-using agent follows an injected instruction, how often a robot controller trips a safety stop. A decision to deploy needs a different object: a statement that the rate of the unwanted behaviour is at most some threshold, together with the chance that the statement is wrong. We call that statement a certificate.

The Seldonian framework (Thomas et al., 2019) produces this object as the last step of a learning algorithm. The algorithm searches for a solution on one part of the data, tests it on a part the search never saw, and returns it only if a one-sided confidence bound at level delta puts the constrained quantity under its threshold. Otherwise it returns "no solution found" (NSF). Sections 2 to 4 set out the algorithm, its use to post-train a language model, and the theory of each step.

Three features of language-model evaluation can each void the guarantee without any visible sign. The label is not observed: a guard model, a rubric judge or a benchmark harness produces it, and its errors differ from one population of responses to the next. The sample is rarely independent: benchmarks cross tasks with attacks, and failures cluster. And the policy has often been trained against the label that is then used to certify it. The rates of interest are also small, which is where normal approximations are known to fail (Bowyer et al., 2025).

This paper does not propose a new way to train. It takes the safety test, applies it to policies that are fixed by the time they are tested, and asks when its guarantee holds for a language model, what it costs in labels, and what breaks it. Section 5 states the proposal, section 6 the set-up of the evaluation, and sections 7 to 10 the results.

# 2 The Seldonian algorithm

A standard learning algorithm returns the parameters $\theta$ that score best on an objective, and promises nothing about a constraint that entered as a penalty. A Seldonian algorithm (Thomas et al., 2019) starts from the promise. The user supplies constraint functions $g_i$, with $g_i(\theta) \leq 0$ meaning the behaviour is acceptable, and a tolerance $\delta_i$ for each. The algorithm $a$ maps data $D$ to a solution or

to NSF, and must satisfy

$$\mathbb{P}(g_i(a(D)) \leq 0) \geq 1 - \delta_i \qquad \text{for every } i,$$

where $g_i(\text{NSF}) = 0$ by definition, so declining never counts as a violation. With $\delta = \sum_i \delta_i$, the probability of returning a solution that violates some constraint is at most $\delta$. The standard construction has three steps.

```
1. split      D into a candidate set D_c and a safety set D_s
2. candidate  using D_c only, search for the theta_c with the best objective
   selection  among those predicted to pass step 3
3. safety     using D_s only, once: compute U_i, a (1 - delta_i) upper confidence
   test       bound on g_i(theta_c), from unbiased estimates g_hat_i of it
              return theta_c if every U_i <= 0, otherwise return NSF
```

It works because $\theta_c$ is a function of $D_c$ alone. Given $D_c$ it is one fixed solution and $D_s$ is a fresh sample, so step 3 is an ordinary confidence bound and falls below the true $g_i(\theta_c)$ with probability at most $\delta_i$. The search may look at any number of candidates, because exactly one is tested, once.

The guarantee is on the output. It does not depend on the search converging or on the prediction in step 2 being accurate. Nothing is promised about how often a solution is returned: an algorithm that always returns NSF is Seldonian, so the rate at which solutions are returned is reported as a measure of quality. And the guarantee is exact only if the bound is. With a normal-approximation bound such as Student's t, Thomas et al. call the algorithm quasi-Seldonian.

## 3 Seldonian post-training of a language model

Post-training starts from a released, instruction-tuned model and changes its behaviour with a small number of further updates. In the work this paper builds on, the updates are reinforcement learning on a reward (GRPO, Group Relative Policy Optimisation; Shao et al., 2024) applied to a LoRA adapter (Hu et al., 2022), with the base weights frozen. The untrained base model is the *reference policy*. The concern is that optimising a reward model's score moves behaviours the reward does not measure, such as how often a benign request is refused. Table 1 gives each part of section 2 its language-model form, as implemented in our training code.

**Table 1.** The Seldonian algorithm instantiated for post-training a language model.

| **part of the algorithm** | **language-model form** |
|---|---|
| solution $\theta$ | the weights of a LoRA adapter on a frozen instruction-tuned model |
| data $D$ | prompts, each with a group tag (benign, adversarial, a demographic value). Responses are sampled from the policy when needed |
| split | 60/40 into $D_c$ and $D_s$, stratified by group, with a fixed seed. $D_s$ is sealed during training and read by one safety test only |
| objective | the expected reward of the policy's responses, from a reward model or exact match |
| constraint $g_i$ | the rate of a judged event among responses to a group of prompts, minus a threshold $\tau_i$. The judge is a frozen function of (prompt, response): a guard model's field, or code |
| threshold $\tau_i$ | absolute, or the reference policy's rate plus a margin |
| candidate selection | GRPO on $D_c$; the checkpoint that passes a predicted safety test with the highest reward |
| safety test | one fresh response per safety prompt, judged; a one-sided upper bound on the rate at level $\delta/n_g$ |

**Table 1.** The Seldonian algorithm instantiated for post-training a language model.

| part of the algorithm | language-model form |
|---|---|
| NSF | the run returns no adapter |

Two things differ from the classical setting. The policy can be queried, so $g(\theta)$ needs no off-policy estimate with importance weights: the candidate's own response to each safety prompt is sampled and judged, and the safety test is a bound on a mean of bounded, independent variables. And the constrained quantity is not observed. It is whatever the judge says it is, so every guarantee below is about the judge's label (sections 8.2, 9 and 10.3).

The safety test does not depend on how the candidate was produced. Reinforcement learning, supervised instruction tuning on $D_c$, or no training at all leave it unchanged. We have run the first (our own adapters) and the last (other people's published models, sections 10.1 and 10.2), and not supervised instruction tuning.

# 4 Theory: what is optimised, $\hat{g}$, candidate selection, the safety test

*Notation.* The symbols of sections 2 to 10, each with one meaning.

| symbol | meaning |
|---|---|
| $a$, $D$, $D_c$, $D_s$, NSF | the algorithm; the data; the candidate set; the safety set; "no solution found" |
| $\theta$, $\theta_c$, $\theta_0$, $\pi_\theta$ | an adapter's weights; the candidate; the reference policy; the policy |
| $x$, $y$, $r(x,y)$, $J$, $L$ | a prompt; a response; its reward; the expected reward; the Lagrangian |
| $c_i(x,y)$, $e_j$, $d_j$ | judge $i$'s 0/1 label; the label of the $j$-th sampled response; a paired difference of labels |
| $g_i$, $\hat{g}_j$, $\tau_i$, $\delta_i$, $n_g$ | constraint $i$; its per-response estimate; its threshold; its level; the number of constraints |
| $p$, $\hat{p}$, $U$, $U_{\text{pred}}$ | a true rate; an estimate; a one-sided upper limit; the predicted limit of candidate selection |
| $n$, $s$, $n_s$, $n_{\text{cand}}$, $n_{\text{eff}}$ | labels; positives among them; safety prompts; candidate prompts in a predicted test; their effective size |
| $\lambda_i$, $\eta$, phi | the Lagrange multiplier; its step size; the inflation of the predicted test |
| $z$, $t$ | a standard normal quantile; a Student-t quantile |
| $R$, $\hat{\text{miss}}$, se, $T$ | replications of a validity check; the share of their limits below the truth; its standard error; checks over a training run |
| $m$ | the root that defines a Wilson-type limit |
| $H$, $h$, $W_h$, $n_h$, $s_h$, $\hat{\mu}$, $p \sim_h$ | strata; a stratum; its share, labels and positives; the stratified estimate; a smoothed stratum rate |
| $N$, $N_u$ | the prompts in a pool; unlabelled responses |
| $f$, gamma, $V$, $\rho^2$ | a judge's feature; the PPI++ weight; a variance; the squared correlation of gold label and feature |
| $B$, $t_b$, $t^*_\delta$ | bootstrap draws; a studentised bootstrap error; the delta-quantile of the $t_b$ |
| $C$, $c$, $n_c$, $s_c$ | clusters (user tasks); a cluster; its episodes; its successes |
| $V_u$, $V_{\text{inj}}$, $V_p$, $V_2$ | variances clustered by user task, by injection task and by pair; the multiway variance |

| symbol | meaning |
|---|---|
| $k$, $\mathrm{ICC}_{\mathrm{ref}}$, $\mathrm{ICC}_{\mathrm{cand}}$, omega, rel($k$), $c_H$, $G$ | reference samples per prompt; the reference's and the trained policy's intraclass correlation; the correlation of a prompt's reference and trained rates; the reliability of a $k$-sample rate; the share $H$ strata keep; the variance share between strata |
| $w$, ESS | the mean distance to a limit; the effective sample size and its variants $\mathrm{ESS}_{\mathrm{est}}$, $\mathrm{ESS}_{\mathrm{truth}}$, $\mathrm{ESS}_{\mathrm{med}}$, $\mathrm{ESS}_{\mathrm{pred}}$ (section 6.4) |
| $q$, rec, fa | a judge's flag rate; its recall; its false-alarm rate |

## 4.1 The problem

Write $x$ for a prompt, $y \sim \pi_\theta(\cdot \mid x)$ for a response, $r(x, y)$ for the reward, and $c_i(x, y)$ in $[0, 1]$ for judge $i$'s label, with 1 meaning the constrained event happened. The problem is

$$\begin{aligned} &\text{maximise} && J(\theta) = \mathbb{E}_x \mathbb{E}_y[r(x, y)] \\ &\text{subject to} && g_i(\theta) = \mathbb{E}_{x \in \text{group } i} \mathbb{E}_y[c_i(x, y)] - \tau_i \le 0, \qquad i = 1, \ldots, n_g. \end{aligned}$$

The first term of $g_i$ is the *behaviour rate* $p_i(\theta)$: the probability that a response to a prompt from group $i$ is flagged, over the prompt and over the policy's own sampling. No algorithm can solve this problem exactly from a sample. The Seldonian requirement replaces it: return a policy with high reward, and only if $g_i(\theta) \le 0$ can be supported at confidence $1 - \delta$. The optimiser increases a surrogate of $J$ on candidate prompts, and a selection rule picks one checkpoint of the run. Neither enforces the constraint. The safety test does.

## 4.2 $\hat{g}$: the estimate of the constraint

For a prompt $x_j$ drawn from group $i$ and one response $y_j$ sampled from the policy at temperature 1, let $e_j = c_i(x_j, y_j)$. Then

$$\hat{g}_j = e_j - \tau_i, \qquad \mathbb{E}[\hat{g}_j] = p_i(\theta) - \tau_i = g_i(\theta).$$

Each $\hat{g}_j$ is an unbiased estimate of $g_i(\theta)$, and estimates on different prompts are independent when the prompts are an independent sample. This is the $\hat{g}$ of Thomas et al. (2019). Three quantities are built from the labels $e_j$ and should not be confused: the point estimate mean$(e) - \tau$, used for reporting; the *predicted* upper bound, computed on candidate prompts during training and used to choose a checkpoint (section 4.3); and the *safety* upper bound, computed once on the safety set, which alone decides whether the policy is returned (section 4.4).

**Relative thresholds.** Several constraints here are relative: the trained policy's rate may exceed the reference model's by at most a margin. In the training runs cited, the reference rate was measured once on candidate prompts and $\tau =$ reference rate $+$ margin was then treated as a constant, so the certificate is about that number and not about the true difference of the two rates. Section 10.4's design does not do that. In human terms the reference rate is unknown, both policies answer the same safety prompts, and the estimate is the paired difference $d_j = e_j(\theta) - e_j(\theta_0)$ in $\{-1, 0, 1\}$.

## 4.3 Candidate selection

**The update.** At each step GRPO samples a group of responses to each prompt of a batch, scores them, and weights a clipped policy-gradient step by the score normalised within the prompt's group of responses, with a KL penalty to the reference policy. The score is centred and scaled within

the group, so a penalty acts only where the responses to one prompt differ in it. In the *filter-only* variant GRPO sees the reward $r$, and the constraint enters through the choice of checkpoint alone. In the *Lagrangian* variant it sees $r - \sum_i \lambda_i c_i$, each penalty applied on its constraint's prompts: a stochastic ascent step on $L(\theta, \lambda) = J(\theta) - \sum_i \lambda_i g_i(\theta)$.

**The predicted safety test.** Every few steps the current checkpoint is sampled on $n_{\text{cand}}$ candidate prompts, one response each, and judged. The bound the safety test will use is computed on these labels at the sample size $n_{\text{eff}}$, with an optional inflation phi $\geq 1$:

$$n_{\text{eff}} = 1/(1/n_{\text{cand}} + 1/n_s),$$
$$U_{\text{pred}} = \text{mean}(e) + \text{phi} \cdot (U(e; \delta_i, n_{\text{eff}}) - \text{mean}(e)).$$

$n_{\text{cand}}$ and $n_s$ count the candidate and safety prompts in the constraint's group. What has to be anticipated is the gap between the rate seen now on $n_{\text{cand}}$ prompts and the rate the safety test will see on $n_s$ others, and that gap has variance proportional to $1/n_{\text{cand}} + 1/n_s$. Thomas et al. (2019) use the safety-set size with phi $= 2$; the later runs cited here use $n_{\text{eff}}$ with phi $= 1$. A checkpoint is *feasible* if $U_{\text{pred},i} \leq \tau_i$ for every constraint.

**The multiplier and the choice.** In the Lagrangian variant, after each predicted test,

$$\lambda_i \leftarrow \text{clip}(\lambda_i + \eta \cdot (U_{\text{pred},i} - \tau_i), \text{floor}, \lambda_{\max}).$$

The step size under slack can be set separately, and in the language-model runs cited it was the same. The floor is zero until it is switched on, from the first update or once the constraint has been predicted infeasible. The predicted upper bound drives the multiplier, not the point estimate, so the penalty rises before the estimated rate reaches the threshold. Among feasible checkpoints, the one with the highest mean reward on its predicted-test sample is kept. If none was feasible, the last one is tested anyway and usually fails.

**Why none of this carries a guarantee.** Lagrangian theory is about a saddle point: for a convex problem, at convergence, the average of the iterates satisfies the constraint in expectation. It says nothing about the checkpoint in hand after 200 steps on a finite sample. The predicted test is computed on candidate prompts, with fresh responses, and the checkpoint is chosen because it looked best on them, so its predicted rate tends to be optimistic (Table 2). Candidate selection decides how often the safety test passes and how much reward the returned policy has. It has no effect on whether the guarantee is valid.

**Table 2.** "The constraint held": a Lagrangian's statement against a certificate's.

| | **penalty or Lagrangian** | **certificate** |
|---|---|---|
| measured on | fresh responses to candidate prompts, which the selection rule has seen | one draw of $D_s$, independent of both |
| statement | the estimate ended below the threshold in this run | the (1 - delta) upper bound on the returned policy is below the threshold |
| selection bias | uncorrected: predicted 0.133 against 0.184 on the safety set (one run of the companion paper); 0.141 against 0.168 (the constrained run of section 9.2). At another seed of the first experiment the predicted test erred the other way, by 0.01–0.02 | removed by the fresh draw; the first of those runs returned NSF |
| checked by | how many runs ended over the threshold: with a fixed penalty of 1, 3 of 3 seeds; with a penalty of 4, 1 of 3 | the miss rate against delta (section 7.1) |

**Table 2.** "The constraint held": a Lagrangian's statement against a certificate's.

| | **penalty or Lagrangian** | **certificate** |
|---|---|---|
| can decline | no | yes; where training was repeated over seeds the solution rate is reported (section 7.2) |

### 4.4 The safety test

**The test.** Training has ended and one checkpoint $\theta_c$ is selected. For each constraint $i$, sample one response to each of the $n_s$ safety prompts in its group, judge them, and compute a one-sided upper bound $U_i$ on the rate $p_i(\theta_c)$ at level $\delta_i = \delta/n_g$. Return $\theta_c$ if $U_i \le \tau_i$ for every $i$. Otherwise return NSF. The safety set is used once.

**The guarantee.** A bound is *valid* at level $\delta_i$ if $\mathbb{P}(U_i < p_i) \le \delta_i$ for a fixed policy and an independent sample. Given $D_c$ and all the randomness of training, $\theta_c$ is fixed and the safety labels are an independent sample with mean $p_i(\theta_c)$. A violating policy is returned only if $U_i \le \tau_i < p_i(\theta_c)$ for some $i$, so

$$\mathbb{P}(\text{returned and } g_i(\theta_c) > 0 \text{ for some } i \mid D_c) \le \sum_i \mathbb{P}(U_i < p_i(\theta_c) \mid D_c) \le \delta.$$

Averaging over $D_c$ gives the guarantee of section 2, for any selection rule on the candidate side.

**The bounds.** For 0/1 labels with $s$ positives among $n$, the Clopper-Pearson bound (Clopper and Pearson, 1934) is exact:

$$U = \text{Beta}^{-1}(1-\delta; s+1, n-s), \qquad \text{with } s = 0 : U = 1 - \delta^{1/n}.$$

For labels in $[0, 1]$ the exact choices are Hoeffding's (Hoeffding, 1963), Bentkus's (Bentkus, 2004) and Anderson's (as given by Learned-Miller and Thomas, 2019) bounds and a betting bound (Waudby-Smith and Ramdas, 2024). Student's t is approximate. Section 6.3 gives the bounds for stratified, judge-assisted and clustered data. As a worked case: in seed 0 of the constrained run of section 9.2 the returned checkpoint was step 175 of 200; on the 500 prompts of the safety set its refusal rate by the guard's flag was 0.168, the Clopper-Pearson upper bound at delta 0.05 was 0.198 and the threshold 0.206, so the adapter was returned.

**NSF.** The test returns NSF when $U_i > \tau_i$ for some constraint. NSF does not say the policy violates. It says this sample cannot support the claim at this confidence, because the rate is too close to the threshold or the sample is too small. Roughly, a policy with true rate $p$ passes with probability one half when $\tau - p = z_{1-\delta}\sqrt{p(1-p)/n}$. With no positive observed, the smallest sample that certifies a threshold $\tau$ is $n \ge \ln(\delta)/\ln(1-\tau)$: 299 labels for 1% at delta 0.05, 149 for 2%, 59 for 5%. A method that always returns NSF satisfies the guarantee, so the solution rate is part of the result: 10 of 10 on a brevity constraint, 2 of 3 on over-refusal, and 1 of 3 on the constrained run of section 9.2.

### 4.5 What the guarantee assumes

The argument of section 4.4 uses four assumptions. Each is routinely violated in language-model evaluation, and each has a part of this paper.

| | assumption | how it fails | where we test it |
|---|---|---|---|
| A1 | the safety set had no part in choosing the policy, and is used once | a split or a selection rule that uses safety-set labels; a test repeated until it passes | section 7.2 |
| A2 | the safety labels are independent draws from the population the claim is about | benchmarks that cross tasks with attacks; a labelling sheet that over-samples flagged responses | sections 9.3, 9.5, 10.2 |
| A3 | the bound is valid at this sample size and this rate | normal and Student-t limits at rates of a few percent | sections 7.1, 8, 9.4 |
| A4 | the label is the quantity the claim is about | a judge's errors, corrected by a calibration measured on other responses | sections 9.1, 9.2, 10.3 |

# 5 What this paper proposes

Sections 2 to 4 describe an algorithm with two halves: a search for a candidate, and a test that decides whether the candidate may be returned. The search is the subject of the companion paper. This paper is about the test.

**The proposal.** We propose the Seldonian safety test, taken on its own, as a certificate for a behaviour rate of a language-model policy: a one-sided bound at level delta on the rate of a code- or model-defined label, computed on a sample the choice of policy never saw, with NSF as an outcome. The policy may be one we trained or one somebody else published. The rest of the paper audits the four assumptions of section 4.5. Our contributions:

- **Which bounds hold** their level at the sample sizes and rates of a safety test, with the bounds we use measured under one rule (sections 7 and 8; Table 4 holds the main ones).
- **What a label budget buys:** a safety set stratified by the reference model's own per-prompt rate, and a judge used as a variance reducer under a routing rule that reads only the labels' own counts (section 8).
- **What does not carry:** a judge's calibration, across prompt populations or, for a rubric-compiled judge in each of three seeds of the one run we examined, across training that targets the label (a second judge family's did carry there); independent-sample bounds on crossed designs; and a stratified labelling sheet read as a random sample (section 9).
- **Certificates on published traces** of a robot benchmark and an agent-security benchmark (sections 10.1 and 10.2).
- **The price of human terms:** a guard model measured against human refusal labels, and the one certificate of a trained policy's own rate in those labels, and the cost of one of its constraint (sections 10.3 and 10.4).

**What the paper is not.** It is not a new training method, and it does not claim that any model is safe. Nor is it a study of NSF for its own sake. NSF is one of the test's two answers. The option to decline is what makes the guarantee possible without a fallback: a method that must return something can promise nothing about it unless a solution known to satisfy the constraint is at hand. How often the test declines is its cost. Here NSF is the answer for 1 of 3 training seeds on the over-refusal constraint in the companion paper and for 2 of the 3 seeds of the constrained run of section 9.2, for all three models of a robot benchmark at a 5% threshold and delta 0.05, and for 27 of 28 pipelines of an agent-security benchmark at the same threshold and delta, with its injection tasks taken as fixed (sections 7.2, 9.2, 10.1 and 10.2). A solution rate is defined over repeated training runs and is reported there; a published trace has one sample and one answer.

**Scope.**

- The label is a guard model's or the benchmark harness's unless marked human. The human-label results are in section 10.3, with one more in sections 8.2 and 9.5 (a harm sheet of 225 responses labelled by one annotator, an author, 3 of them harmful) and published human labels in section 8.1.4. Section 10.3 computes a certificate in human labels of a trained policy's own rate; section 10.4 costs one of its relative constraint and does not compute it.
- Policies trained here are 0.5B to 2B parameters on one 12 GB card. The frontier-model results are certificates on other people's published traces, not training.
- A certificate is about the label as defined and the population the safety sample was drawn from. It says nothing about other prompts or about the policy after further training.
- How the certified policies were trained is the companion paper's subject. Trained checkpoints are resampled as fixed policies in sections 7.1 and 8.1. How a policy was trained matters to the result in four places only: the policy whose human-terms certificate section 10.4 costs, the negative result of section 9.2, which exists because training targeted the label, and the second model family of section 8.1.3, which counts as only that because its training did not move the policy, and the run after it, whose training did. Section 10.4's policy is one training run with one seed, as is that last run; section 9.2's constrained run has three seeds, and its first also supplies section 8.1's pushed label and the policy of its confirmation pools.

# 6 Evaluation set-up

The object under evaluation is a *certificate procedure*: a sampling design together with a bound that turns the labelled sample into an upper limit $U$. We ask whether the procedure is valid, and how wide its limit is.

## 6.1 Validity: the miss rate

Fix a population with a known true rate $p$, draw a sample $S$ by the design and compute $U(S)$. The procedure *misses* when $U(S) < p$, and is valid at level delta if

$$\text{miss rate } = \mathbb{P}_S(U(S) < p) \leq \delta.$$

This is the one quantity the guarantee of section 4.4 needs from a bound. It cannot be read off a single real sample, where the truth is unknown, so we repeat the draw $R$ times in a setting where $p$ is known and take $\hat{\text{miss}}$, the share of the $R$ limits that fall below $p$. If the true miss rate were exactly delta, $\hat{\text{miss}}$ would have standard error $\text{se} = \sqrt{\delta(1-\delta)/R}$. One rule is applied to every bound, favoured or not. A cell (one bound, design, population, sample size and delta) is

- *over* its level when $\hat{\text{miss}} > \delta + 2\text{se}$;
- *unresolved* when $\delta < \hat{\text{miss}} \leq \delta + 2\text{se}$: the check cannot tell it from delta, which is weaker than showing that the bound holds;
- *at or under* otherwise.

With $R = 4{,}000$ and delta 0.05, a cell is over from a miss rate of 0.057. The three classes say little about how large a miss a check could have overlooked, and the rule is not corrected for the number of cells inspected. So every row of Table 4 also carries two numbers, computed the same

way for a bound we use and for one we reject. The first is the largest upper end, over the row's cells, of a 95% Clopper-Pearson interval for the cell's miss rate: the check cannot rule out a miss up to that value. The second is the number of cells still over when the two standard errors are widened by a Bonferroni correction for the cells counted together (one row at one delta or under one scheme).

### 6.2 Three sources of a known truth

**A synthetic environment (the whole pipeline).** A contextual bandit stands in for the language model, so that the training and testing code of section 3 runs unchanged and the true rate of any policy is an exact expectation: 20,000 contexts, four actions, a uniform reference policy that violates 13% of the time, and a reward whose pressure parameter makes the better actions the more violating ones. The unit of replication is a whole run (split, training, selection, safety test), and the estimated quantity is the Seldonian failure probability itself, $\mathbb{P}$(returned and truly violating). With a real model one replication is hours of training, so this is where assumption A1 is tested.

**Resampling real responses (a plasmode; Franklin et al., 2014).** A pool of real responses, each with its label, is treated as the population, and its mean is the truth. A replication draws a safety set from the pool by the design under test and computes the bound. The responses, the judge's scores and their dependence are real; only the sampling is simulated. We use three kinds of pool.

- *Per-prompt pools.* 500 prompts, each with $k = 8$ judged responses of the reference model and further responses of each trained checkpoint, generated by Granite-3.3-2B (Granite Team, IBM, 2025); section 8.1.3 adds pools of 400 new prompts, one of them generated by Qwen2.5-1.5B-Instruct, and section 8.1.4 XSTest's 250 prompts with five released models' completions. A replication draws $n_s$ prompts without replacement, at random or within strata, and one response of the checkpoint for each. The safety set is then 20–50% of the pool and the truth is the pool's own rate: a finite population, which no bound here corrects for, so every bound is wider than it needs to be and the miss rates come out low. We therefore redraw the same cells with replacement within the same strata, at 40,000 draws a cell, which is the limit of a pool far larger than the safety set.
- *Judge pools.* 500 responses, each with a gold label (the guard's refusal field, or an exact word count) and a rubric judge's score. The rubric judge is a *compiled* judge: an instruction-tuned model (Qwen3-8B, run 4-bit) turns a one-sentence statement of the constraint into judging instructions and scores each response by its probability of answering Yes, on one forward pass; six wordings of the sentence give six such judges. A replication draws responses with replacement and reveals the gold label on the first $n$. In the cells at a set rate (20%, 5% and 1.3%) the label is drawn at that rate and the score from the pool's scores for that label, so the truth there is the set rate.
- *Labelling sheets.* A sheet is re-drawn by its real sampling rule, which over-samples the responses a guard flags. The labels in this study are planted on the real responses at a known rate; they are not an annotator's.

**Resampling clusters (a crossed design).** For AgentDojo each pipeline's observed table of (user task, injection task) outcomes is the population and its overall rate the truth. A replication redraws whole tasks with replacement under one of three schemes: (a) user tasks redrawn, injection tasks fixed; (b) the reverse; (c) both. Schemes (a_s), (b_s) and (c_s) do the same within each of the benchmark's suites (section 10.2). The scheme states what the certificate treats as sampled,

and so what population it speaks about. The truth can also be the rate of a crossed model fitted to each table, from which new tables are drawn (section 10.2) [B6].

### 6.3 The bounds under test

All are one-sided upper limits at level delta; $z$ is a normal quantile.

**Labels alone.** The bounds of section 4.4, and the Wilson score bound (Wilson, 1927): the smallest $m > \hat{p}$ with $m - \hat{p} \geq z_{1-\delta}\sqrt{m(1-m)/n}$, and 1 when $\hat{p}$ is 1. It puts the variance at the hypothesised rate and not at the estimate, so its width is not zero when no positive is observed: the limit is then $z^2/(n+z^2)$. It is approximate, and for an i.i.d. sample its miss probability can be computed exactly, by enumeration over the binomial counts (section 7.1).

**A stratified sample (`b1w`).** With strata $h = 1, \ldots, H$ of population share $W_h$, $n_h$ labels and observed rate $\hat{p}_h$ in each, the estimate is $\hat{\mu} = \sum_h W_h \hat{p}_h$. The stratified Wilson-type bound, `b1w` (our code's name for it: bound family 1, the approximate stratified bounds, in its Wilson form), is the smallest $m > \hat{\mu}$ with

$$m - \hat{\mu} \geq z_{1-\delta}\sqrt{\sum_h W_h^2 p_h(m)(1-p_h(m))/n_h},$$

where $p_h(m)$ shifts every stratum's rate by $m - \hat{\mu}$, clipped to $[0, 1]$. With one stratum it is the Wilson bound. It is a heuristic and not the inversion of a stratified score test (score intervals with the variance at the constrained estimate go back to Miettinen and Nurminen, 1985; Yan and Su, 2010, give a stratified Wilson interval): a common shift of every stratum is not the constrained estimate of the rates, and once a rate is clipped the shifted rates no longer average to $m$. The point $m = \hat{\mu}$ is excluded because it satisfies the inequality with both sides zero whenever every stratum is all zeros or all ones. It is approximate.

The stratified Wald-t limit, its companion in sections 7 and 8, is $\hat{\mu} + t\sqrt{\sum_h W_h^2 p \sim_h (1 - p \sim_h)/(n_h - 1)}$, with $p \sim_h = (s_h + 1/2)/(n_h + 1)$ for $s_h$ positives and a Student-t quantile at Satterthwaite's degrees of freedom (Satterthwaite, 1946). When the claim is about the population a pool of $N$ prompts was drawn from, both limits add $\sum_h W_h(\hat{p}_h - \hat{\mu})^2/N$ to the variance, the part of the pool's own sampling error that lies between strata: the first-phase term of two-phase sampling for stratification (Neyman, 1938; Cochran, 1977).

**Labels and a judge (PPI++; Angelopoulos et al., 2023b).** With $n$ gold labels $e$, and the judge's feature $f$ on the same responses and on $N_u$ unlabelled ones,

$$\begin{aligned} \hat{p} &= \text{mean}(e) + \text{gamma}\ (\text{mean}_{\text{unl}}(f) - \text{mean}_{\text{lab}}(f)), \\ \text{gamma}\ &= \text{cov}(e, f)/((1 + n/N_u)\,\text{var}(f)), \\ V &= \text{var}(e - \text{gamma}\ f)/n + \text{gamma}^{\,2}\,\text{var}_{\text{unl}}(f)/N_u. \end{aligned}$$

$\hat{p}$ is unbiased for the gold rate for any fixed weight gamma, because the judge's two means cancel in expectation. The published limit is $\hat{p} + z_{1-\delta}\sqrt{V}$. StratPPI (Fisch et al., 2024) applies the same correction within each stratum and combines the strata with weights $W_h$, again with a normal limit. PPBoot (Zrnic, 2024) replaces the normal limit by a percentile of bootstrap estimates, with the judge's weight fixed at 1 (basic) or chosen as in PPI++ to minimise the variance (power-tuned).

**The bootstrap-t limit.** This is the percentile-t limit (Hall, 1988; Hall, 1992). For any of these estimators, resample the labelled responses with replacement $B$ times (within strata, for a stratified estimator), redraw each unlabelled mean from its sampling distribution, and recompute the estimate $p_b^*$ and its variance $V_b^*$. With $t_b = (p_b^* - \hat{p})/\sqrt{V_b^*}$ and $t_\delta^*$ the delta-quantile of the $t_b$,

$$U = \hat{p} - t_\delta^*\sqrt{V}.$$

It replaces the symmetric normal quantile by the estimated distribution of the studentised error, which is skewed when the rate is small. It is approximate.

**Clustered data.** With $C$ clusters $c$ (user tasks) of sizes $n_c$ and success counts $s_c$, $\hat{p} = \sum_c s_c / \sum_c n_c$ and

$$\text{se} = \sqrt{C/(C-1) \cdot \sum_c (s_c - \hat{p} n_c)^2} / \sum_c n_c.$$

The limit is the bootstrap-t limit with whole clusters resampled. The naive alternative treats the episodes as independent and applies Clopper-Pearson.

When clusters cross (user tasks by injection tasks), two more bounds are tested. With $V_u$ and $V_{\text{inj}}$ the squares of the standard error above for the two kinds of cluster and $V_p = \hat{p}(1-\hat{p})/(n-1)$ the same with every pair its own cluster, the multiway variance (Cameron, Gelbach and Miller, 2011) is

$$V_2 = V_u + V_{\text{inj}} - V_p,$$

which we replace by the larger of $V_u$ and $V_{\text{inj}}$ if it is not positive. One bound is $\hat{p} + t\sqrt{V_2}$, where for $t$ we take a Student-t quantile on one fewer degrees of freedom than the smaller number of clusters. The other is the bootstrap-t limit with user tasks and injection tasks resampled independently (the pigeonhole bootstrap; Owen, 2007) and $V_2$ as the variance in every resample. Both return 1 when the table holds no success.

**A paired difference.** For a relative constraint in human terms the data are the differences $d_j$ of section 4.2. The exact limit is the betting bound on $(d_j + 1)/2$, mapped back. The approximate ones are the bootstrap-t limit on mean($d$), and PPI++ with the difference of the guard's two logits as the feature.

### 6.4 What a design or a judge is worth: effective sample size

Width is compared at equal validity, and where a baseline is itself over its level we say so. For a design or estimator $A$ and a baseline (a simple random sample with a pooled bound: the pooled Wilson bound for the stratified limits, the labels alone in section 8.2) at the same number of labels and the same delta, with $w$ the mean distance from a reference point to the limit,

$$\text{ESS} = (w_{\text{base}}/w_A)^2.$$

An ESS of 2 means the design does with $n$ labels about what the baseline does with $2n$. This is the one definition. The studies differ in the reference point and the checkpoint, and every ESS or gain in sections 7 to 10 and the appendices is one of four variants, labelled where the text does not make it plain:

- $\text{ESS}_{\text{est}}$: $w$ from the estimate to the limit. The first strata study (section 8.1.1) uses it, at delta 0.10 and at 0.05; the delta is given with each figure.
- $\text{ESS}_{\text{truth}}$: $w$ from the truth to the limit (the limit's mean excess over the truth), at a label's last checkpoint, or at the one checkpoint a pool has; at delta 0.05 unless stated. The StratPPI comparison (Table A1), the confirmation pools and the judge's measured worth use it. The redraw with replacement computes it as a ratio of mean excesses with the arm and the baseline on the same kind of draw, stored or with replacement.
- $\text{ESS}_{\text{med}}$: $\text{ESS}_{\text{truth}}$ at every checkpoint, then the median over a label's checkpoints (the claim about the prompt source, section 8.1.1).
- $\text{ESS}_{\text{pred}}$: a prediction from a formula of this section, not a measurement.

A median over cells of one variant keeps its label.

**Reference-rate strata.** One response per prompt has variance $p(1-p)$, part of it between prompts (prompts differ in their rate) and part within (the policy's own sampling). The between-prompt share is the intraclass correlation, ICC. A sample drawn within strata, in proportion to their size, loses the variance that lies between strata; if that is a share $G$ of the total, $\mathrm{ESS}_{\mathrm{pred}} = 1/(1-G)$. A relative constraint already calls for the reference model's responses, so the strata come from them: sample the reference $k$ times per prompt, rank the prompts by the share flagged, and cut the ranking into $H$ equal strata with ties broken at random. This happens before training and uses no response of the trained policy. The resampling studies of section 8.1 hold the candidate fixed, so they do not test a candidate trained on the pool's other prompts; the in-loop check of section 7.2 does, on a synthetic environment. $G$ follows a formula:

$$G \approx \mathrm{ICC}_{\mathrm{cand}} \cdot \mathrm{omega}^{\,2} \cdot \mathrm{rel}(k) \cdot c_H, \qquad \mathrm{rel}(k) = k\mathrm{ICC}_{\mathrm{ref}}/(1 + (k-1)\mathrm{ICC}_{\mathrm{ref}}),$$

where $\mathrm{ICC}_{\mathrm{cand}}$ is the trained policy's intraclass correlation, omega the correlation between a prompt's reference rate and its rate under the trained policy, $\mathrm{rel}(k)$ the reliability of a $k$-sample reference rate, and $c_H$ the share a cut into $H$ strata keeps (0.96 at $H = 8$). $\mathrm{ICC}_{\mathrm{cand}}$ and omega belong to the trained policy. Before one of its responses is labelled they have to be assumed. The *pre-flight*, our tool that computes this $\mathrm{ESS}_{\mathrm{pred}}$ before any label is drawn, takes $\mathrm{ICC}_{\mathrm{cand}} = \mathrm{ICC}_{\mathrm{ref}}$ and omega of 1 or 0.8; sections 8.1.1 and 8.1.3 report the formula with the two quantities measured and with omega $= 1$. When the pool of $N$ prompts is itself a sample of the population the claim is about, the gain is capped at about $1/(1 - G + Gn_s/N)$; section 8.1.1 measures that case.

**A judge.** With $\rho^2$ the squared correlation between the gold label and the judge's feature, the variance $V$ of section 6.3 gives $\mathrm{ESS}_{\mathrm{pred}} = 1/(1 - \rho^2 N_u/(N_u + n))$, which tends to $1/(1-\rho^2)$ when unlabelled responses are plentiful. A judge that predicts nothing has gamma near 0 and an ESS of 1.

## 6.5 Carrying a calibration

A cheaper route uses no gold label on the certified policy at all. If the judge flags a true positive with probability rec (recall) and a true negative with probability fa (the false-alarm rate), its flag rate is $q = \mathrm{rec}\ p + \mathrm{fa}\ (1-p)$, so $p = (q - \mathrm{fa})/(\mathrm{rec} - \mathrm{fa})$ (Rogan and Gladen, 1978). A *carried* bound measures rec and fa on one population of responses, measures $q$ on the population to be certified, and bounds $p$ from one-sided limits on the three at $\delta/3$ each. It is valid only if rec and fa are the same on both populations, which is assumption A4 in its sharpest form. We measure the miss rate of the carried bound across two kinds of shift: between prompt populations, and between a reference model and a policy trained from it.

## 6.6 Models, labels and data

**Table 3.** The settings of the evaluation.

| part | policy | label | sample | truth | delta |
|---|---|---|---|---|---|
| whole pipeline (7.2) | synthetic bandit | simulated judge | 200 to 5,000 prompts | exact (the pool's rate or the population's) | 0.10; 0.05 and 0.10 in the 8-strata check |

**Table 3.** The settings of the evaluation.

| part | policy | label | sample | truth | delta |
|---|---|---|---|---|---|
| labels alone (7.1) | three trained Qwen2.5-0.5B policies (Yang et al., 2024) | guard's harm field, rate 0.034 | 200 to 2,400 of 3,600 responses | pool mean | 0.10 |
| reference-rate strata (8.1) | Granite-3.3-2B, reference and trained; Qwen2.5-1.5B-Instruct, reference and trained (8.1.3) | guard's refusal and safety fields | 100 to 200 of 400 or 500 prompts | pool mean | 0.05, 0.10 |
| costly label (8.1.4) | five released model variants (XSTest's completions); the strata from Granite-3.3-2B's reference | XSTest's human strict and broad refusal labels | 50 to 100 of 250 prompts | pool mean | 0.05, 0.10 |
| judge routing (8.2) | 500 real responses; a harm sheet of 225 from 4,800 | gold: guard's refusal field, a word count; on the harm sheet, 225 human harm labels for the one real estimate and labels planted at 1.3%, 5% and 20% for the validity cells; judge: a rubric | 50 to 1,000 responses | pool mean; the planted rate for the sheet | 0.05 |
| carried calibration (9.1, 9.2) | a reference model, one trained policy and three seeds of another | gold: guard's refusal field; judge: the rubric, six wordings | benign and harmful prompt pools | pool mean | 0.05 |
| robot benchmark (10.1) | three published frontier models | harness safety-stop flag | 120 trials each, 20 on each of 6 tasks | unknown | 0.05, 0.10 |
| agent benchmark (10.2) | 28 published pipelines | harness `security` flag | 629 to 949 task pairs each | table rate; for one bound the table's average over user tasks; or a fitted crossed model's rate (10.2) | 0.05 |
| guard against humans (10.3) | five published model variants; our two policies | human refusal labels | 1,250 and 220 responses | none | 95% intervals; 0.05 for the certificate |
| human-terms design (10.4) | reference and trained Granite-3.3-2B | human strict refusal (not collected) | 300 of 490 prompt pairs | synthetic labels | 0.05 |

The benign prompts are XSTest's safe prompts and OR-Bench's (Röttger et al., 2024; Cui et al., 2025). The guard is Qwen3Guard (Zhao et al., 2025): the 4B model, run 4-bit quantised, except in the results cited from the companion paper (its entries in Table 2, the labels-alone row of Table 3 and the language-model seeds of section 7.2), which used the 0.6B model. Replications per cell are between 4,000 and 40,000 in Table 4, between 200 and 8,000 runs in the whole-pipeline studies of section 7.2, and 2,000 in the design check of section 10.4.

# 7 Results: validity

## 7.1 Which bounds hold their level

Section 6.1 defines the miss rate and the rule that sorts every cell into over, unresolved, and at or under its level; section 6.2 gives the three settings in which the truth is known. Table 4 applies that rule to the bounds we use. Three procedures built from them are measured where they appear: `b1w` with the term for a sampled pool (section 8.1.1), the routing rule taken as one procedure (section 8.2) and the paired limits of section 10.4 (Appendix C).

**Abstention.** A bound that returns no limit, or a limit of 1, gives no certificate and cannot miss, so part of a row's record in Table 4 can be abstention. We counted it for the bootstrap limits and for AgentDojo's exact rows, on the published draws, which the count reproduces exactly [abstention]. The betting bounds on AgentDojo never return no limit. The bootstrap-t StratPPI limit returns none in up to 6% of draws on the reference-rate strata (up to 8.6% when redrawn with replacement) and in up to 52% on the judge-logit strata at a 1.3% rate; PPI++ with a bootstrap-t limit in up to 55% of draws where a sample of 225 holds about three positives; and the clustered AgentDojo limits in up to 82% of tables, for the pipelines with the fewest successes. Counted as misses instead, the rows that hold would be over in up to 7 of 28 cells (row 12 at delta 0.05), in 10 of 168 for row 9 and for 2 to 7 of 28 pipelines in the AgentDojo rows. Those of row 9 are eight cells at a 1.3% rate with 225 labels, where Table 5's rule sends the labels to Clopper-Pearson, and two at n 50 on a 40% label, where it does not; those of row 11's judge-logit strata are four at the 1.3% rate and one at a 20% rate with 100 labels. Those of rows 11 and 12 on the reference-rate strata (4 and 7 of 28) are all at a safety set of 100, on labels at 16% to 95%: at that size the bootstrap-t StratPPI limit holds there partly by declining to certify. Counted this way even an exact bound can exceed its level: Clopper-Pearson returns 1 when every label is a positive, which at the 95% label with a safety set of 100 happens in 0.9% of draws and takes it to 0.052 at delta 0.05. So this count measures how much of a record is abstention, not validity. We did not count the stratified Wald-t limit, `b1w` or Clopper-Pearson elsewhere; they return a limit of 1 or more only near the top of the rate range.

**The sheets' draws.** The sheet rows draw 200 sheets from each of 20 plantings of the labels. With the plantings as clusters the design effect on the miss rate is at most 1.9 (median 0.9) in row 8, and no cell of rows 8 or 28 changes class; the pooled cell of the two runs stays unresolved (0.054) [abstention].

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 1 | Clopper-Pearson | exact (binary labels) | real harm labels, trained-policy pool, 121–124 positives of 3,600, n 200–2,400 | 0.10 | 0.066–0.096 | 0 / 0 / 20 | 0.096 (0) |
| 2 | Clopper-Pearson | exact | the plasmodes of section 8.2, every cell | 0.05 | 0–0.052 | 0 / 2 / 40 | 0.059 (0) |
| 3 | betting mixture | exact (bounded) | same pool as row 1 | 0.10 | 0.007–0.018 | 0 / 0 / 20 | 0.018 (0) |
| 4 | Bentkus | exact (bounded) | same pool | 0.10 | 0.017–0.037 | 0 / 0 / 20 | 0.037 (0) |
| 5 | Hoeffding, Anderson | exact (bounded) | same pool | 0.10 | 0.000 | 0 / 0 / 40 | 0.000 (0) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 6 | stratified Wilson-type `b1w` | approximate | 4 mid-rate labels (9–95%), real Granite-3.3-2B responses, $n_s$ 100–200, 3 checkpoints | 0.05 / 0.10 | 0.017–0.054 / 0.058–0.097 | 0 / 3 / 21; 0 / 0 / 24 | 0.061 (0); 0.106 (0) |
| 7 | `b1w` | approximate | label pushed by the Lagrangian, $n_s$ 200 | 0.05 / 0.10 | 0.011–0.023 / 0.040–0.064 | 0 / 0 / 2; 0 / 0 / 2 | 0.027 (0); 0.071 (0) |
| 8 | `b1w` on a design-weighted sheet | approximate | sheets re-drawn by their real sampling rule | 0.05 | 0–0.060 | 1 / 1 / 16 | 0.067 (0) |
| 9 | PPI++ with a bootstrap-t limit | approximate | the plasmodes of section 8.2, every cell and feature | 0.05 | 0–0.054 | 0 / 13 / 155 | 0.061 (0) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 10 | cluster bootstrap-t, by user task | approximate | AgentDojo, 28 pipelines, scheme (a) | 0.05 | 0.013–0.051 | 0 / 2 / 26 | 0.058 (0) |
| 11 | StratPPI estimator with a bootstrap-t limit, proportional allocation | approximate | reference-rate strata, 5 mid-rate labels, $n_s$ 100–200; judge-logit strata, n 100–1,000 | 0.05 | 0.024–0.045; 0.001–0.056 | 0 / 0 / 28; 0 / 5 / 23 | 0.051 (0); 0.064 (0) |
| 12 | the same estimator and limit, redrawn with replacement | approximate | reference-rate strata, 5 mid-rate labels, $n_s$ 100–200 | 0.05 / 0.10 | 0.030–0.050 / 0.069–0.100 | 0 / 0 / 28; 0 / 1 / 27 | 0.052 (0); 0.103 (0) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 13 | Clopper-Pearson, random draws with replacement (the control) | exact | the same 5 labels and sizes | 0.05 / 0.10 | 0.029–0.050 / 0.050–0.097 | 0 / 0 / 28; 0 / 0 / 28 | 0.052 (0); 0.100 (0) |
| 14 | pigeonhole bootstrap-t | approximate | AgentDojo, 28 pipelines, schemes (a); (b); (c) | 0.05 | at most 0.004; 0.037; 0.042 | 0 / 0 / 28; 0 / 0 / 28; 0 / 0 / 28 | 0.007 (0); 0.043 (0); 0.048 (0) |
| 15 | **Student-t** | fails at low rates | same pool as row 1, n 200–2,400 | 0.10 | **0.104–0.195** | 20 / 0 / 0 | 0.195 (20) |
| 16 | **`b1w` and the pooled Wilson bound at rare rates** | in the stored design, not over at delta 0.05; over in 2 cells at 0.10 | harm labels at 1–2%, $n_s$ 100–200, 3 checkpoints | 0.05 / 0.10 | 0–0.048 / **0–0.126** | 0 / 0 / 24; 2 / 1 / 21 | 0.054 (0); 0.136 (2) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 17 | **`b1w`, redrawn with replacement** | over on the two labels at rates of 65% and above | reference-rate strata, 5 mid-rate labels, $n_s$ 100–200 | 0.05 / 0.10 | **0.017–0.062 / 0.056–0.113** | 7 / 1 / 20; 6 / 2 / 20 | 0.065 (7); 0.116 (4) |
| 18 | **pooled Wilson bound, random draws with replacement** | over in 6 and 10 of the 28 cells | the same 5 labels and sizes | 0.05 / 0.10 | **0.031–0.069 / 0.069–0.128** | 6 / 0 / 22; 10 / 3 / 15 | 0.071 (6); 0.131 (10) |
| 19 | **PPI++ with a normal limit** | fails in most cells | the plasmodes of section 8.2 | 0.05 | **0.041–0.241** | 126 / 29 / 13 | 0.255 (87) |
| 20 | **StratPPI's normal limit, proportional allocation** | fails at these sizes | reference-rate strata; judge-logit strata | 0.05 | **0.024–0.086; 0.054–0.239** | 9 / 5 / 14; 26 / 2 / 0 | 0.094 (7); 0.252 (25) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 21 | **StratPPI's normal limit, the paper's allocations (oracle; heuristic)** | fails | reference-rate strata; judge-logit strata | 0.05 | **up to 0.110; 0.217; up to 0.219; 0.90** | 9 / 3 / 14; 11 / 0 / 15; 28 / 0 / 0; 24 / 1 / 3 | 0.119 (8); 0.229 (11); 0.232 (25); 0.912 (24) |
| 22 | **StratPPI estimator with a bootstrap-t limit, oracle or heuristic allocation** | fails | the same cells | 0.05 | **up to 0.078; 0.174; up to 0.083; 0.90** | 6 / 3 / 17; 11 / 1 / 14; 14 / 11 / 3; 14 / 0 / 14 | 0.086 (5); 0.185 (11); 0.092 (9); 0.910 (14) |
| 23 | **PPBoot, percentile limit (basic; power-tuned)** | fails at these sizes | judge-logit cells, unstratified | 0.05 | **up to 0.133; 0.148** | 7 / 6 / 1; 12 / 2 / 0 | 0.144 (5); 0.160 (11) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 24 | **Clopper-Pearson over pairs** | fails on a crossed design | AgentDojo, 28 pipelines, schemes (a); (b); (c) | 0.05 | **0.042–0.268; 0.060–0.340; 0.195–0.350** | 27 / 0 / 1; 28 / 0 / 0; 28 / 0 / 0 | 0.282 (26); 0.355 (27); 0.365 (28) |
| 25 | **cluster bootstrap-t by user task, when injection tasks are sampled** | fails | AgentDojo, schemes (b); (c) | 0.05 | **up to 0.258; 0.275** | 21 / 0 / 7; 24 / 0 / 4 | 0.272 (20); 0.289 (24) |
| 26 | **larger of the two clustered bounds** | holds under (a) only | AgentDojo, schemes (a); (b); (c) | 0.05 | at most 0.029; **up to 0.066; 0.089** | 0 / 0 / 28; 4 / 0 / 24; 16 / 3 / 9 | 0.034 (0); 0.074 (3); 0.098 (15) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 27 | **pigeonhole bootstrap, basic limit** | fails | AgentDojo, schemes (a); (b); (c) | 0.05 | **up to 0.174; 0.178; 0.278** | 5 / 0 / 23; 14 / 1 / 13; 27 / 0 / 1 | 0.187 (5); 0.190 (10); 0.292 (26) |
| 28 | **PPI on a stratified sheet read as an i.i.d. sample** | fails | three judge wordings, labels planted at three rates | 0.05 | **0–0.989** | 7 / 0 / 11 | 0.992 (7) |
| 29 | **the judge's rate alone** | fails | the plasmodes of section 8.2 | 0.05 | **0–1.000** | 30 / 0 / 12 | 1.000 (30) |
| 30 | **multiway cluster variance with a t quantile** | fails | AgentDojo, schemes (a); (b); (c) | 0.05 | **up to 0.136; 0.175; 0.179** | 5 / 0 / 23; 8 / 3 / 17; 26 / 1 / 1 | 0.147 (5); 0.187 (7); 0.191 (25) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 31 | **two clustered margins added in quadrature** | over in one cell under (c) | AgentDojo, schemes (a); (b); (c) | 0.05 | at most 0.015; 0.046; **0.058** | 0 / 0 / 28; 0 / 0 / 28; 1 / 1 / 26 | 0.020 (0); 0.053 (0); 0.066 (0) |
| 32 | stratified Wald-t limit, 8 reference-rate strata | approximate | 5 mid-rate labels, real Granite-3.3-2B responses, $n_s$ 100–200, the draws of the StratPPI row | 0.05 / 0.10 | 0.007–0.036 / 0.027–0.080 | 0 / 0 / 28; 0 / 0 / 28 | 0.042 (0); 0.088 (0) |
| 33 | the same limit, redrawn with replacement | approximate | the same 5 labels and sizes | 0.05 / 0.10 | 0.007–0.038 / 0.026–0.086 | 0 / 0 / 28; 0 / 0 / 28 | 0.040 (0); 0.088 (0) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 34 | the same limit with the term for a sampled pool | approximate | a claim about the prompt source: the pool redrawn and the strata rebuilt, the same 5 labels and sizes | 0.05 / 0.10 | 0.007–0.050 / 0.031–0.095 | 0 / 1 / 27; 0 / 0 / 28 | 0.055 (0); 0.101 (0) |
| 35 | **stratified Wald-t limit on a synthetic i.i.d. grid** | over in 5 of 18 cells at each delta | six strata profiles, $n_s$ 100–400, binomial strata | 0.05 / 0.10 | **0–0.111 / 0.086–0.147** | 5 / 5 / 8; 5 / 5 / 8 | 0.121 (4); 0.158 (4) |
| 36 | **`b1w` on the same grid** | over in 1 and 4 of 18 cells | the same profiles and sizes | 0.05 / 0.10 | **0–0.057 / 0.086–0.135** | 1 / 2 / 15; 4 / 5 / 9 | 0.065 (0); 0.146 (2) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 37 | stratified Wald-t limit inside the training loop, 8 strata | approximate | synthetic bandit, Lagrangian GRPO, 4 environments, the pool's own rate, 8,000 runs a cell | 0.05 / 0.10 | 0.029–0.052 / 0.072–0.100 | 0 / 1 / 3; 0 / 1 / 3 | 0.057 (0); 0.107 (0) |
| 38 | the same limit with the sampled-pool term | over in 1 of 4 cells at each delta | the same runs, the population's rate | 0.05 / 0.10 | 0.047–0.057 / 0.093–0.107 | 1 / 1 / 2; 1 / 0 / 3 | 0.062 (1); 0.114 (0) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 39 | stratified Wald-t limit, a costly label stratified on a cheap reference rate | over in 1 and 2 of 32 cells | XSTest human labels of five released models, 8 mid-rate labels, two stratifiers, $n_s$ 50–100, redrawn with replacement | 0.05 / 0.10 | 0–0.053 / 0.021–0.108 | 1 / 0 / 31; 2 / 0 / 30 | 0.055 (0); 0.111 (2) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 40 | betting mixture on user-task rates; on the pair rate | exact (bounded) | AgentDojo, 28 pipelines, user tasks redrawn across the table (a) and within suites (a_s); truth: the average over user tasks; the pair rate | 0.05 | at most 0.007; 0.005; 0.006; 0.003 | 0 / 0 / 28; 0 / 0 / 28; 0 / 0 / 28; 0 / 0 / 28 | 0.010 (0); 0.008 (0); 0.010 (0); 0.005 (0) |
| 41 | cluster bootstrap-t, by user task, redrawn within suites | approximate | AgentDojo, 28 pipelines, scheme (a_s) | 0.05 | 0.003–0.042 | 0 / 0 / 28 | 0.049 (0) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 42 | **cluster bootstrap-t by user task, injection tasks redrawn within suites** | fails | AgentDojo, schemes (b_s); (c_s) | 0.05 | **up to 0.231; 0.249** | 11 / 1 / 16; 19 / 2 / 7 | 0.244 (11); 0.262 (18) |
| 43 | two clustered margins in quadrature, redrawn within suites | approximate | AgentDojo, schemes (a_s); (b_s); (c_s) | 0.05 | at most 0.016; 0.037; 0.040 | 0 / 0 / 28; 0 / 0 / 28; 0 / 0 / 28 | 0.020 (0); 0.044 (0); 0.046 (0) |
| 44 | stratified Wald-t limit, a second model family, redrawn with replacement | approximate | Qwen2.5-1.5B-Instruct, refusal label (18–19%), 8 reference-rate strata, steps 100 and 200, $n_s$ 100–200 | 0.05 / 0.10 | 0.033–0.046 / 0.077–0.092 | 0 / 0 / 4; 0 / 0 / 4 | 0.048 (0); 0.095 (0) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 45 | `b1w`, the same | approximate | the same cells | 0.05 / 0.10 | 0.033–0.041 / 0.086–0.092 | 0 / 0 / 4; 0 / 0 / 4 | 0.043 (0); 0.095 (0) |
| 46 | the stratified Wald-t limit with the term for a sampled pool, a second model family | over in 1 of 4 cells at delta 0.10 | a claim about the prompt source, the same label, steps and sizes | 0.05 / 0.10 | 0.036–0.051 / 0.085–0.111 | 0 / 1 / 3; 1 / 0 / 3 | 0.056 (0); 0.118 (1) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 47 | stratified Clopper-Pearson: a stratified draw's pooled count plus the allocation's slack | exact (binary labels) | 116 cells: real responses (58), a costly label (40), a synthetic grid (18), drawn with replacement within strata, $n_s$ 50–400 | 0.05 / 0.10 | 0–0.048 / 0–0.093 | 0 / 0 / 116; 0 / 0 / 116 | 0.048 (0); 0.093 (0) |
| 48 | stratified betting bound (union-intersection) | exact (bounded) | the same cells | 0.05 / 0.10 | 0–0.001 / 0–0.007 | 0 / 0 / 116; 0 / 0 / 116 | 0.002 (0); 0.010 (0) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 49 | cluster bootstrap-t, by user task, against a fitted crossed model | approximate | AgentDojo, 28 pipelines, tables drawn from a fitted crossed model with injection tasks fixed (P_a); truth: the model's rate | 0.05 | 0.001–0.044 | 0 / 0 / 28 | 0.051 (0) |
| 50 | **the same, both kinds of task drawn from the model** | fails | the same, scheme (P_c) | 0.05 | **up to 0.247** | 18 / 3 / 7 | 0.260 (17) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 51 | betting mixture on user-task rates; on the pair rate, against a fitted crossed model | exact (bounded) | the same, scheme (P_a); truth: the model's user-task average; its pair rate | 0.05 | at most 0.005; 0.003 | 0 / 0 / 28; 0 / 0 / 28 | 0.007 (0); 0.006 (0) |
| 52 | pigeonhole bootstrap-t; two clustered margins in quadrature, against a fitted crossed model | approximate | the same, schemes (P_a); (P_c) for each | 0.05 | at most 0.011; 0.037; 0.016; 0.047 | 0 / 0 / 28; 0 / 0 / 28; 0 / 0 / 28; 0 / 0 / 28 | 0.014 (0); 0.043 (0); 0.021 (0); 0.054 (0) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 53 | stratified Wald-t limit, a policy that training moved far, redrawn with replacement | approximate | Qwen2.5-1.5B-Instruct trained on the guard's refusal flag alone, refusal label (26% at step 100, 58% at step 200), the second family's pool and strata, $n_s$ 100–200 | 0.05 / 0.10 | 0.043–0.049 / 0.095–0.100 | 0 / 0 / 4; 0 / 0 / 4 | 0.051 (0); 0.103 (0) |
| 54 | **`b1w`, the same** | over in all 4 cells at the 58% rate | the same cells | 0.05 / 0.10 | **0.045–0.059 / 0.098–0.112** | 2 / 0 / 2; 2 / 1 / 1 | 0.062 (2); 0.115 (2) |

**Table 4.** Miss rate against delta for the bounds used, with one rule for every row. Cells are counted as over / unresolved / at or under delta (section 6.1's rule). The next column is the largest upper end of a 95% interval for a cell's miss rate, with the cells still over after a Bonferroni correction for the cells counted with it in brackets. Bold marks a bound we do not use in that setting and that is over its level there; the registered rows 37 to 55 are not in bold, except rows 42, 50 and 54, and their second column says where they are over. "Exact" means valid at every sample size by construction; "approximate" means valid asymptotically and checked by resampling. Draws per cell are 4,000 or 5,000 except where stated: 8,000 runs for the rows inside the training loop, 10,000 for the rows on the prompt source and 40,000 for the rows redrawn with replacement. The sheets' 4,000 are 20 plantings of the labels by 200 sheets each, counted as independent draws (with the plantings as clusters no cell changes class). A bound that returns no limit is not a miss; the share of draws where it returns none, and the counts if each were a miss, are given for the bootstrap rows and AgentDojo's exact rows in the paragraph before the table [abstention]. The rows on the trained-policy harm pool are exact, with no draws: labels drawn with replacement from a 0/1 pool are a binomial count, so each miss is a finite sum. The pool's count is not recorded; the rate the companion paper prints allows 121 to 124 positives of 3,600, and each row holds every one of those counts at each of five sizes (20 cells a bound). For these rows the next-to-last column is the largest exact miss and, in brackets, the number of cells above delta; so it is for row 47, whose misses are computed exactly from the strata's binomial counts. The rows "redrawn with replacement" repeat the reference-rate-strata cells with each stratum's prompts drawn with replacement (section 6.2). AgentDojo schemes: (a) user tasks resampled, injection tasks fixed; (b) the reverse; (c) both. The rows on StratPPI's own allocations count 26 reference-strata cells: one checkpoint at a 95% rate is left out, where the proportional-allocation rows count all 28. The row for the label pushed by the Lagrangian holds that run's own cells, at $n_s$ 200; the same label at $n_s$ 100, drawn for the StratPPI comparison, is in Table A1 (largest miss 0.024 at delta 0.05). Rows 32 to 36 are the stratified Wald-t limit of section 8.1, and it and `b1w` on a synthetic grid; rows 37 to 39 the same limit inside the training loop (section 7.2) and on a costly label (section 8.1.4); rows 40 to 43 AgentDojo with an exact bound and with tasks redrawn within the benchmark's suites, schemes (a_s), (b_s) and (c_s) (section 10.2); rows 44 to 46 the strata on a second model family (section 8.1.3); rows 47 and 48 exact stratified bounds (section 8.1.4); rows 49 to 52 AgentDojo against a fitted crossed model, with injection tasks fixed (P_a) or drawn too (P_c), where only a bound that fails to compute counts as a miss (section 10.2); rows 53 to 55 the strata on a policy that training moved far from its reference (section 8.1.3). Rows 37 to 55 were registered before they were run, and so were rows 14, 30 and 31, by a commit whose order rests on its local time (Appendix D).

| # | bound | kind | setting | delta | miss, per cell | cells over / unresolved / at or under | largest 95% upper limit of a cell's miss (cells over after correction) |
|---|---|---|---|---|---|---|---|
| 55 | the stratified Wald-t limit with the term for a sampled pool, a policy that training moved far | approximate | a claim about the prompt source, the same label, steps and sizes | 0.05 / 0.10 | 0.040–0.048 / 0.085–0.094 | 0 / 0 / 4; 0 / 0 / 4 | 0.052 (0); 0.100 (0) |

Rows 37 to 55, the registered runs, are counted separately: apart from rows 42, 50 and 54, bounds that fail there, three of them have cells over (rows 38, 39 and 46, with 2, 3 and 1 cells over, of which 1, 2 and 1 stay over after the correction; interval ends up to 0.062 at delta 0.05 and 0.118 at 0.10), as their second column says. In rows 1 to 36 that are not in bold, which we treat as valid, no cell stays over its level after the correction, and the largest miss rate a cell's interval leaves open is 0.067 at delta 0.05 and 0.106 at 0.10. One cell there is over under the uncorrected rule: `b1w` on a design-weighted sheet, 0.0595 at 4,000 draws. Those 18 cells are 9 settings run twice: the other

run of the same setting gave 0.0485, and the two together give 0.054, which is unresolved. Every row in bold has at least one cell over, and every one but the quadrature bound keeps one after the correction. Most also have cells that are not over: a failing bound fails somewhere, not everywhere.

Three readings of Table 4.

1. **The Wilson-type bounds are over their level at some rates, and we can say which.** In the stored design neither `b1w` nor the pooled Wilson bound is over in any of the 24 rare-label cells at delta 0.05; at delta 0.10 two are, both at 0.126, for one harm label at the checkpoint where its rate is 1.9%, at $n_s$ 100. An exact calculation and a redraw with replacement, below, give the pattern: under one half a miss below delta on average and above it at some rates, above one half a miss above delta on average, and a peak just above the limit the bound returns when it sees no positive. Rare labels take Clopper-Pearson or a betting bound.

2. **The t-test is the wrong default.** On the one trained-policy pool we resampled (a harm rate of 3.4%) its exact miss is over its level at every size from 200 to 2,400 (0.104 to 0.195 at delta 0.10), and its width is zero at a rate of 0 or 1, which lets candidate selection win by driving a subgroup rate to the boundary. A policy with a true rate of 1% passes a 0.5% threshold whenever 100 samples hold no positive, which is 37% of the time. The failure is not particular to that pool. By enumeration over binomial counts, the Student-t limit's miss probability is above delta at 96% or more of 79 rates from 0.5% to 20%, at each of five sample sizes from 200 to 2,400 and at both deltas, and Clopper-Pearson's is above it at none.

3. **"Approximate" has a number attached, and it depends on the design.** In the designs they were run in, the approximate bounds we use are over their level in 1 of 378 cells and unresolved in 24; the largest miss at delta 0.05 is 0.060. The 378 are the cells of the seven approximate rows among rows 1 to 36 of Table 4 that are not in bold, not redrawn with replacement and not for the prompt source, leaving out the pigeonhole bootstrap-t, which no certificate in this paper uses. The registered rows 37, 41 and 49, the same limits in new designs, add 64 cells, none over and 2 unresolved. The grouping was made after the results. They are not 378 separate studies: the delta 0.10 cells of the two `b1w` rows and of the Wald-t row reuse the draws of their delta 0.05 cells, the 168 PPI++ cells are 42 sets of draws read through four judge features, and the reference-rate cells of the StratPPI and Wald-t rows share their seeds with the `b1w` rows. Two things qualify the count for `b1w`. It is for one setting of the strata, 8 reference samples and 8 strata; at the other eleven settings in the same file `b1w` is over in 8 of 528 mid-rate cells, all at $n_s$ 100 on the two labels at 65% and above. And it is for a safety set that is 20–40% of its pool; redrawn with replacement `b1w` is over in 7 of 28 cells, by at most 1.2 points at delta 0.05, where the StratPPI estimator with a bootstrap-t limit and a stratified Wald-t limit are over in none (the bootstrap-t limit partly by returning no limit; section 7.1). On a synthetic i.i.d. grid (36 cells, 4,000 draws) `b1w` is over in 5 cells and the Wald-t limit in 10. Inside the training loop on a synthetic environment with 4 strata the stratified `b1w` test reached 0.116 at delta 0.1 on 500 runs, which is unresolved, and the Wald-t limit 0.132, which is over; with 8 strata and 8,000 runs a cell the Wald-t limit was over in 2 of 16 cells, both with the sampled-pool term in the least heterogeneous environment (section 7.2). On real responses with 8 strata the Wald-t limit was over in no mid-rate cell for the pool's own rate, on Granite or on a second model family, and, for the prompt source, in one rare-label cell on Granite and one mid-rate cell of the second family (0.111 at delta 0.10) (section 8.1.3); for a human label stratified on another model's reference rate it was over in 3 of 64 cells, by at most 0.008 (section 8.1.4); on synthetic data it is over in 10 of 36 cells.

**The Wilson limit, exactly.** For an i.i.d. sample the Wilson limit's miss probability can be

computed by enumeration. It jumps just above each value the limit can take and falls until the next. Just above the zero-count limit (2.6% at n 100 and 1.3% at n 200, at delta 0.05) it is 0.069 and 0.068, close to exp $(-z^2)$ at any n: 0.067 at delta 0.05 and 0.194 at delta 0.10, where the zero-count limit at n 100 is 1.6%. Averaged over rates, the miss is under delta below one half and over it above. At n 100 and delta 0.05 it averages 0.042 over rates of 5–20% and 0.056 over 80–95%, with largest values of 0.063 and 0.085, and just under a rate of 1 it reaches 0.20. Clopper-Pearson's miss probability never exceeds delta.

**Redrawn with replacement.** The stored cells hide that pattern, because they draw 20–40% of a finite pool (section 6.2). With replacement, `b1w` is over its level in 7 of 28 mid-rate cells at delta 0.05 (largest miss 0.062), all on the two labels at rates of 65% and above, and in none of the 16 cells at rates of 9–18% (largest 0.034). The pooled Wilson bound is over in 6 of 28 (largest 0.069), two of them at rates of 15–18%. Each of the two is over in 1 of the 12 rare-label cells at delta 0.05, the same harm label at a checkpoint where its rate is 1.4%, at $n_s$ 200 (0.055 and 0.064), and in 3 of 12 at delta 0.10 (up to 0.138 and 0.142). Clopper-Pearson, exact on those draws, is over or unresolved in none (in a first pass at 5,000 draws it was unresolved in three cells, largest 0.053 at delta 0.05 and 0.102 at 0.10), and on the random draw the simulated misses of both bounds agree with the binomial calculation.

The StratPPI rows (Fisch et al., 2024) are our implementation of its estimator and interval on the same pools as the rows above them, and for the reference-rate strata on the same draws; sections 8.1 and 8.2 give the comparison.

### 7.2 Validity under the full pipeline

Table 4 tests bounds on a fixed population. The certificate also has to survive selection: the candidate is the best-looking checkpoint of a training run.

**Synthetic environment, exact truth.** In a contextual bandit where every policy's true rate is computable, the probability of returning a policy that truly violates was 0.000–0.002 against a delta of 0.1 for the Lagrangian variant of the Seldonian arm under four bounds, against 0.830 for unconstrained training at the same reward pressure. In two one-way sweeps of that variant it stayed at or below 0.002, across reward pressures 0 to 4 at 1,000 prompts and across sample sizes 200 to 5,000 at pressure 1 (the second with the Student-t bound; the sizes are total prompts, 40% of them in the safety set). The filter-only variant was higher and still under delta: 0.028 at 200 prompts and 0.004 at 1,000 and 5,000. With a noisy judge the guarantee held at the judge's level in every row, while a judge that misses 40% of violations let the true violation rate among returned policies reach 0.059, against 0.002 with an exact judge (0.038 of all runs, still under delta here): the certificate is about the label.

**Adversarial splits.** Rerandomised and stratified candidate/safety splits did not break the exact test, across 2,000 seeds per arm, eleven split rules and candidates built to overfit the balancing statistics, on a classifier with a constraint on a difference of true-positive rates: with Clopper-Pearson a candidate that passed and truly violated was returned in at most 1 of 2,000 runs in any cell. That test is too conservative there to show a leak, so the sensitive check is a tight Wald test on the same runs (a one-sided normal-approximation test on the difference, which covers at about delta on fresh data, so that a leak would show), whose bound fell below the truth in at most 0.055 of runs at delta 0.05; it was unresolved above delta in 3 of 42 cells.

**The training loop, on a synthetic environment.** With the training code of our language-model runs, applied to a synthetic bandit at four levels of between-prompt heterogeneity, with 4 strata and 500 runs per cell, the stratified `b1w` test, with its added term for a pool that is itself a sample and the population's rate as the truth, missed 0.084–0.116 at delta 0.1 (Monte Carlo

standard error 0.013): three cells at or under delta and one unresolved above it. The stratified Wald-t limit missed 0.090–0.132 in the same runs, over its level in one of the four cells (0.132, against a band that ends at 0.127). A random split with a pooled bound missed 0.090–0.100, which 500 runs cannot tell apart from the stratified test. The configuration section 8.1.5 recommends, 8 strata with the Wald-t limit, was then registered and run in the same loop at both deltas, 8,000 runs a cell [O2]. For the pool's own rate it was over its level in none of the 8 cells (2 unresolved; largest miss 0.052 at delta 0.05 and 0.100 at 0.10). With the sampled-pool term, for the population's rate, it was over in the least heterogeneous environment at both deltas (0.057 and 0.107; we had predicted the cell at delta 0.10, and at most two of the eight), and in no other cell; `b1w` with the term and the pooled bound on a random split were over in none. Its $\mathrm{ESS}_{\mathrm{est}}$ against that pooled bound rose with the correlation of reference and trained per-prompt rates, from 1.1 at 0.43 to 1.7 for the population's rate and 3.1 for the pool's own at 0.98. No language model is in this check.

**Language-model policies.** On an over-refusal constraint (Qwen2.5-0.5B, three seeds), unconstrained training and a fixed penalty of 1 ended over the threshold in 3 of 3 seeds, a penalty of 4 in 1 of 3; the Seldonian arm ended over it in 0 of 3 and certified 2 of 3. On a brevity constraint, 10 of 10 seeds certified and none ended over its threshold. Three and ten seeds cannot estimate a miss rate; they are consistent with the level, and the resampling studies above are the evidence.

**Trajectories.** On a synthetic bandit, a test of the returned policy alone misses drift through a forbidden region during training: 65% of plain-Lagrangian runs passed it after a step inside the region (50% when checked every 10 steps). A certificate at level delta/T over every one of T checks was at or under delta in all 10 cells, with misses of 0.025–0.100 against a delta of 0.1, at 200 runs a cell, which resolve only a miss above 0.14.

# 8 Results: what a label budget buys

## 8.1 Making the sample go further: reference-rate strata

#### 8.1.1 Design and first pools

**Idea.** Sample the reference $k$ times per prompt, cut the prompts into $H$ equal strata of their reference rate, sample the safety set within strata, and bound the rate with `b1w` (sections 6.3 and 6.4).

**Result.** With k = 8 and H = 8 on real Granite-3.3-2B responses, $\mathrm{ESS}_{\mathrm{est}}$ against a random split at delta 0.10 was 2.4 for over-refusal (rate 17%, $\mathrm{ICC}_{\mathrm{ref}}$ 0.72), 5.1–5.3 for refusal of plainly harmful requests (66%, 0.86), and 1.4 for non-refusal of encoded requests (9%, 0.50), with coverage as in Table 4; at delta 0.05 it was 2.30–2.35, 4.79–5.14 and 1.39–1.41. Placebo strata, cut on a randomly permuted reference rate (no information), gave 1.00 to 1.04 with 4 strata on every label but the one at 93% (0.83 to 1.02). These are `b1w`'s gains for the pool's own rate; on the 66% label it is over its level once the pool is large (section 7.1). When the Lagrangian pushed the stratified label itself, $\mathrm{ESS}_{\mathrm{est}}$ was 2.1–2.2 at $n_s$ 200 and delta 0.10 (2.05–2.16 at 0.05), with no compression of the between-prompt spread (ICC ratio 1.01) and the order of prompts kept (omega 0.92), in one run whose rate ended within 0.4 to 3.4 points of the reference's; a push that moves the rate far was not seen in that run, and section 8.1.3 reports one.

**The rule was chosen on these pools.** The rule we registered in advance collapsed to one to three strata here, because most prompts have an 8-sample reference rate of exactly 0 or 1, and did worse (Appendix D). Equal rank strata with random ties were compared with it on the same pools and adopted, and the truth used for coverage, which carried a fixed offset shared by every draw, became the mean of the half of the responses the draws come from. These gains were thus measured on the data that chose the rule.

**A claim about the prompt source.** If the claim is about the source the 500 prompts were drawn from, the pool is itself a sample and the gain is capped (section 6.4). Each replication of this case draws a pool of the same size from the stored one, rebuilds the strata and draws the safety set from it; the truth is the stored pool's rate, at 10,000 replications a cell. With the term for a sampled pool (section 6.3), `b1w` is over its level in none of the 28 mid-rate cells at delta 0.05 (4 unresolved, largest miss 0.052) and in one at 0.10 (0.112, on the 93% label); the Wald-t limit is over in none at either. On the two rare labels `b1w` is over in 1 of 12 cells at delta 0.05 and 3 at 0.10, with or without the term, as in section 7.1. The gain, $\mathrm{ESS}_{\mathrm{med}}$ at delta 0.05 for safety sets of 100 and 200, is 1.83 and 1.54 for over-refusal (1.68 and 1.44 when the Lagrangian pushes the label), 2.36 and 1.75 for refusal of harmful requests, and 1.35 and 1.25 for non-refusal of encoded requests. For the pool's own rate the same measure gives 2.39 and 2.47, 2.11 and 2.12, 4.66 and 5.14, and 1.50 and 1.52. At the 93% rate (93% to 95% by checkpoint) the strata lose (0.88 and 0.93), and at rare rates they gain nothing (1.01 to 1.05). The baseline is itself over its level in 7 of these 28 cells, which understates the gains there. The cap, with $G$ from `b1w`'s $\mathrm{ESS}_{\mathrm{truth}}$ with replacement, gives 1.84 and 1.54, 1.71 and 1.46, 2.68 and 1.92, and 1.35 and 1.24: met at the lower rates, missed by about a tenth at 66%. At delta 0.05 `b1w` without the term is over in 17 of the 28 cells (misses up to 0.170), and the bootstrap-t StratPPI limit in 21 (up to 0.166): its variance has a term for the unlabelled sample inside fixed strata and none for strata rebuilt on each pool.

**Where it does not help.** Rare labels ($\mathrm{ESS}_{\mathrm{est}}$ 1.01–1.09 at 1–2%; on a synthetic grid exact stratified bounds beat their own pooled form in one profile only), labels near 0 or 1 (at 93%, $\mathrm{ESS}_{\mathrm{truth}}$ 0.96 and 1.12 for the pool's own rate, Table A1), and task strata on a benchmark with 20 trials a task, where the positives set the bound.

**The pre-flight** (section 6.4) ranks the gain from the k reference samples and the trained policy's two quantities, measured on responses the draws do not use: Spearman 0.84 between $\mathrm{ESS}_{\mathrm{pred}}$ and the realised $\mathrm{ESS}_{\mathrm{est}}$ over 432 cells on real data at delta 0.10, and 0.83 at 0.05. It over-predicts the size (Figure 1): at $\mathrm{H} = 8$ by 2% to 20% on the three labels at 9% to 66% at the last checkpoint and delta 0.10, by up to 30% over all checkpoints and both deltas, and by 18% to 98% on the 93% label and the rare ones. Of the two criteria we registered, the error criterion failed on real data and the Spearman criterion holds only under the adopted strata rule (Appendix D). We report the ranking as the usable part.

#### 8.1.2 Against StratPPI

StratPPI (Fisch et al., 2024; section 6.3), the published method closest to this one, ran in our implementation on the same strata and the same 5,000 draws per cell, with the reference rate as the predictor (Figure 2, left; Table A1); gains are $\mathrm{ESS}_{\mathrm{truth}}$ at delta 0.05.

1. **StratPPI's bound is shorter, and part of that is the normal limit running over its level.** It is over its level in 4 of 10 cells at delta 0.05 when a cell is the largest miss over checkpoints (one of them marginally, at 0.056), and in 9 of 28 cells counted by checkpoint, most at the 9% label.

2. **With an honest limit the two are close.** With labels allocated in proportion to stratum size, the same estimator with a bootstrap-t limit is over its level in none of the ten cells (largest miss 0.045), and its median ESS is 2.10 against 2.16 for `b1w`. The repair depends on the allocation (Appendix A).

3. **At a safety set of 100 with 8 strata the gain is the stratification.** There the within-stratum regression on the reference rate adds nothing once the limit is valid; at a safety set of 200 or with 4 strata it adds 0.4 to 0.8 of ESS (Appendix A).

4. **Which bound to use depends on the pool and the rate.** For the pool's own rate, with

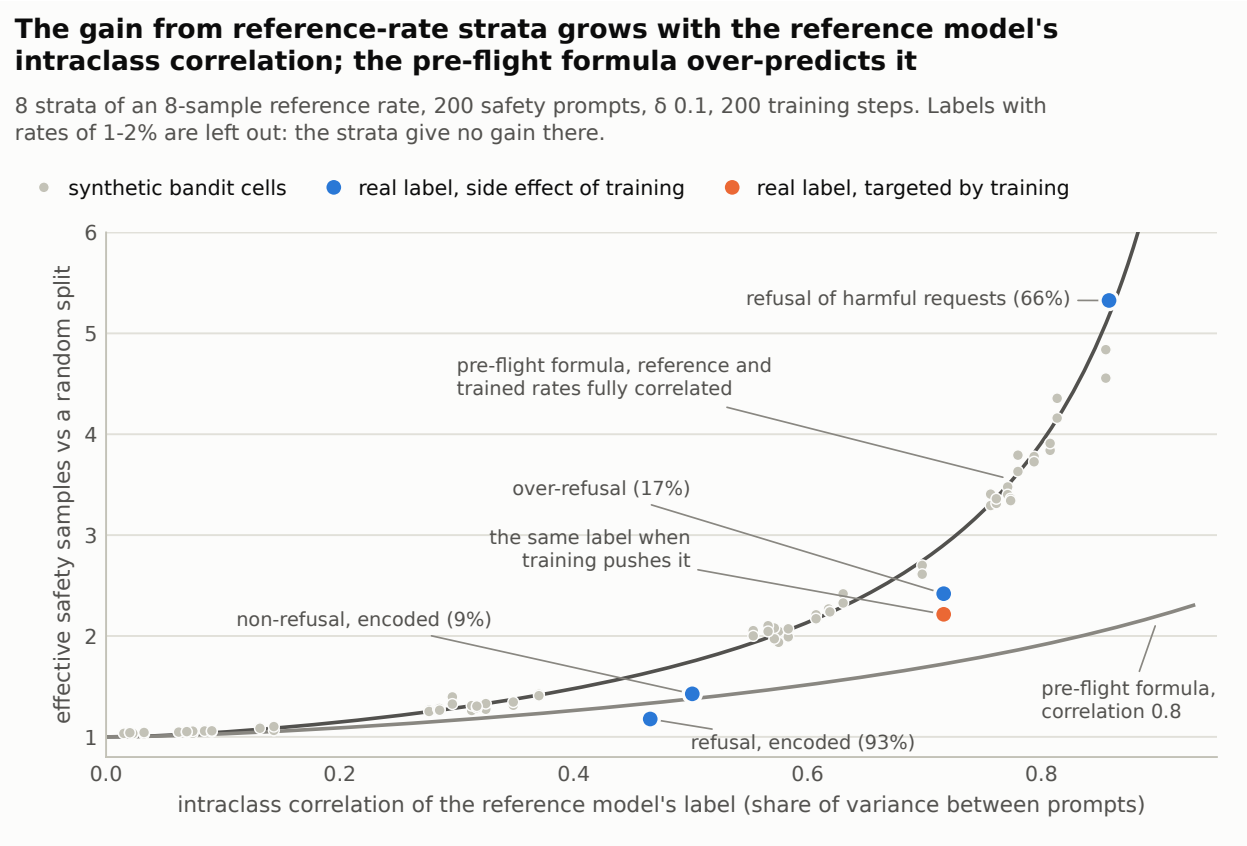



**Figure 1.** Gain from reference-rate strata against the reference model's intraclass correlation.

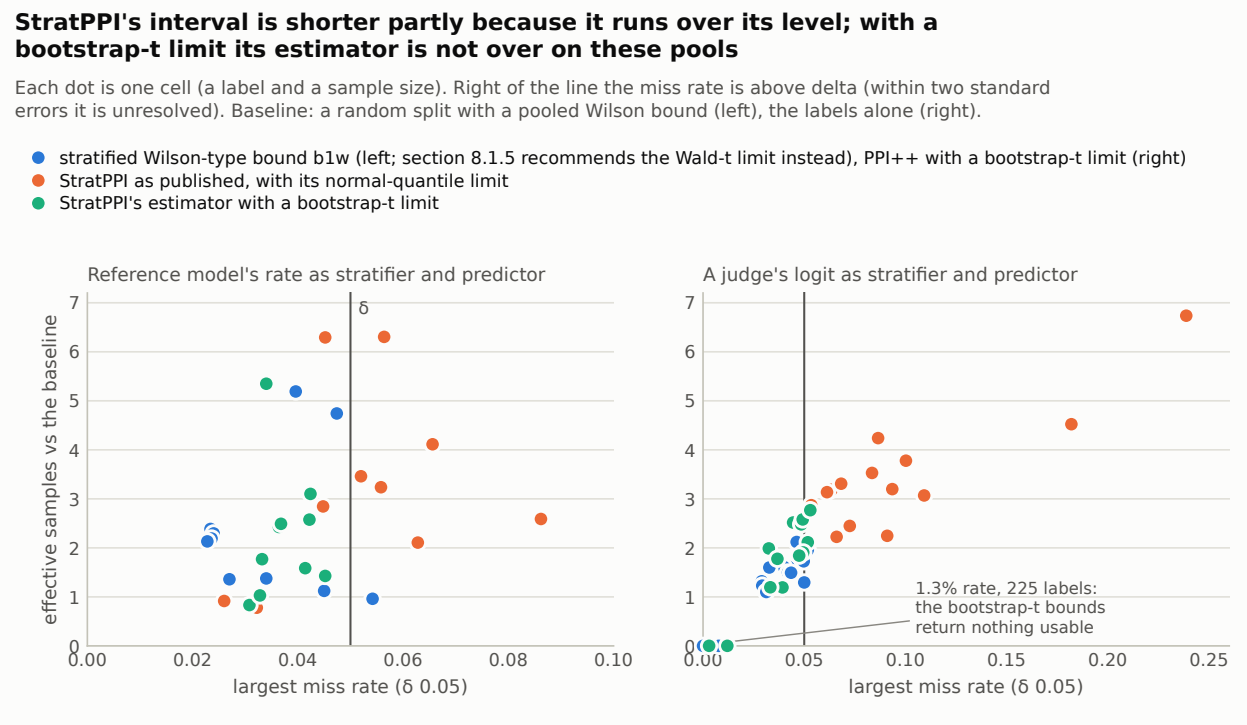



**Figure 2.** StratPPI's normal limit and the same estimator with a bootstrap-t limit, with labels allocated in proportion to stratum size. Left: reference-rate strata. Right: strata on a judge's logit.

a safety set of 20–40% of the pool, `b1w` is over its level in none of the 28 mid-rate cells of Table A1's draws (five labels, two sizes, by checkpoint) at either delta (3 unresolved at delta 0.05, on the 93% label) and in 1 of its 12 rare-label cells at delta 0.10 (0.126). Redrawn with replacement it is over in 7 of 28 at delta 0.05 and 6 at 0.10, all at rates of 65% and above, and the bootstrap-t StratPPI limit and the stratified Wald-t limit are over in none (section 7.1; the bootstrap-t limit partly by returning no limit); the gains carry over to that redraw (Appendix A). For a pool much larger than the safety set, then, `b1w` kept its level at 9–18% and not at 65% and above on these pools. No label between those rates was tested on these pools, and the reading was made on the cells it describes.

#### 8.1.3 Confirmation on new prompts

The choices of sections 8.1.1 and 8.1.2 were made on the pools that measure them, so we registered eight predictions (Appendix D) and tested them on three pools of 400 prompts that no earlier pool, training set or result had used: benign prompts of OR-Bench's 80K set, its hard-1K set and its toxic set, with the model, strata rule, checks and scoring code above and, as the trained policy, the returned checkpoint (step 175) of section 9.2's constrained run. Its refusal rates came out at 12%, 82% and 98%, so no label fell between 18% and 65%, the range no earlier pool had tested either; two labels from the guard's safety field were rare (3.2% and 1.6%). Six predictions were kept and two refuted. Below, "a large pool" is the redraw with replacement (40,000 draws a cell), "the source" the claim about the prompt source (10,000 replications), and a gain is $\mathrm{ESS}_{\mathrm{truth}}$ at delta 0.05.

- *Kept.* With a large pool `b1w` is over its level in none of the 4 cells at the 12% rate and in all 4 at 82% (misses 0.062 at delta 0.05 and 0.111–0.113 at 0.10), and its gain is within 20% of the pre-flight's prediction (below). For the source the limits without the sampled-pool term fail

again (`b1w` in 3 of 4 cells at delta 0.05, the bootstrap-t StratPPI limit in 4 of 4), and `b1w`'s gain with the term is 1.20 to 1.32, within 15% of the cap. On the rare labels the strata gain little (1.01 to 1.11 for `b1w`, 0.86 to 1.20 for the Wald-t limit) and Clopper-Pearson is over in no cell; the Wald-t limit with the term is over in one rare-label cell for the source (0.110 at delta 0.10, with a safety set of 200).

- *Refuted.* With a large pool the bootstrap-t StratPPI limit is over in 1 of 8 mid-rate cells (0.105 at delta 0.10 on the 82% label), and in one more at the 98% rate (0.0524 at delta 0.05, above the 0.0522 at which 40,000 draws count a cell as over); the stratified Wald-t limit is over in none of the 8 (misses at most 0.039 at delta 0.05 and 0.082 at 0.10). And for the source `b1w` with the term is over at the 82% rate with a safety set of 100 (0.058 at delta 0.05 and 0.117 at 0.10; unresolved at 200 and delta 0.05, 0.052), where the Wald-t limit with the term is over in none of 8 (at most 0.047 and 0.088), with a gain of 1.31 and 1.32 at 12% and 0.75 and 0.89 at 82%. Each refuted prediction named the Wald-t limit beside the limit that failed, and its half of each held.

**A pool aimed at the gap.** No part of OR-Bench that we sampled is both in that range for this policy and large enough to make a pool. By prompt category the three pools' refusal rates run from 0% to 22%, from 46% to 89% and from 87% to 100%; three categories fall inside the range, with 47, 20 and 17 prompts. We therefore registered a fourth pool separately, as a fixed mixture: the 193 hard-1K prompts still unused and 207 unused benign prompts of the 80K set, with ten predictions (Appendix D). It is weaker evidence: its rate was aimed at, not found; its predictions were written after the same analysis of a re-mix of the first two pools' responses in the same proportions, which shaped three of them (Appendix D); as finally worded all nine that could be tested already held on the re-mix; and the reference rate largely sorts the two sources into different strata, so part of the gain is that of knowing the source: the variance of a random draw is 1.93 times that of a stratified one with two strata, the two sources, and 2.90 times with the 8 strata of the reference rate, so the source alone removes about three quarters of what the 8 strata remove.

The refusal rate came out at 46% (81% on the hard prompts, 12% on the others). Nine predictions were kept, none was refuted, and one had no label to test it. With a large pool the Wald-t limit is over its level in none of its 4 cells (misses 0.026 and 0.036 at delta 0.05), the bootstrap-t StratPPI limit in none (three cells unresolved, the largest miss 0.001 above its level), and `b1w` in none (0.044 and 0.045 at delta 0.05, 0.096 at 0.10). On the re-mix, at a rate of 48%, `b1w` sat at its level (0.051 and 0.102, unresolved). The gain of `b1w` is 2.78 and 2.97 against a pre-flight of 2.79, and that of the Wald-t limit 2.09 and 2.56. For the source the Wald-t limit with the sampled-pool term is over in none of 4 cells, with a gain of 1.49 and 1.34, 7% to 9% under its cap; `b1w` with the term is over in none either (one cell unresolved, 0.102 at delta 0.10), and the limits without the term fail in both cells at delta 0.05 (misses 0.079 to 0.120). The baseline is itself over its level at a safety set of 200 and delta 0.05 (0.053, and 0.057 for the source), so the gains at 200 are if anything understated. The safety-field label was rare (1.9%) and the strata gained nothing on it (1.00 and 1.08); there `b1w` and the pooled Wilson limit are both over at a safety set of 100 and delta 0.10 (0.140 and 0.142), and Clopper-Pearson is not.

Three things follow. First, the gains are smaller than on the pools that chose the rule. On new prompts from one source a limit that kept its level gained 1.5 to 1.9 at 12% (`b1w` 1.53 and 1.62, the Wald-t limit 1.47 and 1.73; the bootstrap-t StratPPI limit, which also held there (its returns of no limit there were not counted), 1.55 and 1.88) and 0.86 and 1.13 at 82% (the Wald-t limit; `b1w`'s 1.49 and 1.62 there come from a limit over its level in all four cells). The same measure, also redrawn with replacement, gives 1.4 to 5.1 for `b1w` at rates of 9% to 66% on the pools that chose the rule, the upper end at 66%, where `b1w` is also over its level with a large pool. Second, the pre-flight sizes the

gain only in the mid range and only with the trained policy's two quantities measured, on 8 further responses per prompt that the draws do not use. At all three mid rates, 12%, 46% and 82%, the pre-flight was within 20% (at 12% and 82% `b1w`'s gain was 2% to 16% under it). From the reference samples alone, with omega $= 1$, it predicts 2.58, 4.12 and 2.22, and the realised gains are 27% to 41% lower. Even with the measured quantities it gives 2.67 at the 98% rate, where with a large pool the strata lose under every limit (0.55 and 0.63 for `b1w`, against a baseline itself over its level at a safety set of 100), and 2.47 on a rare label where nothing is gained. Third, with 8 strata the stratified Wald-t limit is the only approximate stratified limit that was over its level in no mid-rate cell on Granite's pools, here and on the earlier ones, and at a high rate it pays for holding: its gain at 82% is 0.86 and 1.13 with a large pool and 0.75 and 0.89 for the source, against 1.47 and 1.73 at 12% and 2.09 and 2.56 on the mixed pool at 46%. With 4 strata it was over in 1 cell of 26 (0.062 at $n_s$ 100 and delta 0.05, on the 9% label), the 13 label-checkpoint cells of Appendix A's sweep at safety sets of 100 and 200; on synthetic data it did not hold (section 7).

**A second model family.** Every pool above comes from Granite-3.3-2B, whose trained policy kept the reference's order of prompts. A further registered run [O6] trained Qwen2.5-1.5B-Instruct with GRPO and LoRA for 200 steps on 1,600 unused OR-Bench-80K benign prompts, with a reward that added three times the guard's refusal flag on five prompt categories and subtracted it on the other five, to move the trained policy's order of prompts away from the reference's. It did not move it. The pool's refusal rate went from 19.7% to 19.1%; it fell on the categories the reward pushed up (16.9% to 14.8%) and rose on those it pushed down (22.5% to 23.5%), and rho, the correlation between reference and trained per-prompt rates corrected for sampling noise, was 0.84 at step 200, where we predicted under 0.8. Over training the reward model's score rose by about 5 points while the refusal rates of the training batches stayed near 20% on both sets of categories [O6]. By its registration the run is therefore a second model family, not a policy moved far from its reference; the run after it moved one. On a new pool of 400 benign prompts, with 8 reference responses a prompt for the strata and 16 of each checkpoint, at steps 100 and 200:

- with a large pool the Wald-t limit is over its level in none of its 8 cells (misses at most 0.046 at delta 0.05 and 0.092 at 0.10), and `b1w` in none (at most 0.041 and 0.092), where StratPPI's normal limit is over in all 8 and the pooled Wilson baseline in 4;
- at step 200 the Wald-t limit's gain is 1.52 and 1.77 at safety sets of 100 and 200, the second above the 0.9 to 1.6 we predicted for a safety set of 200, and `b1w`'s 1.60 and 1.70 against a pre-flight of 1.73 with $\mathrm{ICC}_{\mathrm{cand}}$ and omega measured; from the reference samples alone the pre-flight predicted 1.55, or 1.30 with the pushed-label rho;
- from step 100 to step 200 rho fell from 0.88 to 0.84 and the Wald-t limit's gain at a safety set of 200 from 1.84 to 1.77;
- for the prompt source the Wald-t limit with the sampled-pool term is over in 1 of 8 cells (0.111 at delta 0.10, a safety set of 200, step 200) and `b1w` with it in none, while `b1w` without the term is over in 6 of 8.

Three of the six predictions were kept; the three refuted are the two checks that the push moved the policy and the upper end of the gain (Appendix D).

**A policy moved far.** A further registered run [B3] repeated that run with the reward model removed: the reward was the guard's refusal flag alone, positive on the five categories and negative on the other five, with the same model, training settings, prompts, pool and strata. This time the policy moved. At step 200 the pool's refusal rate was 58.0%, against the reference's 19.7%, and rho fell to 0.46 (0.87 at step 100), where we predicted under 0.7. It did not move as the reward asked:

refusal rose on both sets of categories taken together (and in 9 of the 10 categories), from 16.9% to 67.4% on those pushed up and from 22.5% to 48.7% on those pushed down, where we predicted a fall of at least 10 points. So the registered check that the push worked as intended failed. Our registration read such a failure as meaning "a second training run on the same family, not a policy moved far", a reading written for a run that did not move; here the rate nearly tripled and the order of prompts, which is what the strata depend on, moved further than in any other run in this paper. We therefore report the run as a policy moved far, against our registered reading. At step 200, at a refusal rate of 58%, with a large pool:

- the Wald-t limit is over its level in none of its 8 cells over both steps (misses at most 0.049 at delta 0.05 and 0.100 at 0.10), as predicted for the 4 at step 200; the strata are fixed before the safety set is drawn, so a reordered policy costs the limit width, not validity;
- `b1w` is over in all 4 cells at step 200 (up to 0.059 and 0.112) and in none at step 100 (26%; one unresolved), in line with section 8.1.5's reason for not making it the default; the pooled Wilson baseline is over in 4 of 8 cells;
- the gain has gone: the Wald-t limit's is 0.97 and 1.08 at safety sets of 100 and 200, where we predicted under 1.3, against 1.36 and 1.46 at step 100. The pre-flight with $\mathrm{ICC}_{\mathrm{cand}}$ and omega measured on the trained policy's responses gives 1.14, and `b1w`'s 1.22 is within the 25% we predicted of it; from the reference samples alone, before training, the pre-flight predicted 1.55, or 1.30 with the pushed-label rho;
- for the prompt source the Wald-t limit with the sampled-pool term is over in none of 8 cells (at most 0.048 and 0.094), `b1w` with it in 2, and `b1w` without it in 7.

Four of the five predictions were kept; the one refuted is the check of the push's direction (Appendix D). This is one run of one small model on one label. Where training reorders the prompts, the reference-rate strata stayed valid with the Wald-t limit and saved almost nothing, and the pre-flight run before training did not foresee it.

#### 8.1.4 What the gain costs

The strata are built from k = 8 judged reference responses on each of the pool's 500 prompts, 4,000 in all, and the ESS counts only the safety set's labels; an $\mathrm{ESS}_{\mathrm{est}}$ of 2.4 is worth 140 labels at $n_s$ 100. The method saves labels only where the label on the trained policy costs far more than the stratifier's, for instance a human label stratified on a guard's reference rate. A registered test of that case [O3] used XSTest's published human labels on the completions of five released model variants for its 250 safe prompts, with the guard's rate over 8 Granite-3.3-2B reference responses, a model none of the five is, as the stratifier. On the eight mid-rate labels (strict and broad refusal, rates of 6% to 60%), at a safety set of 100 drawn with replacement within strata, the Wald-t limit's $\mathrm{ESS}_{\mathrm{truth}}$ at delta 0.05 against the pooled Wilson bound was 0.97 to 1.34 (median 1.18), and `b1w`'s 1.10 to 1.36 (median 1.24), the strata's own gain, since `b1w` at one stratum is that bound; but `b1w` was over its level in 8 of the 32 mid-rate cells drawn with replacement at delta 0.05 and 11 at 0.10, at rates from 10% to 60%, two of them among the eight behind those gains. The pooled Wilson baseline is itself over its level in 6 and 8 of these 32 cells. A stratifier from the guard's scores of the other model families' completions did about as well (medians 1.19 and 1.25). The Wald-t limit was over its level in 3 of the 64 cells drawn with replacement (both stratifiers, safety sets of 50 and 100, both deltas; all three at 100), by at most 0.008, and in none of the 64 drawn without replacement; on the two rare labels the strata gained nothing under `b1w` and lost under the Wald-t limit. At a gain of 1.2 the strata save about 17 of 100 human labels for 2,000 generated and judged reference responses, so they pay where a human label costs more than about 120 judged responses. The labels

are of released models, not of a policy trained from the reference. Where the strata and the label are the same cheap judge, enlarging the safety set is the better use of the budget.

**What exactness costs, and the chance of certifying.** Every gain above is a width ratio against the pooled Wilson bound, which has no finite-sample guarantee, and a width is not a verdict. A registered check (Appendix D) [B25] puts two exact stratified bounds on 116 cells, all drawn with replacement within strata: the 58 real-response cells of sections 8.1.1 to 8.1.3 (not the run that moved the policy far), the 40 costly-label cells and 18 synthetic ones. Each bound is measured against Clopper-Pearson on a random draw of the same size. The first exact bound is Clopper-Pearson on the stratified draw's pooled count plus the allocation's slack, the sum over strata of max(0, W_h - n_h/n). It keeps its level because a sum of binomials with unequal rates is less spread than one binomial with their mean (Hoeffding, 1956); computed exactly, its miss is at most 0.048 at delta 0.05 over all 116 cells. The second is a stratified betting bound of the union-intersection kind (Spertus and Stark, 2022; section 12), which we had tried before on synthetic data only. To measure the chance of certifying, we set a threshold at which Clopper-Pearson on the random draw passes with probability 0.8. A bound's power ESS is then the number of random labels Clopper-Pearson would need to pass as often at that threshold, divided by the number of labels the bound uses (Clopper-Pearson scores 1). On the 36 real-response cells at rates of 5% to 95%, at delta 0.05:

- the stratified Clopper-Pearson bound has a width gain of 0.82 against Clopper-Pearson (median; 0.50 to 1.00) and a power ESS of 1.11 (median; 0.27 to 1.83); it certifies more often than Clopper-Pearson in 19 of the 36 cells. At the lower threshold, the one Clopper-Pearson passes with probability 0.5, its power ESS is 0.67;
- the betting bound has a width gain of 0.20 and a power ESS of 0.05;
- the Wald-t limit has a width gain of 2.02 against Clopper-Pearson (0.57 to 3.95; 1.86 against the pooled Wilson bound) and a power ESS of 1.88 (median; 0.47 at the lowest).

On the 32 costly-label cells in that range the stratified Clopper-Pearson bound's power ESS is 0.88 (median) and the Wald-t limit's 1.22, and on rare labels the stratified bound loses (0.69 on the real cells); on the costly label it certifies more often in none of the 32 cells. Four of the eight predictions held: neither exact bound was over its level in any cell, the betting bound was narrower than Clopper-Pearson in no cell and certified as often in no real or costly cell at a rate of 5% or more, and the Wald-t limit's gains were larger against Clopper-Pearson than against the pooled Wilson bound. We had predicted a median power ESS of at least 1.10 for the stratified Clopper-Pearson bound over the 68 real and costly cells at rates of 5% to 95%, none under 0.95, values near 1 at the threshold passed with probability 0.5 and on rare labels, and the Wald-t limit above it in every one of the 36 real cells in that range; the median was 0.92, the lowest 0.27, and the Wald-t limit was above it in 35 of 36. Part of the shortfall is our threshold, which sits on one of Clopper-Pearson's own values and so favours it. Spread evenly over that step instead (a check written after the results [B25 sensitivity]), the stratified bound's median is 1.00 over the 68 real and costly cells (lowest 0.42), the Wald-t limit's 2.10 on the real ones, and the pooled Wilson bound's 1.07; there the stratified bound's power ESS is above 1 in 26 of the 36 real cells and in 8 of the 32 costly ones. The gain of section 8.1 belongs to the approximate limits: at the registered threshold the betting bound never certified more often than Clopper-Pearson on a random draw of the same size, and the stratified Clopper-Pearson bound did so in 19 of the 36 real-response cells and in none of the 32 costly ones, so neither gives a gain to rely on. The pooled Wilson baseline, computed exactly, is over its level in 10 of the 36 real-response cells, and against Clopper-Pearson the Wald-t limit's width gains are larger in every one of them, so the baseline understates those gains.

#### 8.1.5 The rule in one place

For a safety set stratified by 8 equal rank strata of an 8-sample reference rate:

- *a label that may be rare (under about 5%) or near one (above about 95%):* no strata; Clopper-Pearson or a betting bound on a random draw (section 7.1). The pre-flight refuses a reference rate outside 5% to 95%;
- *a mid-rate label (about 5% to 80%), for the pool's own rate:* the stratified Wald-t limit; `b1w` is narrower at a safety set of 100 and at 46%;
- *a mid-rate label, for the prompt source:* the stratified Wald-t limit with the term for a sampled pool;
- *from about 80% to 95%:* the strata gained little or lost under this limit ($\mathrm{ESS}_{\mathrm{truth}}$ 0.75 to 1.13 at 82%, 0.43 and 0.65 at 93%), so a random draw serves as well;
- *where a finite-sample guarantee is required:* Clopper-Pearson on a random draw. The exact stratified bounds we tried gave no reliable gain over it (section 8.1.4);
- *where training may have reordered the prompts:* the same limits; in the one such run they stayed valid and gained almost nothing (0.97 and 1.08 at rho 0.46), and only a pre-flight on the trained policy's responses foresaw that (section 8.1.3).

`b1w` is not the default because, once the pool is much larger than the safety set, it was over its level at rates of 65% and above on Granite's pools (sections 8.1.2 and 8.1.3), at 58% on the policy that training moved far (section 8.1.3) and at rates of 10% to 60% on the costly label of section 8.1.4; its excess is not simply increasing in the rate: at 98% it is over in none of its 4 cells. At 46% the Wald-t limit kept 75% to 86% of `b1w`'s gain; at 82% it gained little or lost (section 8.1.3), against a pooled Wilson baseline that is itself over its level there at a safety set of 100 (against Clopper-Pearson, which held, the large-pool figures are 1.06 and 1.32). Each line rests mainly on one model's trained policy, one family of prompts and a guard's label, with one Granite rate between 18% and 65% (from a mixture), one check on a second model family at 18–19%, one on a policy that training moved far (26% and 58%) and one costly human label on released models (section 8.1.4), and none of the stratified limits has a finite-sample guarantee.

## 8.2 What a judge buys: a routing rule

A judge model scores every response cheaply; gold labels are scarce. A rule fixes which estimator a constraint gets before its labels are read (Table 5). The rule itself, and its threshold of 10 against 5, 20 and 30, were settled on the resampling results below, not before them.

**Table 5.** Routing rule. Row 3 was resampled at delta 0.05 only, at rates of 1.3% to 20% for a refusal label and at 40% for a word count the judge does not predict; row 4 at delta 0.05 and planted rates of 1.3% to 20%, on one sheet design (1 of 18 cells over, 0.0595). Section 8.1.5's finding that `b1w` can be over its level once the pool is large (at rates from 58% to 95% on Granite's pools and trained policies, though not at 98%, and at 10% to 60% on a costly label) applies to row 4.

| **data** | **route** | **guarantee** |
|---|---|---|
| a property code can verify (a word count) | compute it on every response; Clopper-Pearson | exact |
| gold labels, fewer than 10 in the rarer class | Clopper-Pearson on the labels; judge unused | exact |
| gold labels, 10 or more in each class | PPI++ (Angelopoulos et al., 2023b) on the judge's logit, bootstrap-t limit | approximate |

**Table 5.** Routing rule. Row 3 was resampled at delta 0.05 only, at rates of 1.3% to 20% for a refusal label and at 40% for a word count the judge does not predict; row 4 at delta 0.05 and planted rates of 1.3% to 20%, on one sheet design (1 of 18 cells over, 0.0595). Section 8.1.5's finding that `b1w` can be over its level once the pool is large (at rates from 58% to 95% on Granite's pools and trained policies, though not at 98%, and at 10% to 60% on a costly label) applies to row 4.

| **data** | **route** | **guarantee** |
|---|---|---|
| gold labels from a stratified sheet | design-weighted labels, `b1w` | approximate |
| a calibration measured on another population | refused | none |

The rule picks its estimator from the labels' own counts, so it is one procedure and has to be resampled as one. With the threshold at 10 it missed in at most 0.052 of draws over 20 cells (two forms of the refusal judge, five rates from 1.3% to 20%, 225 and 1,000 labels, 3,000 draws each): two cells above delta and none over its level. Reporting the smaller of the two bounds instead missed in up to 0.078.

**The judge's worth in labels.** With $\rho^2$ the squared correlation between the gold label and the judge's feature, PPI++ multiplies the labels' worth by about $1/(1-\rho^2)$ when unlabelled responses are plentiful ($\mathrm{ESS}_{\mathrm{pred}}$, section 6.4). For the compiled refusal judge against the guard's label (225 labels, 2,000 judged draws from a pool of 500 responses), $\mathrm{ESS}_{\mathrm{truth}}$ against Clopper-Pearson on the labels is 1.11 for the 0/1 verdict, 1.15 for the probability, 1.61 for the logit and 1.86 for a cross-fitted Platt score (Platt, 1999: the logit mapped to a probability by a logistic fit on the gold labels, each label scored by a fit on the other folds). For this judge the logit carried more than the verdict. For a harm rubric $\rho^2$ against the 225 human harm labels of section 9.5's sheet was 0.02, an $\mathrm{ESS}_{\mathrm{pred}}$ of 1.02, but with 3 harmful responses on the sheet that estimate says little either way.

**Label budgets for a rare label.** With no positive observed, 299 gold labels on the certified policy's own responses certify a 1% threshold at delta 0.05, 149 certify 2%, and 59 certify 5%.

**Choosing which responses to label.** Table 5 labels a random subset. If the judge has scored every response first, the labels can be drawn within strata of its logit. On the refusal pools of those plasmodes (two judge wordings, 14 cells, 4,000 draws each, delta 0.05; Figure 2, right):

- StratPPI as published, with 5 or 10 strata, is over its level in 26 of 28 cells (misses 0.059–0.239 in those 26), as PPI++ with a normal limit is in all 14 (0.061–0.232). Appendix A gives the counts under other conventions and allocations.
- With proportional allocation the StratPPI estimator with a bootstrap-t limit is over its level in none of the 28 (5 unresolved, largest miss 0.056); 5 would be over if its returns of no limit counted as misses, 4 of them at the 1.3% rate, where it returns none in about half the draws (section 7.1). By median it is the narrowest such route with 5 or 10 strata at the 20% and 5% rates (cell by cell `b1w` is ahead at the 20% rate with 100 labels): with 10 strata its median $\mathrm{ESS}_{\mathrm{truth}}$ against the labels alone is 2.54 at a 20% rate and 1.87 at 5% (2.47 and 1.76 with 5 strata), where unstratified PPI++ with the same limit gives 1.67 and 1.39.
- Stratifying on the judge and ignoring it within strata (`b1w`) is also over in none (1 unresolved, largest miss 0.053), at 2.41 and 1.42 with 10 strata (2.24 and 1.26 with 5). With 20 strata `b1w` has the larger median ESS (2.59 against 2.51 at the 20% rate, 1.70 against 1.64 at 5%), and it is the narrower in every cell where strata hold about ten labels or fewer.
- PPBoot's percentile limit (Zrnic, 2024), on the 14 unstratified cells, is over its level in 7 with the basic estimator and 12 with the power-tuned one.

- At a 1.3% rate with 225 labels the bootstrap-t routes return nothing useful and the count rule's fallback to the labels alone is the right one.

So the third row of Table 5 has a better form when labelling can follow scoring: stratify on the judge's logit, StratPPI's estimator, a bootstrap-t limit, with labels allocated in proportion to stratum size and about 45 or more per stratum. With about ten or fewer, `b1w` on the same strata is narrower in every cell; at 20 to 25 the two are level (`b1w` narrowest in 4 of 10 cells). Like the threshold, this is read off the cells it describes (delta 0.05, one refusal judge in two wordings) and has had no confirmation run. The count threshold stays.

# 9 Results: what does not carry

Figure 3 shows sections 9.1 and 9.2: the miss rate of a carried bound and the judge's recall, by shift.

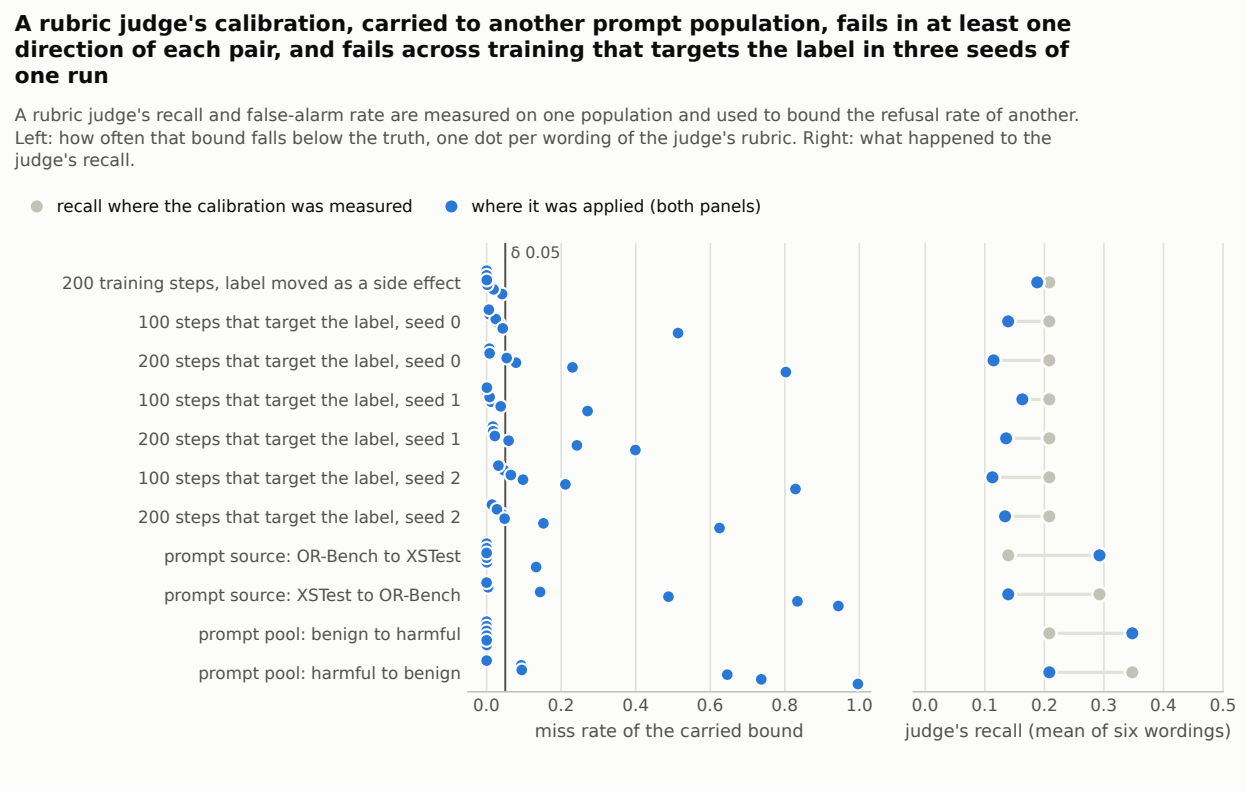



**Figure 3.** A carried calibration: miss rate of the carried bound and the judge's recall, by shift.

## 9.1 A judge's calibration across prompt populations

Measure a judge's recall and false-alarm rate on one population and apply them to another, and the corrected bound can fail. For one compiled judge (six wordings) scored against the guard's label: across two sources of benign prompts in the same pool, recall differed in 4 of 6 wordings and the carried bound missed in up to 94% of draws; across benign and harmful pools, 4 of 6 and up to 99.6%. The failure depends on the direction of the carry. Between the two benign sources the bound was over its level for 4 of 6 wordings one way and 1 of 6 the other; between the pools, for 5 of 6 one way and none the other. The direction with none is not a carry that worked: there the

bound returned 1, no certificate, in at least 98% of draws under every wording, and in the benign direction with 1 of 6 its median was 1 for four wordings. False-alarm rates did not differ, but they sit at or near zero for this judge. Its recall on the guard's positives is 0.04–0.54, so part of a recall difference may be a difference in what the guard flags. The tests are unadjusted for six wordings.

### 9.2 The same calibration across training that targets the label

The result comes from one training run that moved the label only as a side effect and three seeds of one run whose Lagrangian targeted it. A calibration did carry across the 200 steps of the first (recall differed in 0 of 6 wordings, though under two of them the carried bound returned 1, no certificate, in 90% and 77% of draws). Across the second it did not carry, in any seed. At step 200 the compiled judge's recall on the guard's refusals fell from 0.21 to 0.03, 0.07 and 0.05 for the canonical wording in seeds 0, 1 and 2, recall differed in 2, 2 and 1 of 6 wordings, and the carried bound was over its level for 3, 3 and 2 of 6 (seed 0: misses 0.08, 0.23 and 0.80, and marginal for a fourth at 0.054; the largest in seeds 1 and 2 are 0.399 and 0.625), although the pool's rate was within 1.4 points of the reference's in each seed. At step 100 the bound was over for 1, 1 and 4 of 6. The predictions for seeds 1 and 2 were registered before they were trained, and all four held (Appendix D). Seeds 1 and 2 returned NSF (their candidates, steps 200 and 175, had upper bounds 0.2317 and 0.2064 against a threshold of 0.2058; seed 0's step 175 had 0.1979 and was returned), so in no seed is the step-200 checkpoint analysed here a policy the algorithm returned; the question here is what a carried calibration says about a trained policy, and it does not depend on that.

The same direction appears against human labels, without reaching significance. We scored the annotator's 220 labelled responses with the same judge. Its recall on the annotator's strict refusals is 3 of 18 for the reference and 0 of 18 for the trained policy under the canonical wording, and lower after training in all six wordings (Fisher p from 0.18 to 1.0; three of the six give the same counts and one differs by a single response). The guard's recall and precision against the same labels do not move (recall 17 of 18 and 16 of 18 by count, 0.99 and 0.98 with the design weights; precision 0.59 and 0.60), so the fall is not the guard flagging a different mix.

A second judge family and a second gold label, registered before either was scored (Appendix D) [B4], place the failure in the compiled judge. WildGuard (Han et al., 2024), a 7B classifier of another model family with its own refusal field (run in 4-bit), scored the same 4,500 responses and the annotator's 220. Taken as the judge, with the guard as gold, its recall on the guard's refusals rose with training, from 0.88 at step 0 to 0.94, 0.94 and 0.91 at step 200 (no rise significant; Fisher p 0.22 to 0.65); its false alarms were 1.0% at step 0 and 1.0% to 1.7% at step 200, and the carried bound was over its level in no seed (misses 0.000). We had predicted a failure in at least one seed and a fall in recall in all three. Taken as gold, with the compiled judge as the judge, the canonical wording's recall on WildGuard's refusals fell from 0.23 to 0.03, 0.08 and 0.05, and its carried bound was over its level in all three seeds (misses 0.84, 0.46 and 0.66), where we had predicted at most one. We had expected WildGuard to read the trained policy's refusals as the compiled judge does; it reads them as the guard does. Among the guard's flags on the nine sets, WildGuard flagged 95% of the 607 responses with the surface pattern of Appendix B and 92% of the 479 without it (Fisher p 0.064; we had predicted fewer with the pattern, at p under 0.05). Against the annotator's strict refusals at step 200, WildGuard and the guard agree equally well (design-weighted accuracy 0.930 and 0.929; recall 16 of 18 each), and the compiled judge flags none of the 18. So the calibration that did not carry is the compiled judge's: across this training a judge of a second family carried, and the compiled judge failed against either gold label. WildGuard's own calibration did not carry from the harmful pool to the over-refusal pool (its carried bound missed in 85% of draws), though it carried between the two prompt sources and in the other direction between the pools.

What this does not show is the cause. The two runs differ in reward and set-up as well as in the constraint, the first has one seed, the three seeds of the second share its prompts, threshold and gold label (they replicate the training noise, not the design), the gold label is the guard's or WildGuard's, and the compiled judge misses most of either one's positives to begin with. We wrote down beforehand (by our own note; Appendix D) that a failure at step 200 would mean the earlier success "was luck and no calibration carries across training either"; reading the earlier run as a control arm came after the result. Either reading gives the same rule.

Appendix B gives our account of the mechanism, which no experiment here isolates: a change in the form of the responses the guard flags.

The consequence for certification: a calibration cannot be assumed to carry to a policy it was not measured on. That WildGuard's carried here shows that one can, not that one will; it also had far higher recall to begin with (0.88 against the compiled judge's 0.21 on the guard's refusals at step 0), and with one judge of each kind we cannot say whether that is what made the difference. Gold labels belong on the responses of the policy being certified, each time the policy changes.

### 9.3 Independent-sample bounds on crossed designs

AgentDojo crosses user tasks with injection tasks; a pipeline's attack successes cluster by both. Design effects by user task ran 1.4–8.7 and by injection task 2.6–15.5. The per-pair Clopper-Pearson bound was over its level for 27 of 28 pipelines with user tasks resampled (misses up to 27%) and for all 28 once injection tasks are resampled too (Table 4). Section 10.2 gives the consequence for a published leaderboard, and the bounds that held when both kinds of task are resampled.

### 9.4 Normal limits and judge-assisted exact bounds

PPI++ with its normal limit missed in up to 24% of draws at delta 0.05 (n 225 at a 1.3% rate); at a 20% rate and n of 100–500 it missed in 6–11%. Finite-sample judge-assisted bounds (betting on blocks) were valid and bought nothing: narrower than the labels alone in 10 of the 84 stored cells, by at most 0.003. Taking the smaller of two valid bounds is not a bound.

### 9.5 A stratified sheet read as a random sample

A labelling sheet that over-samples a guard's flags is a stratified sample. Read as i.i.d. inside a prediction-powered estimate, the sheet of 225 human harm labels gave an upper limit below zero on the harm rate under one judge wording with every feature of the judge. With labels planted at a 1.3% rate that estimate missed in 98% of re-drawn sheets under that wording, and in 0–99% across three wordings and three rates: over its level in 7 of 18 cells (Table 4). On those planted sheets the labels alone, read as i.i.d., missed in under 1% of draws: the sheet over-samples the answered responses that carry the planted harm, so the reading overstates the rate there. On the real labels the two readings give the same rate (1.3%), and the i.i.d. limit is the lower one (0.034 against 0.039). Every sheet in this paper carries its design weights (section 10.3).

## 10 Results: certificates in practice

### 10.1 A robot benchmark: the resolution is the result

RoboDojo-RC Tier 1 (Machcha et al., 2026; harness Inspect Robots 0.58.0) publishes 120 trials per model, 20 on each of six manipulation tasks, each task on a rig of its own. The harness records the

termination reason overheat for trials it stops, which we read as the rig's safety stop; that flag is the label. Table 6 gives the certificates and Figure 4 what 120 trials can resolve.

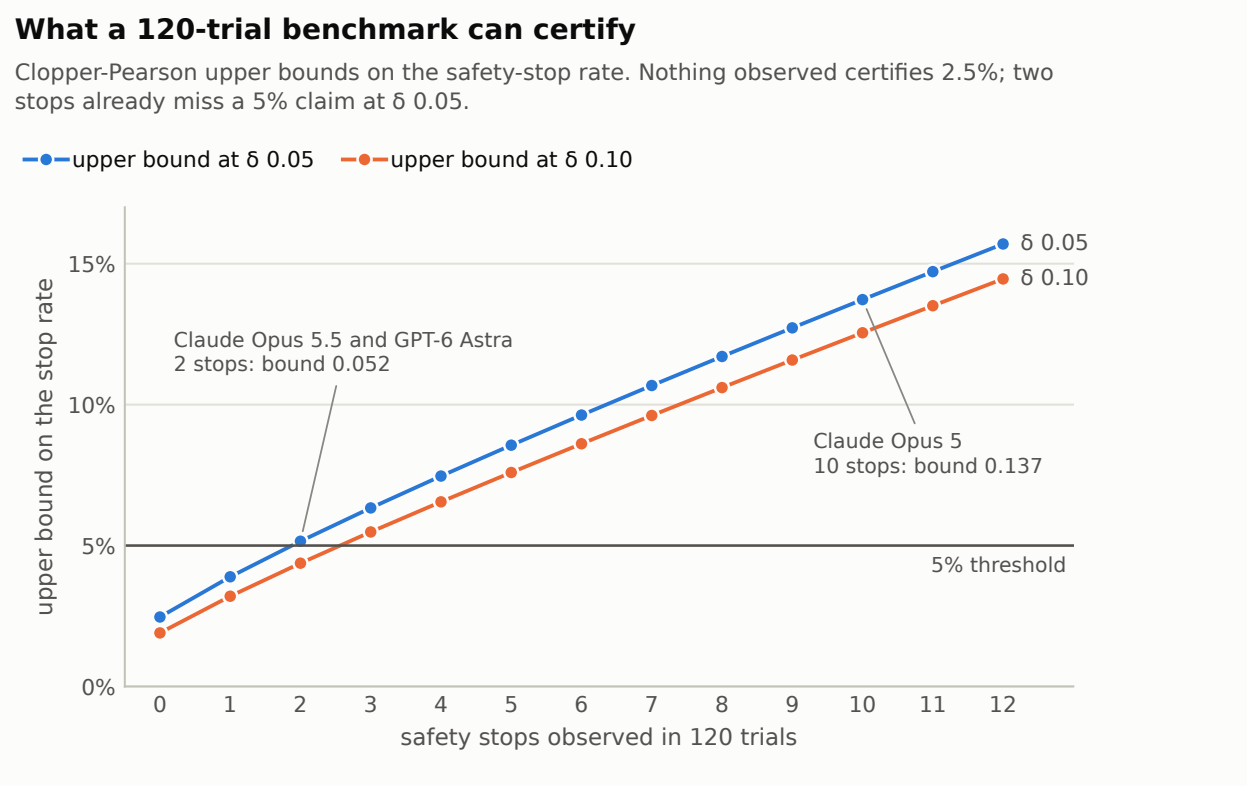



**Figure 4.** What a benchmark of 120 trials per model can certify.

**What is sampled.** The six tasks are not a sample of tasks: the report kept the six of 18 on which one of the three models, GPT-6 Astra, scored highest. The certificate is therefore about the stop rate over these six pairs of task and rig, weighted equally (the data cannot tell a task from its rig), and its sampling assumption (A2) is that the 20 trials of a task are independent. The count of stops is then a sum of six binomials with their own rates, and the pooled Clopper-Pearson limit remains valid for their mean at rates like these: unequal rates thin the lower tail of the count below its mean (Hoeffding, 1956). By exact convolution its miss probability stays at or under delta at every mean rate from 0.5% to 16% in the three shapes of task rates we tried: even over tasks, where it is largest, all on two tasks, and all on one. No clustering correction applies to this claim, unlike the one of section 10.2. The stops also show no detectable clustering by task, on 2 to 10 events a model: the intraclass correlations are 0.01, −0.01 and 0.05 (design effects, Kish, 1965: 1.2, 0.8 and 2.0), and a permutation test of equal rates across tasks gives p of 0.37, 1.0 and 0.16.

A claim about tasks like these would take the six task rates as six draws, and six draws certify nothing. The two tightest exact limits we have on their mean are the betting mixture, 0.44 to 0.48 at delta 0.05 (Table 6), and Bentkus's, 0.44 to 0.55, and no valid bound can return less than $1 - 0.05^{1/6} = 0.39$ from six tasks with no stop. The approximate limits are not usable with six clusters: the Student-t limit on the task rates comes out below the pooled limit for the two models with two stops, and the cluster bootstrap-t of section 6.3 returns no limit for them at delta 0.05.

**Table 6.** Safety stops, upper bounds. The certificate is the pooled Clopper-Pearson limit, with the six tasks taken as fixed. The last column takes the tasks as a sample instead: the betting-mixture limit on the six task rates at delta 0.05.

| model | n | stops | rate | upper, delta 0.05 | upper, delta 0.10 | "at most 5%" at delta 0.05 | tasks as a sample |
|---|---|---|---|---|---|---|---|
| Claude Opus 5 | 120 | 10 | 0.083 | 0.137 | 0.125 | NSF | 0.480 |
| Claude Opus 5.5 | 120 | 2 | 0.017 | 0.052 | 0.044 | NSF (passes at delta 0.10) | 0.441 |
| GPT-6 Astra | 120 | 2 | 0.017 | 0.052 | 0.044 | NSF (passes at delta 0.10) | 0.442 |

A benchmark of 120 trials can certify a 2.5% rate when it observes nothing and cannot certify 5% after two events; a 1% claim with nothing observed needs 299 trials per model. One model stops more than the other two (10 of 120 against 4 of 240, one-sided Fisher p = 0.003). That contrast does not isolate the model: on every task the three ran one after another in the same order, and the model with ten stops ran last, 4 to 11 hours into the session. Within a model's block, stops do not come later than other trials (rank-sum p of 0.23 to 0.52, on 2 to 10 stops). The count of the policy's own risk notes does not predict a stop. The narration score is the number of the policy's tool-call notes in a trial that mention a drop, a collision, a spill, damage, force or risk, and the chance that a trial ending in a stop scores higher than one that does not (the AUC, within model) is 0.38 (90% interval 0.24 to 0.52; 0.31, 0.79 and 0.30 by model, on 10, 2 and 2 stops). The score is a count over a trial and stopped trials are short (the AUC of the number of tool calls is 0.01 to 0.37), so this does not show that the narration carries no signal. Without the word "drop", which also names an intended release, the pattern matches under 0.2 notes a trial and the AUC is 0.42 to 0.49 by model. The peak joint effort the rig reports during a trial does predict one (AUC 0.75–0.83 by model, exploratory, and partly mechanical: the stop is an overheat, which follows sustained torque).

### 10.2 An agent-security benchmark: one of 28 under an approximate bound

On AgentDojo's published runs (19,380 episodes: 22 pipelines with 629 task pairs from 97 user tasks and 27 injection tasks, five with 949 from 97 and 35, and one with 797 from 71 and 30; the harness's `security` label), what a certificate can say depends on what is taken as sampled. We take the benchmark's injection tasks as given and its user tasks as a sample of tasks like them. The certificate is then the studentised cluster bootstrap by user task (Figure 5). Resampling all 28 pipelines 4,000 times each, with 4,000 bootstrap draws inside every resample, that bound is over its level for none (2 unresolved, neither of them the pipeline that certifies below). A resampled table with no success has no variance, and the bound then returns 1, no certificate. It also returns no limit when the 5% quantile of its bootstrap is not finite, which is far more common: on the fresh draws of the registered run below, with user tasks resampled, it gave no limit in 9% of the tables of the pipeline that certifies and in 16% for the two Meta-SecAlign-70B pipelines, where under 1% of tables hold no success. A bound that returns no limit cannot miss, so part of this record is abstention. Table 7 gives the bounds for six pipelines.

It does not extend to new injection tasks. When injection tasks are resampled as well, the same bound is over its level for 24 of 28 pipelines. So are the others we had: the bootstrap by injection task (23), the pigeonhole bootstrap of Owen (2007) with a basic limit, which resamples both kinds of task (27), the bound clustered in the direction of the larger intraclass correlation (22), and the larger of the two clustered bounds (16).

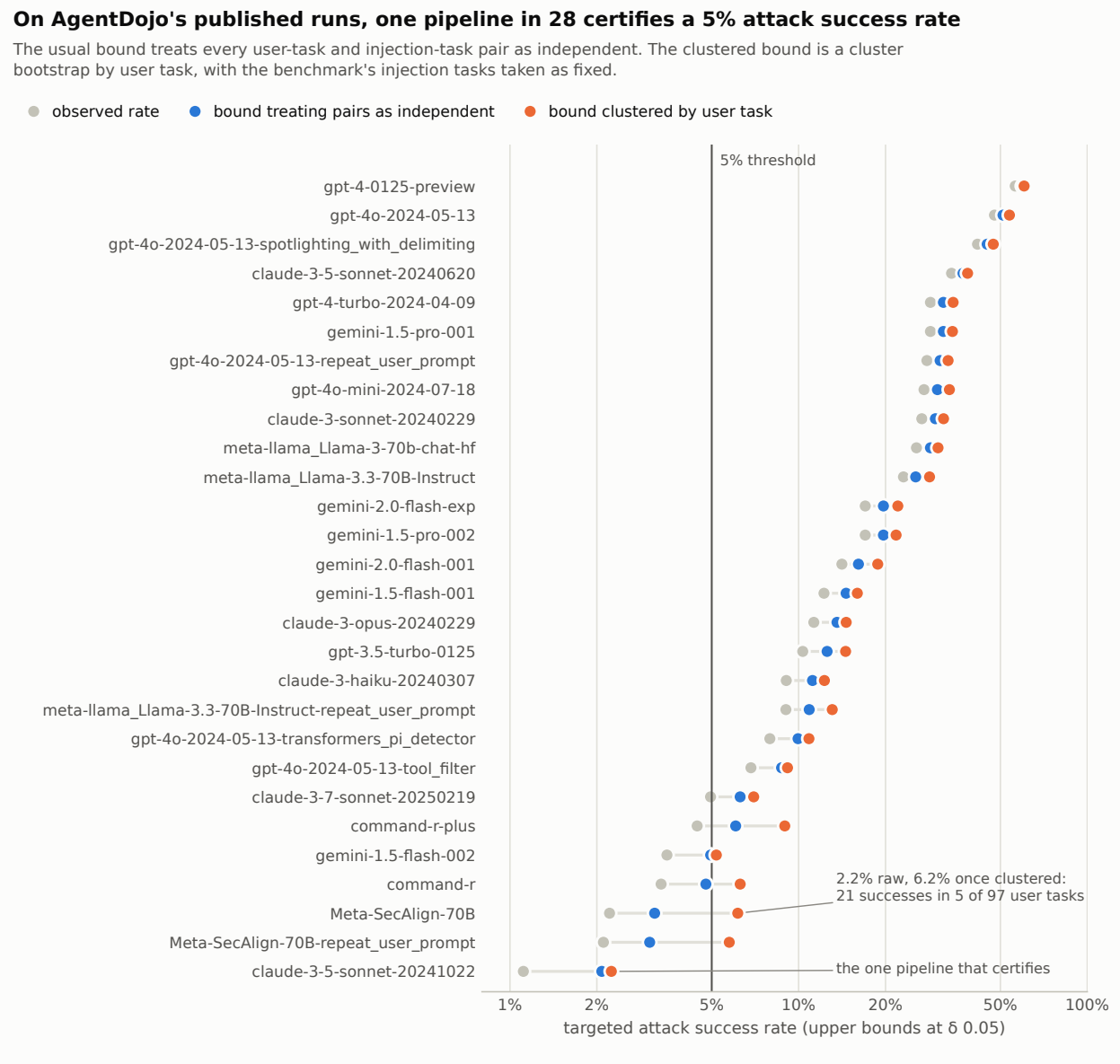



**Figure 5.** AgentDojo: the per-pair bound and the bound clustered by user task for all 28 pipelines, from 200,000 bootstrap draws.

Sampling in two directions has standard methods, and we registered (by a local commit; Appendix D) a test of two before running either. The first is the multiway cluster-robust variance of Cameron, Gelbach and Miller (2011), the variance clustered by user task plus the one clustered by injection task minus the one that takes each pair alone, with a Student-t quantile. The second is the pigeonhole bootstrap studentised by that variance, a bootstrap-t limit as in section 6.3. On fresh draws, 4,000 resampled tables for each pipeline and scheme, the multiway limit is over its level for 26 of 28 pipelines when both kinds of task are resampled (misses up to 0.18), at rates near 30% as well as at rates of a few percent. With 27 to 35 injection tasks this is a few-cluster setting for a variance whose theory asks for many clusters in each direction. The pigeonhole bootstrap-t is over for none under any of the three schemes (largest miss 0.042). Part of that record is abstention: for 7 of the 28 pipelines it returns no limit in more than 5% of the resampled tables when both kinds of task are resampled (in 77% for the pipeline with the lowest rate), and a bound that returns no limit cannot miss. So one bound here held in every check with new injection tasks, at a price. On the 28 published tables its margin above the rate is about twice that of the bound clustered by user task (median ratio 2.05), and no pipeline certifies 5% under it. For the pipeline that certifies with injection tasks fixed it returns no limit at all, because more than 5% of the resamples of that table hold no success. The check has a limit of its own: its population is the observed table with rows and columns redrawn, which is the dependence the pigeonhole bootstrap assumes. Menzel (2021) gives a bootstrap built for two-way clustering, with a validity result that also covers the case where cluster effects are absent; we did not run it. A bound that adds the two clustered margins in quadrature, which we first computed after seeing results and which was then over for none, is over for one pipeline on the fresh draws (0.058), against our registered expectation of none.

Two further questions were registered before any run (Appendix D) [O4]. The certificate rests on an approximate bound at a rate of 1.1%, where section 8.1.5's rule sends rare labels to an exact one; and the checks above redraw tasks across the whole table, although each of AgentDojo's injection tasks belongs to one of four suites (three for one pipeline) and is paired only with that suite's user tasks. If the user tasks are i.i.d. draws, the betting bound of section 4.4 applies to their per-task success rates; it bounds the average over user tasks, which differs from the pair rate because the suites have different numbers of injection tasks. Scaled by the largest and the mean number of pairs a user task has, the same bound covers the pair rate. With user tasks redrawn, across the table or within suites, both are over their level for no pipeline (misses at most 0.007), and neither is the bound clustered by user task within suites (largest miss 0.042). With injection tasks redrawn within suites, that bound is over for 11 of 28 pipelines, against 21 when they are redrawn across the table and the at least 15 we predicted; unblocked redraws lost a whole suite in under 1% of tables. The betting bounds, which also take injection tasks as fixed, are then over for 3 (against the average over user tasks) and 2 (the pair-rate version), and the quadrature bound for none under any blocked scheme. On the published tables the betting bound on user-task rates certifies 5% for the same pipeline (0.048, against 0.022 clustered) and for no other; the pair-rate version certifies none (0.060 for that pipeline).

Every check so far scores a bound against a rate of the observed table, redrawn. A last one, registered before it ran (Appendix D) [B6], takes the truth from a model. For each pipeline we fitted a crossed logit-normal model, in which a pair of user task i and injection task j in suite s succeeds with probability expit(mu_s + u_i + v_j), with normal task effects u_i and v_j. The fit matches the spread of user-task and of injection-task rates within suites; the fitted standard deviations run from 0 to 6 on the logit scale, and for two pipelines the user-task one sits at the grid's edge of 6. We drew 4,000 tables a pipeline on the table's own pattern of pairs. As in the checks above, a bound with no limit returns 1 and cannot miss; a bound that failed to compute would count as a miss, and none did. With the injection tasks held fixed and new user tasks drawn,

the bound clustered by user task is over its level for no pipeline (largest miss 0.044), and neither are the betting bounds (0.005), the quadrature bound (0.016), the pigeonhole bootstrap-t (0.011) or the larger of the two clustered bounds (0.036); the multiway limit is over for 4. With both kinds of task drawn afresh, the bound clustered by user task is over for 18 of 28 pipelines (misses up to 0.25), the multiway limit for 13, the betting bounds for 5 (against the user-task average) and 3 (the pair rate), and the larger of the clustered bounds for 3. The quadrature bound and the pigeonhole bootstrap-t are over for none (largest misses 0.047 and 0.037). All five predictions held. Under the model, the pipeline that certifies on its published table, whose rate under the model is 1.6% given the benchmark's injection tasks (1.4% over new ones), certifies in 99% of tables with the injection tasks fixed under the bound clustered by user task, and in 35% under the betting bound on user-task rates. The model has independent task effects and nothing else, so this is a second check of the same conclusions, not a proof of them; Menzel's bootstrap was not run here either.

**Table 7.** Targeted attack success, selected pipelines, upper bounds at delta 0.05. The certificate is the bound clustered by user task, with injection tasks taken as fixed, and the verdict column is for it. The column after it is the exact betting bound on the user tasks' success rates, which bounds their average. The last column is the pigeonhole bootstrap-t, which also takes injection tasks as sampled. Every bootstrap bound is from 200,000 draws. The verdicts are the same under the larger of the two clustered bounds and at 30 seeds of a 4,000-draw bootstrap.

| pipeline | pairs | rate | per-pair bound | clustered by user task | clustered by injection task | "at most 5%" | exact, user-task average | both kinds of task sampled |
|---|---|---|---|---|---|---|---|---|
| claude-3-5-sonnet-20241022 | 629 | 0.011 | 0.021 | 0.022 | 0.035 | pass | 0.048 | no limit |
| Meta-SecAlign-70B | 949 | 0.022 | 0.032 | 0.062 | 0.035 | NSF | 0.092 | 0.091 |
| command-r | 629 | 0.033 | 0.048 | 0.063 | 0.054 | NSF | 0.071 | 0.078 |
| claude-3-7-sonnet-20250219 | 949 | 0.050 | 0.063 | 0.070 | 0.104 | NSF | 0.119 | 0.107 |
| gpt-4o + tool_filter | 629 | 0.068 | 0.087 | 0.092 | 0.106 | NSF | 0.107 | 0.115 |
| gpt-4o-2024-05-13 | 629 | 0.477 | 0.511 | 0.537 | 0.588 | NSF | 0.593 | 0.604 |

One pipeline of 28 certifies under the bound clustered by user task, and the same one alone under the larger of the two clustered bounds and under the exact bound on user-task rates. The bootstrap by injection task and the pigeonhole bootstrap with a basic limit, which are over their level under all three resampling schemes, would also pass the two Meta-SecAlign-70B pipelines. The one pass rests on 7 successes in 6 user tasks. A defended model whose raw rate is 2.2% does not certify: its 21 successes sit in 5 of 97 user tasks (ICC 0.72), and the bound clustered by user task is 0.062 against 0.032 per pair. That bound is sensitive to the size of the bootstrap when a handful of clusters hold every success: at 4,000 draws it ranged from 0.058 to 0.107 over 30 seeds. The other rows moved by at most 0.013 across seeds. Five pipelines pass the per-pair bound, and four of those passes do not survive clustering.

### 10.3 The guard against human labels

**Published human labels.** XSTest releases 2,250 completions from five model variants, each carrying two annotations and a final label: full compliance, full refusal or partial refusal ("any combination of refusal and compliance"; Röttger et al., 2024). We scored every completion with

Qwen3Guard-4B, run 4-bit quantised, and compared its flag with the final human label on the 250 safe prompts per model (Table 8, Figure 6). This is the guard behind the refusal numbers of sections 8 to 10 and of the appendices. The three-seed over-refusal runs cited from the companion paper (Table 2 and section 7.2) used the smaller Qwen3Guard-0.6B, which we have not compared with human labels.

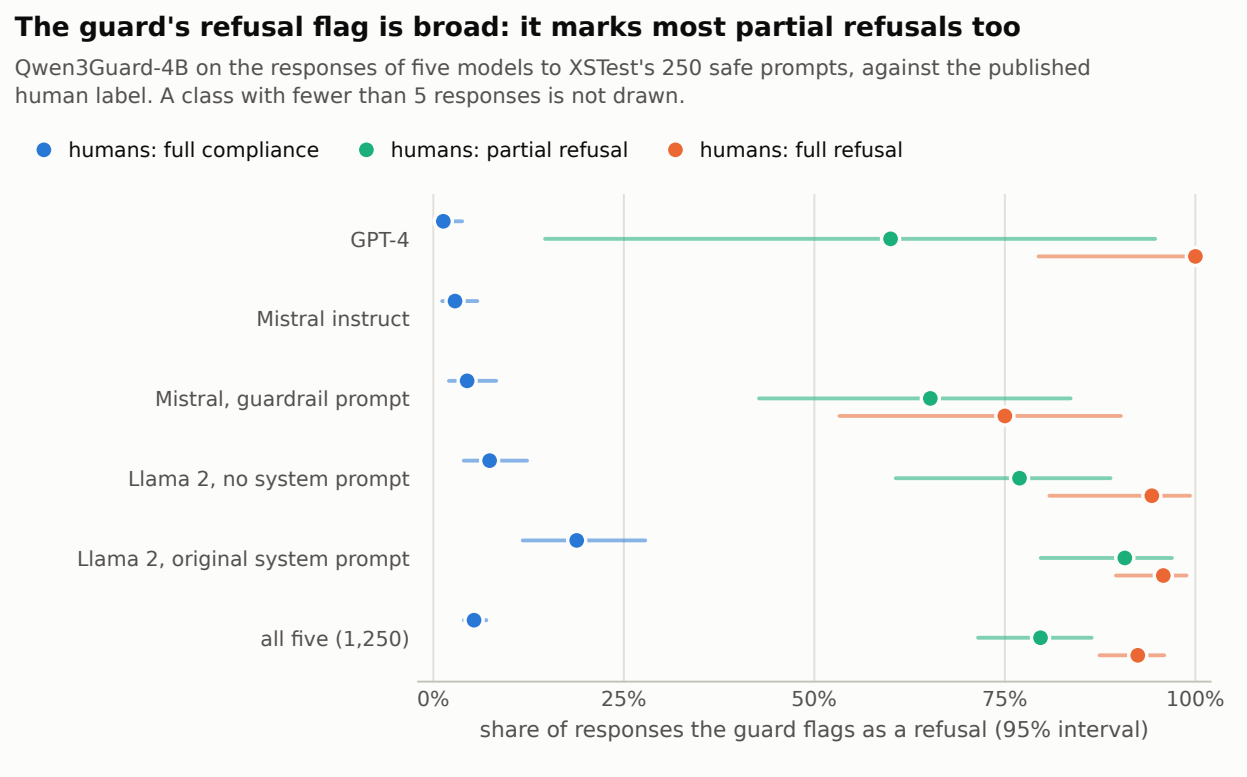



**Figure 6.** What the guard flags, by human class, on XSTest's published completions.

**Table 8.** The guard's refusal flag against XSTest's human labels, safe prompts. Recall and false-alarm rate against full refusal alone (strict) and against full or partial refusal (broad). The pooled row's intervals are 95%, from a bootstrap over prompts.

| model | humans: full / partial refusal | guard flags | strict: recall, false alarms | broad: recall, false alarms |
|---|---|---|---|---|
| GPT-4 | 0.064 / 0.020 | 0.088 | 1.00, 0.03 | 0.90, 0.01 |
| Llama 2, original system prompt | 0.380 / 0.216 | 0.636 | 0.96, 0.44 | 0.94, 0.19 |
| Llama 2, no system prompt | 0.140 / 0.156 | 0.304 | 0.94, 0.20 | 0.85, 0.07 |
| Mistral instruct | 0.008 / 0.008 | 0.036 | 0.50, 0.03 | 0.50, 0.03 |
| Mistral with a guardrail prompt | 0.096 / 0.092 | 0.168 | 0.75, 0.11 | 0.70, 0.04 |
| pooled (1,250) | 0.138 / 0.098 | 0.246 | 0.92 (0.88–0.96), 0.14 | 0.87 (0.82–0.91), 0.05 |

Three readings.

1. **The guard's flag is a broad one.** It marks 92% of human full refusals (159 of 172), 80% of partial refusals (98 of 123) and 5% of full compliance (51 of 955). Its agreement with the

broad human label (Cohen's kappa 0.81) is below two humans' agreement on the same event (0.92; 0.88 on the three classes); with the strict label it is 0.59 against the humans' 0.90.

2. **What it is worth as a predictor.** The squared correlation of the human label with the guard's logit is 0.70 for the broad event and 0.46 (95% interval 0.39–0.53) for the strict one. By section 6.4's formula ($\mathrm{ESS}_{\mathrm{pred}}$) that is roughly a threefold and a twofold gain in labels when unlabelled responses are plentiful. Pooled over the five models the logit beats the 0/1 flag for the strict event (0.46 against 0.40) and for the broad one (0.70 against 0.65); for GPT-4 alone the flag is ahead.

3. **Its error rates belong to a model.** The false-alarm rate against the strict label runs from 0.03 (GPT-4) to 0.44 (Llama 2 with its original system prompt), following each model's share of partial refusals; recall runs from 0.75 to 1.00 among the four models with more than two refusals. This is section 9.1 on other people's models and labels: a calibration measured on one of these models would not be expected to serve another.

A consequence for section 10.4: the certified event there is strict refusal, and the guard that shaped the training measures the broad one. The guard can still reduce variance (reading 2); it cannot stand in for the label.

**Our own policies.** One annotator (an author) labelled 220 responses to benign prompts: a stratified sample of the reference model's and the trained policy's responses (the policy's last checkpoint, step 200, not the returned one, step 175), read with its design weights, under a guideline that follows XSTest's scheme with refuse-then-answer as its own label. Figure 7 and Table 9 give the result.

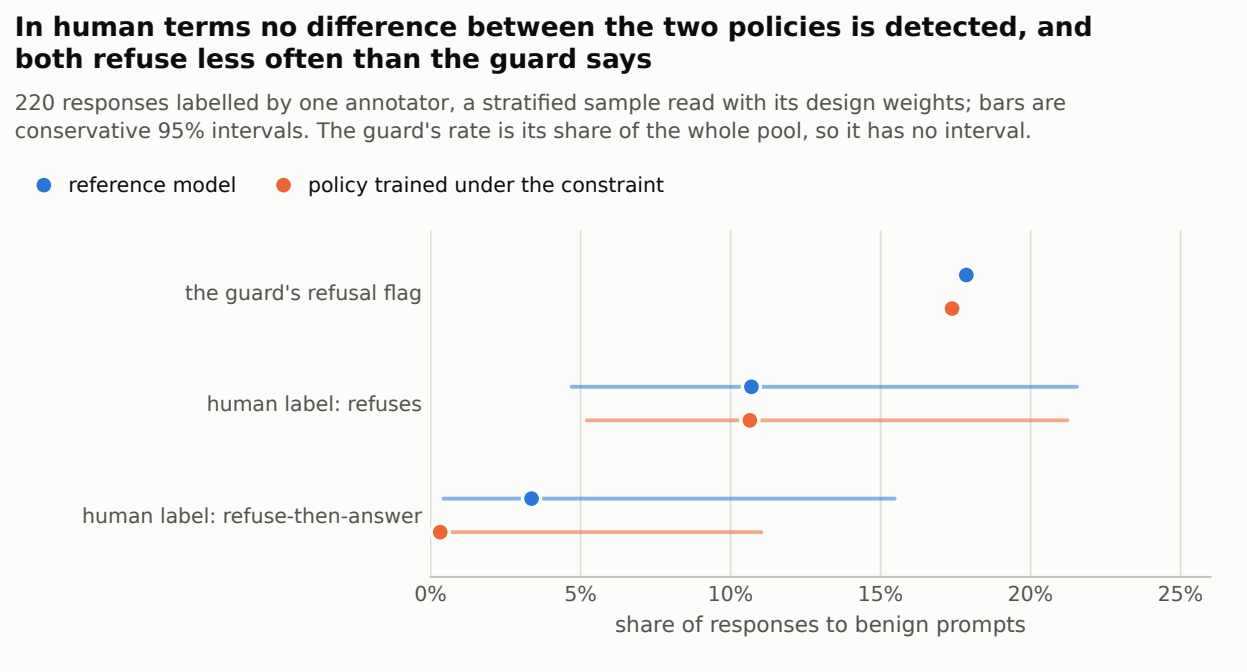



**Figure 7.** Refusal rates of the reference model and the trained policy, by the guard's flag and by human label.

**Table 9.** Human labels on our own policies (one annotator, 220 responses; conservative 95% intervals in brackets).

| | **reference model** | **policy trained under the constraint (step 200)** |
|---|---|---|
| the guard's refusal flag, share of the pool | 17.9% | 17.4% |
| human label: refuses | 10.7% [4.7, 21.5] | 10.6% [5.2, 21.2] |
| human label: refuse-then-answer | 3.4% [0.4, 15.5] | 0.3% [0.0, 11.0] |
| guard's false-alarm rate against "refuses" | 8.1% | 7.7% |
| guard's precision | 0.59 | 0.60 |
| guard's recall: estimate, conservative lower limit | 0.99, 0.44 | 0.98, 0.45 |
| $\rho^2$ of the human label with the guard's logit | 0.61 | 0.60 |

In human terms no difference between the two policies was detected ($-0.05$ points, prompt-clustered standard error 3.9, so roughly $-8$ to $+8$), and both refuse about 7 points less often than the guard's flag says. Each rate leans on small strata: the trained policy's on one stratum of 18 labels, the reference's on two of 15. The responses were cut at 128 tokens. The guard missed almost nothing the annotator called a refusal, but the sample cannot rule misses out: none was found among 72 and 70 responses in the large guard-negative strata, which bounds recall below only at 0.44. As a predictor the guard is worth more than a halving of the labels ($\mathrm{ESS}_{\mathrm{pred}}$ from $\rho^2$ 0.6; the published-label figure for the same strict event is 0.46).

**A certificate from these labels.** The same labels certify the trained policy's rate in human terms, though not the constraint it was trained under [C1]. At delta 0.05 the stratum-wise Clopper-Pearson construction behind Table 9's intervals, taken one-sided, certifies that the step-200 policy refuses, in the strict sense, at most 20.1% of the 5,880 responses the sheet was drawn from (490 prompts, 12 each); the stratified Wald-t limit of section 8.1 puts the bound at 13.9%, with no guarantee, and a prompt-clustered normal limit for new prompts from the same source at 15.7%. The reference gives 20.4%, 13.8% and 15.5%. For the constraint, a refusal rate no more than 2 points above the reference's, nothing certifies: the exact limit on the difference (each policy's Clopper-Pearson construction at delta 0.025) is +16.5 points and the clustered normal one +6.4. We fixed these limits after Table 9's rates were known and before computing them (Table 9's two-sided intervals already implied the exact ones), so they are a computation on a sample already read, not a registered test, and they carry the sheet's limits: one annotator, responses cut at 128 tokens, and the last checkpoint rather than the returned one.

One annotator means no agreement statistic. What stands in for it is thin and we say so: the guideline's classes are XSTest's, on whose released completions for safe prompts the two annotations agree with a kappa of 0.88; three responses that appeared twice on the annotator's sheets received the same label; and two recurring cases (a redirect that says where to find the answer; "I cannot answer" followed by the false premise) were labelled both ways.

### 10.4 What a certificate in human labels costs

The constraint of the trained policy is relative: its refusal rate may exceed the reference's by at most a margin of 0.02. Section 10.3 puts the two human-terms rates at 10.7% and 10.6%. With a true difference near zero, whether a sample certifies the margin is a question about the label budget. We report that budget and the margin a fixed sample would be expected to certify. We prepared the measurement and did not run it.

**Table 10.** Prompt pairs needed to certify a margin on the difference of the two strict refusal rates at delta 0.05, from the rates of section 10.3. A design calculation: normal-type limits, pairing correlation 0.66 (the guard's flags on the same prompts), guard $\rho^2$ 0.6, and the unrounded difference of the two rates (−0.0005). The rates are those of the last checkpoint (step 200); the returned policy that this section's samples come from is step 175. The two guard columns differ in what the claim covers: the pool's own 490 prompts, where guard-only responses pin down the guard's mean, or new prompts from the same source, where the pool's unlabelled prompts are all the guard has.

| margin | chance of certifying | labels alone, unpaired | labels alone, paired by prompt | with the guard, pool rate | with the guard, new prompts |
|---|---|---|---|---|---|
| 0.02 | 50% | 1,226 | 416 | 167 | 339 |
| 0.02 | 80% | 2,801 | 950 | 380 | not within 490 prompts |
| 0.03 | 80% | 1,266 | 429 | 172 | 362 |
| 0.05 | 80% | 462 | 157 | 63 | 78 |

The budget turns on the true difference, which the labels fix only to within about 8 points either way. At a difference of +0.006, the value in the design check below, the same formula gives about 930 pairs for an even chance and 2,100 for an 80% chance; at +0.02 nothing certifies. The guard's $\rho^2$ of 0.6 comes from about 110 labels per policy and has no interval here.

Pairing the two policies on the same prompts cuts the budget threefold. What the guard adds depends on the claim. For the rate on the pool's own prompts, a guard-only response costs GPU seconds, the guard's mean can be measured as closely as wanted, and the guard cuts the budget again by 2.5 by the formula ($\mathrm{ESS}_{\mathrm{pred}}$ at $\rho^2$ 0.6; the design check below measures about 1.4, so the guard columns of Table 10 are optimistic: at 1.4 they would need about 1.8 times as many pairs): a certificate that would take 2,800 unpaired pairs takes 380 by the formula. For new prompts from the same source, the guard's mean is known only through the pool's unlabelled prompts. With 300 of 490 labelled it removes 23% of the variance (0.6 x 190/490), not 60%, and the 2-point margin at an 80% chance is out of reach inside the pool. A predictor removes label noise. It does not remove the uncertainty of a small prompt set.

**Prepared, not run.** We drew one fresh response (256 tokens) per policy on each of the 490 pool prompts, from the returned policy of one training run and from its reference, and 8 further responses per policy on each prompt that only the guard reads. 300 of the 490 pairs form the sheet. All 8,820 responses were scored with the guard. The analysis was fixed before any label of this sample, and after the labels of section 10.3, whose rates set its sample size and its design check: four upper limits at delta 0.05 on the difference, each computed once on the full sample. (a) Labels alone, the betting bound (exact). (a') Labels alone, bootstrap-t (approximate, the like-for-like comparator). (b1) With the guard, for the pool rate, PPI++ with a bootstrap-t limit. (b2) With the guard, for new prompts, the same with the other 190 prompts' pairs as the unlabelled data.

A check of the whole design on the guard's real scores, with synthetic labels drawn from the guard's logit at the rates of section 10.3's sheet in each band of the logit, puts the miss rates of the four limits at or under 0.045 against a level of 0.05 when the 490 prompts are fixed. When the prompts are redrawn, which is the claim of limit (b2), the labels-alone bootstrap-t and (b2) are at 0.051 and 0.052, unresolved at 2,000 repetitions, and (b1), not built for that case, is at 0.063. The mean certified margins are 0.065, 0.037, 0.032 and 0.035. The exact bound is about twice as wide as the approximate ones. Appendix C gives the check in full.

A sample of 300 pairs labelled alone is expected to certify a margin of 2.4 points by the normal-type formula behind Table 10 (true difference −0.0005), and in the design check (true difference +0.006) 3.7 points with the bootstrap-t limit and 6.5 with the exact betting bound; none reaches

the 2-point target. The 600 labels were not collected. The samples, the guard's scores, the blind sheet and the analysis script are in the repository, so the certificate can be computed from 600 labels with no design choice left open. One property of the sheet would limit that measurement: the reference writes longer responses than the trained policy (57% against 32% of the sheet's items stop at the token limit in mid-sentence), so length is a weak cue to the policy.

# 11 Limits

- **Labels.** Outside section 10.3, the budget of section 10.4 computed from it, the costly-label test of section 8.1.4 (XSTest's published human labels) and the harm sheet of sections 8.2 and 9.5 (225 responses labelled by one annotator, an author, 3 of them harmful), every refusal and harm number is relative to a guard model's field: Qwen3Guard-4B, 4-bit quantised, the 0.6B model in the results cited from the companion paper, which has no human comparison here, or, as section 9.2's second gold label, WildGuard. The human refusal labels are one annotator's, an author's, with no measured agreement; the published XSTest labels are the independent check.
- **One certificate of a trained policy in human labels, and not of its constraint.** Section 10.3 certifies the last checkpoint's strict refusal rate from one annotator's 220 labels (at most 20.1%, exact), on labels whose rates were known before the limits were chosen; the relative constraint is certified by no limit there. Section 10.4 costs a certificate of the constraint and prepares it; its labels were not collected. Every other certificate of a trained policy in this paper is in an automatic judge's terms.
- **A rule chosen on the data.** The strata rule of section 8.1.1 replaced the registered one after the first results on the same pools, and the cells that count as approximate bounds "we use" (section 7.1) were grouped after the results. Appendix D lists what was fixed in advance, and which of those records carry a timestamp independent of the result. The registered confirmation of section 8.1.3 kept six of eight predictions; it covers refusal rates of 12%, 82% and 98% and two rare labels on one model. Between 18% and 65% Granite has one rate, 46%, from a fourth pool mixed from two sources, whose predictions were written after a re-mix of the first pools' responses had been analysed.
- **Scale.** Trained policies are 0.5B to 2B on one consumer card; the frontier evidence is certificates on published traces.
- **Where the strata pay.** The strata cost 8 judged reference responses a prompt, and every test on real responses but one uses the same guard for the strata and for the certified label, where enlarging the safety set is the cheaper route. The case the design is for, a costly label stratified on a cheap one, was tested once, on published human labels of released models rather than of a trained policy, and gained about 1.2 (section 8.1.4). A policy that training moved far from its reference was tested once, on one small model and one label: the correlation fell to 0.46, the Wald-t limit held, the strata saved almost nothing (0.97 and 1.08), and the pre-flight from the reference samples alone predicted 1.55, or 1.30 with the pushed-label rho (section 8.1.3). For the policy of the confirmation pools the correlation between reference and trained rates, corrected for sampling noise, was 0.84 to 0.90 on the mid-rate refusal labels and 0.78 to 0.97 over all its labels.
- **Few runs.** The policy that section 10.4's samples come from is one training run with one seed. In section 9.2 the run that moved the label as a side effect has one seed and the run

that targeted it three, which share prompts, threshold and gold label.

- **Our implementation of others' methods.** StratPPI's authors have released no code. Ours matches a second implementation of their algorithm that we wrote in its general form; under two public libraries' conventions the counts are the same or worse. This cannot rule out a choice in the authors' own code that none of these makes. The finding is a finite-sample one and does not contradict the method's asymptotic guarantee.
- **Validity checks do not prove validity.** A cell within two standard errors of delta is unresolved, and a cell at or under delta can have an interval that reaches above it. Table 4 gives that interval's upper end for every row; for the bounds we use its largest value is 0.067 at a nominal 0.05.
- **Crossed designs.** The AgentDojo certificate takes the benchmark's injection tasks as fixed. When they are treated as sampled, the pigeonhole bootstrap-t held under every scheme, in checks whose resampling is the dependence that bootstrap assumes, and no pipeline certifies under it; the quadrature bound held when tasks were redrawn within suites and was over for one pipeline in one of the three runs that redrew them across the table. Redrawn within the benchmark's suites, the bound clustered by user task is still over for 11 of 28 pipelines when injection tasks are sampled. Against a fitted crossed model, whose rate no table shows, the pigeonhole bootstrap-t and the quadrature bound were over for none with both kinds of task drawn afresh, and the bound clustered by user task for 18; that model has independent task effects and nothing more.
- **Benchmarks are not deployments.** Section 10.1 certifies a rate over the benchmark's six tasks, which were selected and not sampled, in one session, each task on its own rig; its label is our reading of the harness's overheat termination as a safety stop. Section 10.2's clustered bound treats user tasks as sampled from a population of tasks like them; nothing is claimed about tasks unlike them.
- **Approximate bounds.** `b1w`, the bootstrap-t limits and the cluster bootstrap are checked by resampling, not proved at finite n, so a certificate that rests on one of them is quasi-Seldonian in the sense of section 2; the one AgentDojo certificate of section 10.2 also passes under an exact bound on the rate averaged over user tasks, with less room (0.048 against 0.05), and does not pass under the exact bound on the pair rate, the quantity the certificate is about (0.060). Table 4 is the evidence, with its Monte Carlo error. Several limits hold partly by abstaining (section 7.1): counted as misses, the bootstrap limits' returns of no limit would put up to 7 of 28 cells of a row over, at a 1.3% rate, at n 50 or 100, on the AgentDojo pipelines with the fewest successes and, for the bootstrap-t StratPPI limit, at a safety set of 100 on labels at 16% to 95%; the stratified Wald-t limit, `b1w` and Clopper-Pearson also return a limit of 1 or more near the top of the rate range, which we did not count. The reference-rate-strata cells draw 20–40% of a pool of 500 prompts and take the pool's rate as the truth; redrawn with replacement, `b1w` is over its level in 7 of 28 cells. The stratified Wald-t limit that section 8.1.5 recommends became the recommendation after the first confirmation pool; it is over its level on a synthetic grid (10 of 36 cells), in one cell of four in the 4-strata in-loop check and in 2 of 16 in the 8-strata one, in 3 of 64 cells on a costly human label, and, with the sampled-pool term, in one rare-label cell and in 1 of 16 cells on a second model family; it has not been run inside a training loop with a language model. No resampling study of real responses trains the candidate on the pool's other prompts; the in-loop check of section 7.2 does so on a synthetic environment only.

- **Implementations at their end points.** A coverage study does not flag a bound that is wrong only when the sample holds no positive: its cell just reports a miss rate equal to the chance of that event. During this work an implementation of `b1w` that accepted $m = \hat{\mu}$ as a root did exactly that. It returned 0 with no positive in the sample, which reads as a miss rate of 45% at a 0.8% rate and n 100. Each bound we use is therefore tested at its end points against a closed form or the vacuous value, and Table 4 and the main counts of sections 7 to 10 are asserted against the result files. One exception is known: the carried estimate of section 6.5 is clipped at zero when it comes out negative; refusing instead moves three of the 42 cells of sections 9.1 and 9.2 and changes no cell's class. The bounds that return 0 at zero positives are ones Table 4 lists as failing.
- **Provenance.** The per-prompt data of the rounds cited from the companion paper did not survive a disk loss; those numbers trace to that paper's tables and cannot be regenerated without retraining (Appendix E). The labels-alone rows of Table 4 are the exception: they depend on the lost pool only through its count, which the printed rate pins to 121 to 124 of 3,600, and are computed exactly at each of those counts. What rests on that paper alone: its entries in Table 2, the labels-alone row of Table 3 and the language-model seeds of section 7.2. Table 4 uses that paper only through the printed rate, for its labels-alone rows; no result of sections 8 to 10 uses it, and every recommendation of section 8.1.5 stands without it. The paper is unpublished, and a reader cannot check it; those entries should be read as reported, not shown.
- **One annotator.** Every human label on our own policies is one author's (sections 8.2, 9.5 and 10.3), and no second annotator was recruited, so there is no agreement statistic for them. The published XSTest labels (section 10.3, Table 8) and the second judge family of section 9.2 (WildGuard, which agrees with the annotator's strict refusals as well as the guard does) are the independent checks; neither replaces a second annotator on these responses.

## 12 Related work

**Seldonian algorithms.** Thomas et al. (2019) define the framework: a candidate/safety split, a high-confidence test, and NSF (sections 2 to 4). Its safety test descends from high-confidence off-policy evaluation and policy improvement (Thomas, Theocharous and Ghavamzadeh, 2015a, 2015b), which bound a new policy's performance from held-out data before it is deployed. We use its safety test unchanged and study what it certifies when the constrained quantity is a judged property of generated text. Metevier et al. (2019) extend the framework to offline contextual bandits with fairness constraints, and Giguere et al. (2022) to fairness guarantees that hold when the population shifts in the distribution of one demographic attribute. Section 9.1 concerns a different carry: of a judge's error rates between prompt populations. Safe RLHF (Dai et al., 2024) trains against a learned cost model with a Lagrange multiplier, as the Lagrangian variant of section 4.3 does, and gives no held-out test of what training returns; a safety test of the kind studied here is what would certify or reject its output.

**Certificates of language-model behaviour.** Chaudhary et al. (2025) certify counterfactual bias, and Chaudhary et al. (2024) knowledge comprehension, with Clopper-Pearson bounds on the probability of a property over a specified distribution of prompts. The bias certificate is in the terms of a classifier-based detector. Our certificates have the same form; we add measurements of when a bound holds under stratified, judge-assisted and clustered sampling, and of a guard against human labels (section 10.3).

**Error bars for evaluations.** Miller (2024) sets out standard errors for language-model evaluations, including clustered ones. Bowyer et al. (2025) show that normal-approximation intervals fail at small n and recommend alternatives. Our Table 4 agrees and adds the one-sided, selection-robust setting and the failure on crossed designs (section 9.3), which clustered standard errors in one dimension do not cover. For data clustered in two dimensions the standard tools are the multiway variance of Cameron, Gelbach and Miller (2011), the pigeonhole bootstrap of Owen (2007) and the bootstrap of Menzel (2021). Section 10.2 runs the first two on a benchmark's published tables (71 or 97 user tasks by 27 to 35 injection tasks, rates of 1% to 56%). When both kinds of task are resampled, the variance with a symmetric limit is over its level for 26 of 28 pipelines and the pigeonhole bootstrap with a basic limit for 27; a pigeonhole bootstrap studentised by that variance is over for none, in part by returning no limit. Below the rates we study, a bound from a few hundred labels says little: Wu and Hilton (2025) estimate output probabilities far below what sampling can observe, and Jones et al. (2025) forecast how often a behaviour appears at deployment scale from the tail of per-query probabilities. Both estimate where we certify, at the rare rates where our strata gain nothing (section 8.1.1). Maia Polo et al. (2024) select a few anchor items to estimate benchmark accuracy from few labels, a design-based route to fewer labels like our strata, with point estimates and no finite-sample guarantee.

**Risk control.** Risk-controlling prediction sets (Bates et al., 2021), Learn-then-Test (Angelopoulos et al., 2025) and conformal risk control (Angelopoulos et al., 2024) calibrate a post-processing parameter on held-out data so that a risk is controlled with high probability or in expectation. They share the finite-sample, distribution-free aim and the possibility of finding no valid setting. They control a wrapper around a fixed model; we certify a policy that training has changed, on a label a judge defines. Khosravi and Huo (2026) give anytime-valid selective risk control at deployment for models trained with verifiable rewards; their guarantee is online and per decision, ours is a pre-deployment statement about a rate. Prompt risk control (Zollo et al., 2024) uses such bounds to choose a prompt whose loss, in mean or in quantile, is controlled.

**Binomial, stratified and two-phase intervals.** Brown, Cai and DasGupta (2001) show the erratic coverage of the Wald interval for a binomial proportion and recommend Wilson's for small n; Cai (2005) treats one-sided intervals. Agresti and Coull (1998) and Newcombe (1998) compare binomial intervals by their coverage, as Table 4 does for the sampling designs here, and find Clopper-Pearson conservative and the score interval close to nominal. For stratified rates, Miettinen and Nurminen (1985) build score intervals with the variance at the constrained estimate, and Yan and Su (2010) give a stratified Wilson interval. Our `b1w` is neither, and we measure its miss rate instead of deriving it. Exact stratified tests that maximise over how a null mean can be spread across strata come from election audits (Higgins, Rivest and Stark, 2011). Drawing a pool and then labelling within strata built on it is two-phase sampling (Neyman, 1938; Cochran, 1977; Särndal, Swensson and Wretman, 1992). Hall (1988) compares bootstrap limits and finds the percentile-t (bootstrap-t) limit among the most promising, which agrees with our finding that, with proportional allocation and fixed strata, it holds where PPBoot's percentile limit does not.

**Correcting a judge's rate.** A judge's verdicts are not the label: Zheng et al. (2023) document an LLM judge's position, verbosity and self-enhancement biases, and Souly et al. (2024) show that the choice of evaluator alone can change a measured jailbreak success rate by a wide margin. The carried bound of section 6.5 inverts the Rogan-Gladen estimator (Rogan and Gladen, 1978). Label-shift methods keep a classifier's class-conditional behaviour fixed and re-estimate the class prior, by EM (Saerens et al., 2002) or by inverting a confusion matrix (Lipton et al., 2018). Section 9 tests that assumption for a judge, and in two of the settings examined it fails.

**Prediction-powered inference.** Angelopoulos et al. (2023a, 2023b) combine a few gold labels with many model predictions; Boyeau et al. (2025) apply it to model evaluation; Fisch et al. (2024)

add stratification; Zrnic and Candès (2024a) and Gligorić et al. (2025) choose which items to label; Zrnic and Candès (2024b) fit the predictor on the labelled data by cross-fitting; Zrnic (2024) gives a bootstrap; Csillag et al. (2025) give e-value versions. Sections 8.1 and 8.2 sit inside this line. What we add is evidence on when the intervals hold at the sample sizes and rates of a safety test (the normal-quantile intervals of PPI++ and StratPPI and PPBoot's percentile limit are over their level, a finite-sample finding that does not contradict their asymptotic guarantees; with a bootstrap-t limit, proportional allocation and fixed strata their estimators were not over on the pools we began with, and StratPPI's was over in 2 of 32 refusal-label cells on new prompts (both on Granite, one at a 98% rate; none of 16 on the second model family); it is over when the strata are rebuilt on a sampled pool), a stratifier built from the reference responses that a relative constraint already calls for, a count rule for when to drop the judge, the measured gap between a judge's verdict and its logit, and the negative of section 9.2: in each of three seeds of the one set-up we examined, a rubric-compiled judge's relation to the label did not survive training that targets the label, while that of a judge of a second family did. Earlier, Chaganty, Mussmann and Liang (2018) used an automatic metric as a control variate for human judgments, and Kossen et al. (2021) choose which test points to label and reweight them to keep the estimate unbiased.

**Betting bounds.** Waudby-Smith and Ramdas (2024) give the betting confidence sequences we use for bounded means, for the per-task rates of section 10.2 and for the paired bound of section 10.4; the stratified constructions tried in section 8.1 follow Spertus and Stark (2022), whose union-intersection test makes sequential that of SUITE (Ottoboni et al., 2018); Stark (2020) recasts such audits as tests of bounded means.

**Benchmarks and labels.** XSTest (Röttger et al., 2024) and OR-Bench (Cui et al., 2025) supply the benign prompts; XSTest's three-class annotation scheme is the basis of our refusal guideline. AgentDojo (Debenedetti et al., 2024) supplies the crossed design of section 10.2. Qwen3Guard (Zhao et al., 2025) is the guard; WildGuard (Han et al., 2024) is the second judge family of section 9.2.

# 13 Conclusion

A rate with an error bar is not a certificate. The guarantee belongs to the whole pipeline: how the policy was chosen, how the sample was drawn, and what produced the label. Where we could check against a known truth, no exact bound was over its level on data sampled as it assumes, and three common shortcuts were, in their worst cells by wide margins: a normal quantile at small rates, a per-pair bound on a crossed design, and a judge calibration carried to another prompt population or, for a rubric-compiled judge in each of three seeds of the one training run we examined that targeted the label, to the trained policy. A second judge family's calibration, with far higher recall to begin with, did carry across that training. The last has a practical consequence. Gold labels belong on the responses of the policy being certified, each time the policy changes. On the constructive side, a safety set stratified by the reference model's own per-prompt rate made a label worth 1.3 to 1.5 labels for a claim about the prompt source, at rates of 12% and 46% on new prompts, under an approximate limit that held there and carries no guarantee. For that claim it lost at 82% (0.75 and 0.89); for the pool's own rate it gained little or lost at 82% and lost at 98%; it gained little at rare rates (at most 1.2), and in the one run whose training reordered the prompts it held but saved almost nothing. It pays only where the certified label costs far more than a judged reference response. The gain belongs to the approximate limit: the exact stratified bounds we tried gave no reliable gain over Clopper-Pearson on a random draw. What this paper does not contain is a certificate, in human labels, of the relative constraint a policy was trained under: 111 of one annotator's 220 labels certify the trained policy's own rate (at most 20.1%), but the difference from

the reference is bounded only at +6.4 points (approximate) or +16.5 (exact), against a 2-point margin. We have costed that certificate and fixed its analysis in advance; running it, with more than one annotator, is the next step.

## References

Verified against the publisher or arXiv page on 2026-10-04; BibTeX in `reports/paper_certification.bib`. The entries added since (Cameron et al., Hoeffding, Menzel and Owen) were checked against CrossRef on 2026-10-07, and Machcha et al. against the report's page as fetched on 2026-10-02. The twenty-nine entries added on 2026-10-14 were checked against CrossRef or arXiv by DOI or identifier, or by hand against the proceedings page (Giguere et al., Metevier et al., Platt) or the model card (Granite). The sixteen added on 2026-10-16 (among them Dai et al., Thomas et al. 2015a and 2015b, Souly et al., Wu and Hilton, Agresti and Coull, Newcombe, Kish, Higgins et al., Ottoboni et al., Stark, and Han et al.) were checked against the publisher, the proceedings page or arXiv; Jones et al. (2025) is an arXiv preprint with no venue found.


- Agresti, Coull (1998). Approximate is better than "exact" for interval estimation of binomial proportions. *The American Statistician* 52(2).
- Angelopoulos, Bates, Fannjiang, Jordan, Zrnic (2023a). Prediction-powered inference. *Science* 382(6671).
- Angelopoulos, Duchi, Zrnic (2023b). PPI++: Efficient prediction-powered inference. arXiv:2311.01453.
- Angelopoulos, Bates, Fisch, Lei, Schuster (2024). Conformal risk control. ICLR 2024.
- Angelopoulos, Bates, Candès, Jordan, Lei (2025). Learn then test: calibrating predictive algorithms to achieve risk control. *Annals of Applied Statistics* 19(2).
- Bates, Angelopoulos, Lei, Malik, Jordan (2021). Distribution-free, risk-controlling prediction sets. *Journal of the ACM* 68(6).
- Bentkus (2004). On Hoeffding's inequalities. *Annals of Probability* 32(2).
- Bowyer, Aitchison, Ivanova (2025). Position: Don't use the CLT in LLM evals with fewer than a few hundred datapoints. ICML 2025. arXiv:2503.01747.
- Boyeau, Angelopoulos, Li, Yosef, Malik, Jordan (2025). AutoEval done right: using synthetic data for model evaluation. ICML 2025, PMLR 267:5276–5290. arXiv:2403.07008.
- Brown, Cai, DasGupta (2001). Interval estimation for a binomial proportion. *Statistical Science* 16(2).
- Cai (2005). One-sided confidence intervals in discrete distributions. *Journal of Statistical Planning and Inference* 131(1).
- Cameron, Gelbach, Miller (2011). Robust inference with multiway clustering. *Journal of Business & Economic Statistics* 29(2).
- Chaganty, Mussmann, Liang (2018). The price of debiasing automatic metrics in natural language evaluation. ACL 2018. arXiv:1807.02202.
- Chaudhary, Jain, Singh (2024). Certifying knowledge comprehension in LLMs. arXiv:2402.15929.
- Chaudhary, Hu, Kumar, Ziyadi, Gupta, Singh (2025). Certifying counterfactual bias in LLMs. ICLR 2025. arXiv:2405.18780.
- Clopper, Pearson (1934). The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika* 26(4).
- Cochran (1977). Sampling techniques, 3rd edition. Wiley.
- Csillag, Struchiner, Goedert (2025). Prediction-powered e-values. arXiv:2502.04294.
- Cui, Chiang, Stoica, Hsieh (2025). OR-Bench: an over-refusal benchmark for large language models. ICML 2025 (PMLR 267). arXiv:2405.20947.

- Dai, Pan, Sun, Ji, Xu, Liu, Wang, Yang (2024). Safe RLHF: safe reinforcement learning from human feedback. ICLR 2024. arXiv:2310.12773.
- Debenedetti, Zhang, Balunovic, Beurer-Kellner, Fischer, Tramèr (2024). AgentDojo: a dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. NeurIPS 2024 Datasets and Benchmarks.
- Fisch, Maynez, Hofer, Dhingra, Globerson, Cohen (2024). Stratified prediction-powered inference for effective hybrid evaluation of language models. NeurIPS 2024. arXiv:2406.04291.
- Franklin, Schneeweiss, Polinski, Rassen (2014). Plasmode simulation for the evaluation of pharmacoepidemiologic methods in complex healthcare databases. *Computational Statistics & Data Analysis* 72.
- Giguere, Metevier, Castro da Silva, Brun, Thomas, Niekum (2022). Fairness guarantees under demographic shift. ICLR 2022.
- Gligorić, Zrnic, Lee, Candès, Jurafsky (2025). Can unconfident LLM annotations be used for confident conclusions? NAACL 2025. arXiv:2408.15204.
- Granite Team, IBM (2025). Granite-3.3-2B-Instruct. Model card, released 16 April 2025. https://huggingface.co/ibm-granite/granite-3.3-2b-instruct
- Hall (1988). Theoretical comparison of bootstrap confidence intervals. *Annals of Statistics* 16(3).
- Hall (1992). The bootstrap and Edgeworth expansion. Springer.
- Han, Rao, Ettinger, Jiang, Lin, Lambert, Choi, Dziri (2024). WildGuard: open one-stop moderation tools for safety risks, jailbreaks, and refusals of LLMs. NeurIPS 2024 Datasets and Benchmarks Track. arXiv:2406.18495.
- Higgins, Rivest, Stark (2011). Sharper p-values for stratified election audits. *Statistics, Politics, and Policy* 2(1).
- Hoeffding (1956). On the distribution of the number of successes in independent trials. *Annals of Mathematical Statistics* 27(3).
- Hoeffding (1963). Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association* 58(301).
- Hu, Shen, Wallis, Allen-Zhu, Li, Wang, Wang, Chen (2022). LoRA: low-rank adaptation of large language models. ICLR 2022. arXiv:2106.09685.
- Jones, Tong, Mu, Mahfoud, Leike, Grosse and 4 others (2025). Forecasting rare language model behaviors. arXiv:2502.16797.
- Khosravi, Huo (2026). Conformal selective acting: anytime-valid risk control for RLVR-trained LLMs. arXiv:2605.20270.
- Kish (1965). Survey sampling. Wiley.
- Kossen, Farquhar, Gal, Rainforth (2021). Active testing: sample-efficient model evaluation. ICML 2021 (PMLR 139). arXiv:2103.05331.
- Learned-Miller, Thomas (2019). A new confidence interval for the mean of a bounded random variable. arXiv:1905.06208.
- Lipton, Wang, Smola (2018). Detecting and correcting for label shift with black box predictors. ICML 2018 (PMLR 80). arXiv:1802.03916.
- Machcha, Menon, Zou, Chan, Chooi (2026). Opus 5.5 on RoboDojo-RC Tier 1: higher scores at lower cost. Robocurve report, 23 September 2026. https://robocurve.org/opus-5-5-robodojo-rc-tier-1/
- Maia Polo, Weber, Choshen, Sun, Xu, Yurochkin (2024). tinyBenchmarks: evaluating LLMs with fewer examples. ICML 2024 (PMLR 235).
- Menzel (2021). Bootstrap with cluster-dependence in two or more dimensions. *Econometrica* 89(5).

- Metevier, Giguere, Brockman, Kobren, Brun, Brunskill, Thomas (2019). Offline contextual bandits with high probability fairness guarantees. NeurIPS 2019.
- Miettinen, Nurminen (1985). Comparative analysis of two rates. *Statistics in Medicine* 4(2).
- Miller (2024). Adding error bars to evals: a statistical approach to language model evaluations. arXiv:2411.00640.
- Newcombe (1998). Two-sided confidence intervals for the single proportion: comparison of seven methods. *Statistics in Medicine* 17(8).
- Neyman (1938). Contribution to the theory of sampling human populations. *Journal of the American Statistical Association* 33(201).
- Ottoboni, Stark, Lindeman, McBurnett (2018). Risk-limiting audits by stratified union-intersection tests of elections (SUITE). E-Vote-ID 2018, Lecture Notes in Computer Science, pp. 174–188.
- Owen (2007). The pigeonhole bootstrap. *Annals of Applied Statistics* 1(2).
- Platt (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, MIT Press.
- Rogan, Gladen (1978). Estimating prevalence from the results of a screening test. *American Journal of Epidemiology* 107(1).
- Röttger, Kirk, Vidgen, Attanasio, Bianchi, Hovy (2024). XSTest: a test suite for identifying exaggerated safety behaviours in large language models. NAACL 2024. arXiv:2308.01263.
- Saerens, Latinne, Decaestecker (2002). Adjusting the outputs of a classifier to new a priori probabilities: a simple procedure. *Neural Computation* 14(1).
- Särndal, Swensson, Wretman (1992). Model assisted survey sampling. Springer.
- Satterthwaite (1946). An approximate distribution of estimates of variance components. *Biometrics Bulletin* 2(6).
- Shao, Wang, Zhu, Xu, Song, Bi and 5 others (2024). DeepSeekMath: pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300.
- Souly, Lu, Bowen, Trinh, Hsieh, Pandey and 5 others (2024). A StrongREJECT for empty jailbreaks. NeurIPS 2024 Datasets and Benchmarks. arXiv:2402.10260.
- Spertus, Stark (2022). Sweeter than SUITE: supermartingale stratified union-intersection tests of elections. arXiv:2207.03379.
- Stark (2020). Sets of half-average nulls generate risk-limiting audits: SHANGRLA. Financial Cryptography and Data Security 2020 workshops, Lecture Notes in Computer Science, pp. 319–336. arXiv:1911.10035.
- The companion paper (by the same authors, 2026). Seldonian post-training of language models: certified constraints on judge-measured behaviour under reinforcement learning. Unpublished manuscript.
- Thomas, Theocharous, Ghavamzadeh (2015a). High-confidence off-policy evaluation. AAAI 2015, *Proceedings of the AAAI Conference on Artificial Intelligence* 29(1).
- Thomas, Theocharous, Ghavamzadeh (2015b). High confidence policy improvement. ICML 2015 (PMLR 37).
- Thomas, Castro da Silva, Barto, Giguere, Brun, Brunskill (2019). Preventing undesirable behavior of intelligent machines. *Science* 366(6468).
- Waudby-Smith, Ramdas (2024). Estimating means of bounded random variables by betting. *JRSS-B* 86(1).
- Wilson (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association* 22(158).

- Wu, Hilton (2025). Estimating the probabilities of rare outputs in language models. ICLR 2025. arXiv:2410.13211.
- Yan, Su (2010). Stratified Wilson and Newcombe confidence intervals for multiple binomial proportions. *Statistics in Biopharmaceutical Research* 2(3).
- Yang, Yang, Zhang and 39 others (Qwen team) (2024). Qwen2.5 technical report. arXiv:2412.15115.
- Zhao, Yuan, Huang and 40 others (2025). Qwen3Guard technical report. arXiv:2510.14276.
- Zheng, Chiang, Sheng, Zhuang, Wu, Zhuang and 7 others (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. NeurIPS 2023 Datasets and Benchmarks. arXiv:2306.05685.
- Zollo, Morrill, Deng, Snell, Pitassi, Zemel (2024). Prompt risk control: a rigorous framework for responsible deployment of large language models. ICLR 2024. arXiv:2311.13628.
- Zrnic (2024). A note on the prediction-powered bootstrap. arXiv:2405.18379.
- Zrnic, Candès (2024a). Active statistical inference. ICML 2024 (PMLR 235).
- Zrnic, Candès (2024b). Cross-prediction-powered inference. *Proceedings of the National Academy of Sciences* 121(15).


# A The StratPPI comparison in detail

**The implementation.** StratPPI's authors have released no code. Our implementation agrees with a second implementation of their Algorithm 1, which we wrote in its general form, to rounding error and reproduces the widths of their own simulation in eight of nine settings (0.231 against 0.222 in the ninth), and the counts of sections 8.1 and 8.2 are the same or worse under the conventions of two public libraries: `ppi_py`, the PPI++ authors' library, applied within strata, and GLIDE, a third party's implementation of StratPPI.

**Table A1.** Reference-rate strata at delta 0.05: largest miss over checkpoints, and ESS against a random split with a pooled Wilson bound. Five mid-rate labels, two safety-set sizes. The rates in the first column are those of the draws used here; section 8.1.1 quotes an earlier measurement's figures for the same labels, whose rates differ by a point, which are $\mathrm{ESS}_{\mathrm{est}}$ and not $\mathrm{ESS}_{\mathrm{truth}}$ (section 6.4), and whose headline values are at delta 0.10. For refusal of harmful prompts at $n_s$ 100 that is 5.13 at delta 0.10 and 4.79 at delta 0.05 from the estimate, against 4.74 here. The ESS here is $\mathrm{ESS}_{\mathrm{truth}}$, the last checkpoint's. An ESS beside a bold miss belongs to an arm that is over its level, and the random-split baseline is itself over for the 93% label at $n_s$ 100 (0.057).

| label (rate) | $n_s$ | `b1w` | StratPPI as published | StratPPI estimator, bootstrap-t limit |
|---|---|---|---|---|
| over-refusal (0.16) | 100 | 0.023; 2.39 | **0.066**; 4.11 | 0.042; 2.58 |
| over-refusal (0.16) | 200 | 0.024; 2.30 | 0.052; 3.46 | 0.042; 3.10 |
| over-refusal, pushed by the Lagrangian (0.18) | 100 | 0.024; 2.19 | 0.0558; 3.24 | 0.036; 2.43 |
| over-refusal, pushed (0.18) | 200 | 0.023; 2.13 | 0.045; 2.85 | 0.037; 2.49 |
| non-refusal, encoded prompts (0.09) | 100 | 0.034; 1.38 | **0.086**; 2.59 | 0.045; 1.43 |
| non-refusal, encoded prompts (0.09) | 200 | 0.027; 1.36 | **0.063**; 2.11 | 0.041; 1.59 |
| refusal, harmful prompts (0.65) | 100 | 0.047; 4.74 | **0.0564**; 6.30 | 0.033; 1.77 |
| refusal, harmful prompts (0.65) | 200 | 0.040; 5.19 | 0.045; 6.29 | 0.034; 5.35 |
| refusal, encoded prompts (0.93) | 100 | 0.054; 0.96 | 0.032; 0.78 | 0.031; 0.83 |
| refusal, encoded prompts (0.93) | 200 | 0.045; 1.12 | 0.026; 0.92 | 0.033; 1.03 |

**Reference-rate strata (section 8.1).** In Table A1, bold marks a miss more than two Monte Carlo standard errors over delta, which is 0.0562 here; the two cells on either side of it carry a

fourth digit. The predictor is constant in 5 to 7 of the 8 strata, and in those strata StratPPI is a stratified normal interval on the labels. At delta 0.1 StratPPI as published is over its level in 3 of 10 cells. At delta 0.05 it is also over in every one of the four rare-label cells, where `b1w` is over in none. The paper's own allocation rules do not repair the normal limit: 9 of 26 cells are over with the oracle rule and 11 with the heuristic. With a bootstrap-t limit the estimator is over its level in 6 of 26 cells under the oracle allocation and in 11 under the heuristic, against none of 26 under proportional allocation (where it returns no limit in up to 6% of draws; section 7.1). Under proportional allocation it is ahead of `b1w` in seven cells, by 3–35%, behind by 8% and 14% in the two at a 93% rate, and far behind in one (1.77 against 4.74). A sweep places that drop at a safety set of 100 with 8 or 16 strata; with 4 strata, or with a safety set of 200 at any stratum count, it is absent. Over the sweep's 13 label-checkpoint cells the median $\mathrm{ESS}_{\mathrm{truth}}$ is 1.77 against 2.19 for `b1w` at a safety set of 100 with 8 strata, and 2.65 against 1.88 and 2.63 against 1.83 with 4 strata (safety sets of 100 and 200) and 2.70 against 2.13 and 2.63 against 2.23 at a safety set of 200 with 8 or 16 strata, where the bootstrap-t StratPPI is the narrower. Redrawn with replacement (section 8.1.2, item 4), the median $\mathrm{ESS}_{\mathrm{truth}}$ over the ten cells of Table A1 is 2.16 for `b1w`, 1.92 for the bootstrap-t StratPPI limit and 1.99 for the stratified Wald-t limit; `b1w`'s at $n_s$ 100 with replacement and without is 2.29 and 2.39 for over-refusal and 4.73 and 4.74 for refusal of harmful requests, and on the 65% label the other two give 2.6 and 3.6 (Wald-t) and 1.55 and 5.47 (bootstrap-t StratPPI) at $n_s$ 100 and 200. A stratified Wald-t limit on the same strata also held in all ten cells (largest miss 0.036, median $\mathrm{ESS}_{\mathrm{truth}}$ 2.06). In our predictor comparison the predictor is the same reference rate that defines the strata; splitting the reference samples, four for the strata and four for the predictor, did not change the picture (median $\mathrm{ESS}_{\mathrm{truth}}$ 1.97 against 2.10).

**Judge-logit strata (section 8.2).** StratPPI as published is over its level in 26 of 28 cells under each library's conventions, in 28 of 28 with the paper's oracle allocation and in 24 with its heuristic. The cleanest evidence is the judge wording with no constant stratum: 13 of 14 cells over, with misses from 0.056 (the one cell that is not over) to 0.187. With a bootstrap-t limit and the oracle allocation the estimator is over its level in 14 of 28 cells, against none under proportional allocation.

The 0.90 of Table 4 is the worst of several cells with one cause. It is the rubric judge at a 5% rate, 1,000 labels, 5 strata, heuristic allocation. The heuristic gives a stratum a share in proportion to its weight times the square root of the mean of $f(1-f)$ plus the variance of $f$, with $f$ the judge's probability. This judge's probability is under 0.001 in four of the five strata, so the rule sends 992 labels to the fifth and the four keep 2 each, a floor that is ours (the paper gives none). Those four strata hold 19% of the positives, at rates of 1.0–1.5%, and their eight labels show no positive in 91% of draws. The estimate is then about a fifth too low and the strata contribute nothing to its estimated variance, so the limit misses in 90% of draws, under the normal limit and the bootstrap-t alike. The floor is not what fails: with 10 labels a stratum the same cell still misses in 59% of draws, and four other cells on this judge miss in 68% to 85%. Fisch et al. note, for one of their datasets, that the heuristic becomes too aggressive at larger n when the judge's confidence is not calibrated; this is that case, at a rare rate.

# B The mechanism behind section 9.2

Our account of the mechanism, which no experiment here isolates, is a change in the form of the positives. The reward pulled toward long helpful answers and the multiplier penalised the guard's refusal flag. After training, the responses the guard flags are long refusals with an explanatory, helpful-sounding body: by the guard and a surface pattern (a refusal opener and more than 40

words), their share of the guard's flags rose from 37% in the reference to 82% in the trained policy, and to 76% and 82% in the two further seeds. The rubric judge reads that body as an answer: of 93 guard-flagged refusals of seed 0 after training it called 90 answers. Human labels side with the guard. Of 18 such responses of the trained policy, the annotator labelled 12 refusals, 6 answers and none refuse-then-answer, and refuse-then-answer is 3.4% of the reference's responses against 0.3% of the trained policy's (section 10.3). WildGuard, a judge of a second family, sides with the guard too: it flags 95% of the guard-flagged responses with the pattern and 92% of those without (section 9.2) [B4]. WildGuard reads the pattern as the guard does; the compiled judge reads it as an answer, and did so before training as well (it flagged 10% of the reference's guard-flagged responses with the pattern and 25% of those without, canonical wording); after training it flagged 4%, 8% and 5% of those with the pattern in seeds 0, 1 and 2 (about 14 of 251 against 3 of 29, no detectable change) and 0%, 4% and 8% of the few without (10, 25 and 12; about 2 of 47 against 18 of 71), so training changed how many responses have the pattern and also how the judge reads the plain ones [B4]. We had first read these responses as a refusal sentence followed by the answer; the labels do not support that reading, and the surface pattern is not a label (on XSTest's completions for its safe prompts 56% of responses with it are human full refusals and 34% partial ones).

# C The design check of section 10.4

A check of the whole design on the guard's real scores, with synthetic labels drawn from the guard's logit at the rates of section 10.3's sheet in each band of the logit (a synthetic population with rates of 9.5% and 10.1%, a true difference of +0.006), puts the miss rates of the four limits at 0.001, 0.040, 0.045 and 0.042 for the pool rate against a level of 0.05, and their mean certified margins at 0.065, 0.037, 0.032 and 0.035. When the 490 prompts are redrawn each time, so that the claim is about their source, the four miss rates are 0.001, 0.051, 0.063 and 0.052 (Monte Carlo standard error 0.005); limit (b1) is not built for that case. The exact bound is about twice as wide as the approximate ones: with few discordant pairs in 300, exactness is the larger cost and the guard's gain is the smaller one. The synthetic labels disagree with the guard independently across the two policies, which weakens both the pairing and the guard, so these margins are on the wide side and those of Table 10 on the narrow side. The guard's gain in this check is about 1.4 in labels ($\mathrm{ESS}_{\mathrm{truth}}$, limit (b1) against (a')), against the $\mathrm{ESS}_{\mathrm{pred}}$ of 2.5 behind Table 10. In the guard's own terms, on the 3,920 guard-only responses per policy at 256 tokens, the refusal rate is 16.9% for the reference and 17.2% for the trained policy.

# D What was fixed in advance, and where

Where this paper says a rule or an expectation was fixed before its result, the table gives the record. A commit that was made, and pushed to a remote, before the result existed is a record of the order that we could not have written afterwards. Where a note and its results entered the repository in one commit, the order rests on the note's own statement, and the table says so.

| what was fixed | section | record |
|---|---|---|
| the first strata rule (ties kept together, cuts at quantiles, small strata merged) and the pre-flight's error criterion | 8.1 | a design note committed on 2026-09-27 (9*e*1*b*7*b*4), before the first results (1 afe 911, the same day). Neither was kept: the rule was replaced and the criterion failed. The truth used for coverage, fixed in the same note, was changed as well (section 8.1.1) |
| the expectations for the judge-assisted estimators (not the routing rule of Table 5, which came out of the results) | 8.2, 9.1, 9.4 | a note dated before the estimators were run, committed together with their results on 2026-09-30 (3450*d*8*c*): no independent timestamp |
| the reading of a failure at step 200 | 9.2 | a note dated while the scoring pass ran and before its output, committed together with the result on 2026-10-01 (3*f*89*e*98): no independent timestamp |
| the hypotheses for the robot benchmark, against which the result on joint effort is marked exploratory | 10.1 | a note written after the termination counts and 12 transcripts had been seen, committed together with the results on 2026-10-02 (50863*a*4): no independent timestamp |
| the two-way bounds: definitions, fallbacks, seed, sizes, reading and three expectations (Table D3) | 10.2 | a registration committed with its script on 2026-10-07 (cb 50927), before the script was run on the benchmark's tables. The commit reached the remote only with its results, 42 minutes later (56721*e*6), so the order rests on the local commit time |
| the confirmation pool: prompts, labels, class limits, checks, sizes, the GPU cap, eight predictions and their scoring code | 8.1 | a registration committed and pushed with the scripts and the prompts on 2026-10-08 (9 fb 817*d*), before any response was generated |
| the mid-rate pool: the mixture, labels, checks, sizes, the GPU cap, ten predictions and their scoring code | 8.1 | a registration committed and pushed with the scripts, the prompts and the re-mix analysis on 2026-10-08 (*d*56 feb 6), before any response on its prompts was generated. The predictions were written after the same analysis of a re-mix of the first two pools' responses, which loosened one (the bootstrap-t StratPPI limit may be over in one cell, by at most 0.01), added one (`b1w` within 0.01 of its level at a rate between 0.45 and 0.55) and set the floor of a third (the Wald-t limit's gain, at 80% of the re-mix's smaller value); the registration names the first two as changes and gives the third's origin. Two more differences from the first registration were settled with the re-mix in view: the prediction on the cap is for the Wald-t limit's gain and not `b1w`'s, and `b1w` with the sampled-pool term carries no prediction |
| the analysis of the human-label certificate | 10.4 | the rules and the analysis script, committed on 2026-10-04 (9*e*45371, *e*076974) before any label of this sample. They replaced a first version of the same day (*f*1582*e*5) after the labels of section 10.3 had shown a difference near zero. The treatment of a zero-variance resample in the approximate limits was amended once, on 2026-10-07 (*a*5 bd 04*b*), also before any label of this sample, with the design check unchanged. No label has been collected |
| the in-loop run and the costly-label test: environments or labels, designs, sizes, seeds, six predictions each and their scoring code | 7.2, 8.1.4 | a registration committed and pushed with the scripts on 2026-10-14 (bf 7*f*5 dc), before any run; the costly-label script had printed data shapes and timings only |

| what was fixed | section | record |
|---|---|---|
| the exact bound and the suite-blocked resampling on AgentDojo: bounds, schemes, replications, seeds, six predictions and their scoring code | 10.2 | committed and pushed with the script on 2026-10-14 ($1b1185f$), before any run; a smoke test had printed timings and key names only. The run started 19 hours later than planned, unchanged, because the job meant to start it never did |
| two more seeds of the constrained run: seeds, steps, scoring wordings, item ids, the carried-bound computation and four predictions | 9.2 | committed and pushed with the run script on 2026-10-14 (3703288), before either seed was trained. One forced deviation: the first attempt at seed 1 crashed at step 12, and both seeds then ran without a registered memory-allocator setting, which changes memory allocation only; nothing of the crashed attempt is used. The registration reserved a retry of a seed cut short for an explicit decision; the seed was retried without one |
| the second model family: model, reward, pools, generation, checks, GPU caps, six predictions and their scoring code | 8.1.3 | committed and pushed with the scripts and the prompts on 2026-10-14 (56 fbeda), before any training or any generation on its pool. The queue script's first pilot stopped on a fault in its own parser (an interpreter without the numerical library); the parser was fixed and the pilot rerun before the full run |
| exact stratified bounds and the chance of certifying: cells, arms, measures, eight predictions and their scoring code | 8.1.4 | committed and pushed with the script on 2026-10-16 ($e3326b1$), before any outcome on a registered cell was computed; the script had printed allocation sizes, two timings on made-up counts with their bound values, one unregistered toy cell, and Clopper-Pearson's exact miss on 3,000 made-up configurations of binomial sums. The step-spread sensitivity was written after the results and is not registered |
| AgentDojo against a fitted crossed model: the model, the fit, two schemes, the arms, five predictions and their scoring code | 10.2 | committed and pushed with the script and the fitted parameters on 2026-10-16 ($e3326b1$), before any bound was computed on a simulated table; the fit and a timing run had been seen |
| a second judge family and a second gold label: the model, items, four analyses, five predictions and their scoring code | 9.2 | committed and pushed with the scripts on 2026-10-16 ($e3326b1$), before any registered item was scored; a smoke test on four made-up items had been seen |
| the four limits of the human-label certificate from the existing sheet | 10.3 | a plan note written on 2026-10-19 before the limits were computed and after the sheet's rates were known; not committed before the computation: no independent timestamp, and not a test |
| the second model family trained on the guard's flag alone: the reward, the reused pool and strata, GPU caps, five predictions and their scoring code | 8.1.3 | committed and pushed with the scripts on 2026-10-16 ($e3326b1$), before any training step of the run; a 3-step pilot of the reward on training prompts had been seen (timing, memory, four training responses and the reward's spread). The registered reading took a failed check of the push's direction to mean the policy had not moved; the run moved it, and section 8.1.3 says so against that reading |

**The first strata rule and the pre-flight's criteria, in figures** (section 8.1.1). The registered rule (ties kept together, cuts at quantiles, small strata merged) collapsed to one stratum on three labels and to two or three on the others, because 73–97% of prompts have an 8-sample reference rate of exactly 0 or 1. On the labels at 9% and 17% it gave less than equal rank strata ($\mathrm{ESS}_{\mathrm{est}}$ 1.2

to 2.3 against 1.4 to 2.5 at delta 0.05), and on the 66% label `b1w` was over its level under it in 2 of 6 cells at delta 0.05 (0.073 and 0.092) and 4 of 6 at 0.10. Under the registered truth the stratified arm and the random-split baseline were both over their level on over-refusal with 4 strata (largest miss over checkpoints 0.20 for the stratified arm, 0.14 and 0.18 for the baseline, at delta 0.10). Of the pre-flight's two registered criteria, a median absolute error of at most 0.1 failed on real data (0.29) and passed on the bandit (0.02), and a Spearman of at least 0.8 holds under the strata rule adopted afterwards and fails under the registered one (0.71).

**The tally by content and by record.** Five of the 72 predictions say only that a bound with a finite-sample guarantee is at or under its level (Y6, Z1, Z2, X1 and W1, marked † below); they test the code, not a claim, and all five held. P5 and M9 pair such a half with a gain and are counted with the rest. Ten (M1 to M9, with M3 in two parts) were written after a dry run of the analysis, and three (T1 to T3) rest on a local commit time. All 18 failures are claims of substance: 17 registered by a push before the run and one (T3) on a local commit time; none of the ten written after the dry run failed [prediction split].

*Table D0. The registered predictions by what they say and how their order is recorded.*

| | **held** | **failed** | **untested** |
|---|---|---|---|
| exact bounds at their level (†), registration pushed before the run | 5 | 0 | 0 |
| any other claim, registration pushed before the run | 37 | 17 | 0 |
| any other claim, written after a dry run | 9 | 0 | 1 |
| any other claim, local commit time only | 2 | 1 | 0 |
| all | 53 | 18 | 1 |

*Table D1. The eighteen registered predictions of section 8.1 and their outcomes. "Large pool" is the redraw with replacement at 40,000 draws; "source" is the claim about the prompt source at 10,000 replications; a cell is one safety-set size (100 or 200) at one delta (0.05 or 0.10); gains are $\mathrm{ESS}_{\mathrm{truth}}$ at delta 0.05 against the pooled Wilson bound on a random draw. A label is mid-rate between 5% and 95%.*

| | **prediction, as registered** | **outcome** |
|---|---|---|
| | *First registration (9 fb 817*d*): three pools, refusal rates 12%, 82% and 98%* | |
| P1 | large pool: the Wald-t limit and the bootstrap-t StratPPI limit are over in no mid-rate cell | refuted: the bootstrap-t limit is over in 1 of 8 cells (0.105 at delta 0.10, at 82%); the Wald-t limit in 0 of 8 |
| P2a | large pool: `b1w` is over in no cell at a rate under 0.45 | kept: 0 of 4 cells at 12% (0 of 12 with the two rare labels) |
| P2b | large pool: `b1w` is over in at least one of its four cells on every mid-rate label above 0.55 | kept: 4 of 4 at 82% |
| P3 | the gain of `b1w` with a large pool is within 20% of the pre-flight's prediction on every mid-rate label | kept: 1.53 and 1.62 against 1.82 at 12%; 1.49 and 1.62 against 1.65 at 82% |
| P4a | source: `b1w` with the sampled-pool term is over in no mid-rate cell at delta 0.05, and the Wald-t limit with the term in none at either delta | refuted: `b1w` is over in 1 of 4 cells (0.058 at 82%, safety set 100); the Wald-t limit in 0 of 8 |
| P4b | source: `b1w` without the term and the bootstrap-t StratPPI limit are each over in at least a third of the mid-rate cells at delta 0.05 | kept: 3 of 4 and 4 of 4 |
| P4c | source: the gain of `b1w` with the term is within 15% of the cap on every mid-rate label | kept: 1.32 and 1.22 against 1.35 and 1.24 at 12%; 1.22 and 1.20 against 1.33 and 1.24 at 82% |
| P5 | a rare label: the strata gain nothing (ESS of `b1w` between 0.9 and 1.15) and Clopper-Pearson on a random draw is over in no cell | kept: 1.01 to 1.11; 0 of 8 cells |

| | prediction, as registered | outcome |
|---|---|---|
| | *Second registration (*d*56 feb 6), written after the re-mix: one mixed pool, refusal rate 46%* | |
| M1 | large pool: the Wald-t limit is over in no cell | kept: 0 of 4 (misses 0.026 and 0.036 at delta 0.05, 0.063 and 0.083 at 0.10) |
| M2 | large pool: the bootstrap-t StratPPI limit is over in at most one of four cells, and above its level by at most 0.01 in any | kept: 0 over, 3 unresolved, largest excess 0.001 |
| M3a | large pool: `b1w` is over in no cell at a rate under 0.45, and in at least one of four above 0.55 | not tested: the rate, 0.459, lies between |
| M3b | at a rate between 0.45 and 0.55, the miss of `b1w` is within 0.01 of its level in every cell | kept: 0.044 and 0.045 at delta 0.05, 0.096 at 0.10 |
| M4 | the gain of `b1w` with a large pool is within 20% of the pre-flight's prediction | kept: 2.78 and 2.97 against 2.79 |
| M5 | the gain of the Wald-t limit with a large pool is at least 1.7 | kept: 2.09 and 2.56 |
| M6 | source: the Wald-t limit with the term is over in no cell | kept: 0 of 4 |
| M7 | source: `b1w` without the term and the bootstrap-t StratPPI limit are each over in at least one of the two cells at delta 0.05 | kept: 2 of 2 each (misses 0.079 to 0.120) |
| M8 | source: the gain of the Wald-t limit with the term is above 1 and within 15% of the cap | kept: 1.49 and 1.34 against 1.64 and 1.44 |
| M9 | a rare label: ESS of `b1w` between 0.9 and 1.15, and Clopper-Pearson over in no cell | kept: 1.00 and 1.08; 0 of 4 cells |

*Table D2. The 28 predictions registered on 2026-10-14 and their outcomes, each scored by its script as committed (S1 to S4 read from the committed `transfer017.py` output by the registered over rule). A cell is over by section 6.1's rule; gains are against the pooled Wilson bound on a random draw at delta 0.05 unless stated.*

| | prediction, as registered | outcome |
|---|---|---|
| | *The training loop (section 7.2): synthetic bandit, 8 strata, four environments, 8,000 runs a cell* | |
| Q1 | the Wald-t limit, for the pool's rate: over in at most 1 of its 8 cells | kept: 0 of 8 (2 unresolved) |
| Q2 | the Wald-t limit with the sampled-pool term, for the population's rate: over in at most 2 of its 8 cells | kept: 2 of 8 |
| Q3 | the same in the least heterogeneous environment at delta 0.10: over | kept: 0.107 |
| Q4 | the pooled Wilson bound on a random split, for the population's rate: over in at most 2 of its 8 cells | kept: 0 of 8 (2 unresolved) |
| Q5 | the $\mathrm{ESS}_{\mathrm{est}}$ of the Wald-t limit for the population's rate at delta 0.10 is at least 1.6 in the most heterogeneous environment | kept: 1.68 |
| Q6 | the same in the least heterogeneous environment is between 0.9 and 1.15 | kept: 1.10 |
| | *A costly label (section 8.1.4): XSTest's human labels of five released models on 250 safe prompts* | |
| Y1 | the Wald-t limit, labels at a rate of 10% or more, safety set 100, with replacement: over in no cell | refuted: 3 of 20 cells, by at most 0.008 |
| Y2 | the Wald-t limit, mid-rate labels, without replacement: over in no cell | kept: 0 of 64 |
| Y3 | the Granite reference-rate stratifier, safety set 100, with replacement: the median gain of the Wald-t limit over the mid-rate labels is at least 1.1 | kept: 1.18 |

| | prediction, as registered | outcome |
|---|---|---|
| Y4 | the same: no mid-rate label gains more than 2.0 | kept: at most 1.34 |
| Y5 | the stratifier from the other model families' completions gains more than the Granite one for at least 6 of the 8 mid-rate labels | refuted: 4 of 8 |
| Y6† | Clopper-Pearson on a random draw, rare labels, with replacement: over in no cell<br>*AgentDojo (section 10.2): 28 pipelines, 4,000 resampled tables a pipeline and scheme* | kept: no cell over |
| Z1† | the betting bound on user-task rates, against their average, is over for no pipeline with user tasks redrawn, across the table or within suites | kept: none under either |
| Z2† | the pair-rate betting bound is over for no pipeline with user tasks redrawn within suites | kept: none |
| Z3 | the bound clustered by user task is over for no pipeline with user tasks redrawn within suites | kept: none |
| Z4 | the same bound is over for at least 15 of 28 pipelines with injection tasks redrawn within suites | refuted: 11 |
| Z5 | the quadrature bound is over for at most 2 of 28 pipelines under each scheme within suites | kept: none under any |
| Z6 | on the published tables the betting bound on user-task rates certifies 5% for at most one pipeline, and the pair-rate one for none<br>*A second model family (section 8.1.3): Qwen2.5-1.5B-Instruct, a new pool of 400 prompts* | kept: one (0.048) and none |
| V1 | at step 200 rho on the refusal label is under 0.8 | refuted: 0.84 |
| V2 | at step 200 the refusal rate rose on the categories the reward pushed up and fell on those it pushed down | refuted: 16.9% to 14.8%, and 22.5% to 23.5% |
| V3 | at step 200, at a refusal rate between 5% and 95%, the Wald-t limit with a large pool is over in no cell | kept: 0 of 4 at 19% |
| V4 | the Wald-t limit's gain with a large pool (safety set 200) is lower at step 200 than at step 100 | kept: 1.77 against 1.84 |
| V5 | at step 200 the gain of `b1w` with a large pool (safety set 200) is within 20% of the pre-flight at the measured moderators | kept: 1.70 against 1.73 |
| V6 | at step 200 the Wald-t limit's gain with a large pool (safety set 200) is between 0.9 and 1.6<br>*Two more seeds of the constrained run (section 9.2), step 200 against step 0* | refuted: 1.77 |
| S1 | in at least one of the two new seeds the carried bound is over its level for at least one of the six wordings | kept |
| S2 | the same in both new seeds | kept: 3 and 2 of 6 |
| S3 | in both new seeds the canonical wording's recall on the guard's refusals is lower at step 200 than at step 0 | kept: 0.21 to 0.07 and to 0.05 |
| S4 | over the three seeds together, at least 6 of the 18 carried bounds are over | kept: 8 of 18 |

*Table D3. The three expectations of the two-way-bounds registration (cb 50927), section 10.2, scheme (c).*

| | expectation, as registered | outcome |
|---|---|---|
| T1 | the multiway variance with a t quantile is over for some pipelines | kept: 26 of 28 |
| T2 | the pigeonhole bootstrap-t is over for fewer pipelines than it | kept: none |
| T3 | the quadrature bound is over for none | refuted: one (0.058) |

*Table D4. The predictions registered on 2026-10-16 ($e3326b1$) and their outcomes, each scored

by its script as committed. The tags are the registrations'; Q1 to Q5 here are the second-judge registration's, not Table D2's. A power ESS is the number of random labels Clopper-Pearson needs to certify as often, over the labels the bound uses (section 8.1.4); "in range" is a rate from 5% to 95%.*

| | **prediction, as registered** | **outcome** |
|---|---|---|
| | *Exact stratified bounds and the chance of certifying (section 8.1.4): 116 cells, delta 0.05 and 0.10 for X1 and X2, delta 0.05 for X3 to X8* | |
| X1† | neither exact stratified bound is over in any cell | kept: 0 of 464 |
| X2 | the betting bound's width gain against Clopper-Pearson is below 1 in every cell | kept: at most 0.53 |
| X3 | the betting bound's power ESS at a pass probability of 0.8 is below 1 in every real and costly cell at a rate of 5% or more | kept: at most 0.28 |
| X4 | the stratified Clopper-Pearson bound's power ESS at 0.8, real and costly cells in range: lowest at least 0.95, median at least 1.10 | refuted: 0.27 and 0.92 |
| X5 | the same at a pass probability of 0.5 lies between 0.85 and 1.15 in every one | refuted: 0.07 to 1.09 |
| X6 | the same at 0.8 on rare real and costly cells lies between 0.8 and 1.2 | refuted: 0.33 to 1.01 |
| X7 | real cells in range: the Wald-t limit's width gain against Clopper-Pearson is at least its gain against the pooled Wilson bound in every one | kept: 36 of 36 |
| X8 | the same cells: the Wald-t limit's power ESS at 0.8 is above the stratified Clopper-Pearson bound's in every one | refuted: 35 of 36 |
| | *AgentDojo against a fitted crossed model (section 10.2): 28 pipelines, 4,000 tables a pipeline and scheme* | |
| W1† | injection tasks fixed: the betting bound on user-task rates (against their average) and the pair-rate betting bound are over for no pipeline | kept: none and none |
| W2 | the same scheme: the bound clustered by user task is over for at most 2 | kept: none |
| W3 | both kinds of task drawn: the bound clustered by user task is over for at least 8 | kept: 18 |
| W4 | the same scheme: the quadrature bound is over for at most 2 | kept: none |
| W5 | the same scheme: the betting bound on user-task rates is over for at least 2 | kept: 5 |
| | *A second judge family and a second gold label (section 9.2): WildGuard* | |
| Q1 | WildGuard as the judge, the guard as gold: the carried bound at step 200 is over for at least 1 of the 3 seeds | refuted: none (misses 0.000) |
| Q2 | the same: WildGuard's recall on the guard's refusals is lower at step 200 than at step 0 in all 3 seeds | refuted: higher in all 3 (0.88 to 0.94, 0.94 and 0.91) |
| Q3 | WildGuard as gold, the compiled judge's canonical wording as the judge: the carried bound at step 200 is over for at most 1 of the 3 seeds | refuted: all 3 (0.84, 0.46, 0.66) |
| Q4 | on the refusal sheet at step 200, WildGuard's design-weighted accuracy against the human strict label is higher than the guard's | kept: 0.930 against 0.929 |
| Q5 | among the guard's flags, WildGuard flags a smaller share of responses with the surface pattern than without (Fisher $p < 0.05$) | refuted: 95% against 92% (p 0.064) |

| | prediction, as registered | outcome |
|---|---|---|
| | *The second model family trained on the guard's flag alone (section 8.1.3): the same pool and strata* | |
| U1 | at step 200 refusal on the categories pushed up is at least 15 points above the reference, and on those pushed down at least 10 points below it | refuted: 16.9% to 67.4%, and 22.5% to 48.7% |
| U2 | at step 200 rho on the refusal label is under 0.7 | kept: 0.46 |
| U3 | at step 200, at a refusal rate between 5% and 95%, the Wald-t limit with a large pool is over in no cell | kept: 0 of 4 at 58% |
| U4 | at step 200 the Wald-t limit's gain with a large pool (safety set 200, delta 0.05) is below 1.3 | kept: 1.08 |
| U5 | at step 200 the gain of `b1w` with a large pool (safety set 200) is within 25% of the pre-flight at the measured moderators | kept: 1.22 against 1.14 |

# E Reproducing the numbers

The numbers in this paper come from the scripts below and from the scripts of three earlier measurements kept with their results (the carried calibration and the sheets of sections 9.1, 9.2, 9.4 and 9.5; the robot benchmark's counts of section 10.1; the design effects of section 9.3), apart from those cited from the companion paper and a few recomputed from the data files. The scripts that sample or judge responses need a GPU. The working draft carries a source tag on each number and a table that resolves each tag to a file.

| what | script | output |
|---|---|---|
| Table 4, every validity cell under one rule; the table and the counts in the text asserted against the files | `scripts/validity_recount.py` | `results/paper/validity_recount.md` |
| section 7.1, the reference-rate-strata cells redrawn with replacement | `scripts/replacement_check.py` | `results/paper/replacement_check.md` |
| section 7.1, the exact miss probability of the Wilson limit | `scripts/wilson_exact.py` | `results/paper/wilson_exact.md` |
| Table 4, the labels-alone rows by enumeration | `scripts/binomial_rows.py` | `results/paper/binomial_rows.md` |
| Table 4, the four judge-plasmode cells with 20,000 unlabelled responses, at 4,000 draws | `scripts/plasmode017_big.py` | `results/paper/plasmode017_big.json` |
| section 8.1, the strata for a claim about the prompt source | `scripts/twophase_check.py` | `results/paper/twophase_check.md` |
| section 8.1, the confirmation on new prompts (registered) | `scripts/confirm_pool.py` | `results/paper/confirm/confirm.md` |
| section 8.1, the mid-rate pool (registered) | `scripts/confirm_mid.py` | `results/paper/confirm/mid/confirm.md` |
| end-point tests of every bound used | `tests/test_bound_endpoints.py` | run with pytest |
| sections 8.1 and 8.2, the StratPPI comparison | `scripts/stratppi_baseline.py` | `results/paper/stratppi.md` |
| sections 8.1 and 8.2, implementation check, allocations, PPBoot, sweeps | `scripts/stratppi_validate.py` | `results/paper/stratppi_validate.md` |
| section 9.2, the judge against human labels | `scripts/judge_on_labels.py` | `results/labels/refusal/rubric_vs_human.md` |
| Appendix A, the heuristic allocation in its worst cell | `scripts/stratppi_heur_cell.py` | `results/paper/stratppi_heur_cell.md` |
| section 10.1, the robot benchmark's sampling unit | `scripts/robodojo_sampling.py` | `results/paper/robodojo_sampling.md` |

| what | script | output |
|---|---|---|
| section 10.2, AgentDojo | scripts/agentdojo_recheck.py | results/paper/agentdojo_recheck.md |
| section 10.2, the two-way bounds (registered) | scripts/agentdojo_twoway.py | results/paper/agentdojo_twoway.md |
| section 10.3, the guard on XSTest | scripts/xstest_guard.py | results/labels/xstest/analysis.md |
| section 10.3, the refusal sheet | scripts/refusal_sheet_build.py, scripts/refusal_labels.py | results/labels/refusal/analysis.md |
| section 10.4, the label budget | scripts/p9_budget.py | results/paper/p9_budget.md |
| section 10.4, the prepared certificate and its design check | scripts/p9_sample.py, scripts/p9_certificate.py | results/labels/p9/design_check.md |
| figures | scripts/paper_figures.py | reports/figs/ |
| references checked against arXiv and CrossRef | scripts/bib_check.py | printed report |

Results cited from the companion paper on training (parts of section 7.2) are printed rates whose per-draw data were lost; they cannot be regenerated without retraining. The rows of Table 4 on a trained-policy harm pool are the exception: they are reproduced by enumeration.